



Université Cadi Ayyad

École Nationale des Sciences Appliquées de Marrakech

Filière : Génie Cyber Défense et Systèmes de Télécommunications Embarqués

Projet réalisé dans le cadre de la formation :

École Nationale des Sciences Appliquées de Marrakech

RAPPORT DE PROJET

Landing zone AWS sécurisée et automatisée : du réseau au DevSecOps avec Terraform

---

Encadré par :

Pr. ACHBAROU Omar

Réalisé par :

CHAGDALY Hiba

Année Universitaire : 2025/2026

Dédicace

À ceux qui ont éclairé mon chemin.

Je dédie ce travail à mes parents, pour leur amour inconditionnel, leur patience et leurs sacrifices silencieux. Leur confiance a toujours été pour moi une source précieuse de force et de motivation.

À ma famille, pour son soutien constant, ses encouragements et ses prières tout au long de ce parcours.

À mes enseignants et encadrants, pour la transmission de leur savoir, leur exigence et leur bienveillance.

À moi-même, pour avoir su transformer un imprévu en opportunité. Ce projet n'est pas né d'un stage classique, mais d'une décision personnelle de poursuivre l'apprentissage et de me dépasser, seule face aux défis techniques, avec la conviction que la persévérance vaut toutes les circonstances idéales.

À mes amis sincères, pour leur présence dans les moments de doute et pour les instants de partage qui ont rendu ce parcours plus riche.

Que ce rapport soit un humble témoignage de la reconnaissance que je leur porte.

Remerciements

Je tiens à exprimer ma sincère gratitude à toutes les personnes qui ont contribué à la réalisation de ce travail.

Je remercie tout d'abord Dieu de m'avoir accordé la force, la patience et la persévérance nécessaires pour mener ce projet à son terme.

J'adresse mes plus sincères remerciements à mon encadrant académique, **Pr. Omar Achbarou**, chef de la filière Génie Cyber Défense et Systèmes de Télécommunications Embarqués (GCD-STE), pour avoir accepté d'encadrer ce projet auto-dirigé, pour sa confiance, sa disponibilité et pour ses conseils avisés tout au long de ce travail.

Je souhaite également exprimer ma reconnaissance envers l'ensemble des membres du jury, qui me font l'honneur d'évaluer ce rapport et de porter un intérêt à ce projet.

Je remercie l'ensemble du corps professoral de l'ENSA Marrakech pour la qualité de l'enseignement dispensé tout au long de mon parcours, ainsi que le personnel administratif pour son soutien.

J'adresse une pensée particulière à mes camarades de promotion et à mes amis pour leur soutien moral, leur entraide et les moments de partage qui ont enrichi ces années d'études.

Enfin, je remercie profondément mes parents et ma famille pour leur amour inconditionnel, leur patience et leur soutien indéfectible. Ce travail n'aurait pas été possible sans leurs sacrifices et leur confiance.

Acknowledgements

I would like to express my sincere gratitude to all those who contributed to the completion of this work.

I thank God for the strength, patience and perseverance required to carry this project through to completion.

I wish to express my deepest thanks to my academic supervisor, **Professor Omar Achbarou**, Head of the Cyber Defence Engineering and Embedded Telecommunications Systems (GCD-STE) programme, for agreeing to supervise this self-directed project, for his trust and availability, and for his valuable guidance throughout this work.

I am also grateful to the members of the examination board for the honour of evaluating this report and for the interest they have shown in this project.

I thank the academic staff of ENSA Marrakech for the quality of the teaching provided throughout my studies, as well as the administrative staff for their support.

I would like to thank my classmates and friends for their moral support, their collaboration and the shared experiences that have enriched these years of study.

Finally, I wish to express my profound gratitude to my parents and my family for their unconditional love, their patience and their unwavering support. This work would not have been possible without their sacrifices and their trust.

Résumé

Le présent rapport expose la conception et la mise en œuvre d'une *landing zone* AWS sécurisée, réalisée dans le cadre d'un projet auto-dirigé de fin d'études en filière GCDSTE à l'ENSA Marrakech. Le travail s'appuie sur le scénario d'Atlas FinTech, entreprise fictive marocaine de traitement de paiements, confrontée à l'absence d'une fondation cloud et à l'exigence d'une infrastructure auditable et maîtrisée. Il s'aligne sur les objectifs de la certification AWS Solutions Architect Associate (SAA-C03).

Le problème traité est la construction d'une base organisationnelle et technique reproductible. Une configuration manuelle fragilise la sécurité et compromet la traçabilité. L'objectif est une architecture industrialisée, décrite en Infrastructure as Code avec Terraform, en s'appuyant sur les référentiels AWS Well-Architected, CIS AWS Benchmarks et AWS FSBP, la sécurité étant une contrainte de conception dès la première phase.

La démarche s'organise en onze phases. Un bootstrap pose le socle : séparation des identités, backend d'état S3 verrouillé par DynamoDB, garde-fous budgétaires. L'architecture s'étend à une organisation multi-comptes, un réseau segmenté à trois niveaux, une couche de calcul sur ECS Fargate exposée par un ALB, un durcissement opérationnel (KMS, GuardDuty, Security Hub, AWS Config), l'observabilité par CloudWatch et SNS, un pipeline DevSecOps (GitHub Actions via OIDC, scan d'images) et une gouvernance FinOps. Le stockage S3, un CDN CloudFront et un découplage des alertes (GuardDuty $\rightarrow$ EventBridge $\rightarrow$ SQS $\rightarrow$ Lambda $\rightarrow$ DLQ) complètent la fondation. Les apports sont une architecture reproductible et l'intégration de contrôles de sécurité automatisés dans le cycle de déploiement.

Mots-clés : AWS, landing zone, Terraform, IaC, DevSecOps, CI/CD, GitHub Actions, IAM, multi-comptes, ECS Fargate, CloudFront, GuardDuty, Security Hub, FinOps.

Abstract

This report presents the design and implementation of a secure AWS *landing zone*, carried out as a self-directed final-year project in the GCDSTE program at ENSA Marrakech. The work is based on the scenario of Atlas FinTech, a fictitious Moroccan payment processing company facing the absence of a cloud foundation and the need for an auditable and cost-controlled infrastructure. It also aligns with the objectives of the AWS Solutions Architect Associate (SAA-C03) certification.

The problem addressed is the construction of a reproducible organizational and technical foundation. Manual configuration weakens security and compromises traceability. The objective is an industrialized architecture, described in Infrastructure as Code with Terraform, drawing on the AWS Well-Architected Framework, CIS AWS Benchmarks, and AWS FSBP guidelines, with security enforced as a design constraint from the very first phase.

The approach unfolds across eleven phases. A bootstrap lays the foundation : identity separation, an S3 state backend locked by DynamoDB, and budgetary guardrails. The architecture then extends to a multi-account organization, a three-tier segmented network, a compute layer on ECS Fargate exposed through an ALB, operational hardening (KMS, GuardDuty, Security Hub, AWS Config), observability via CloudWatch and SNS, a DevSecOps pipeline (GitHub Actions via OIDC, image scanning), and FinOps governance. S3 object storage, a CloudFront CDN, and a security alert decoupling mechanism (GuardDuty $\rightarrow$ EventBridge $\rightarrow$ SQS $\rightarrow$ Lambda $\rightarrow$ DLQ) complete the foundation. The main contributions are a reproducible architecture and the integration of automated security controls into the deployment cycle.

Keywords : AWS, landing zone, Terraform, IaC, DevSecOps, CI/CD, GitHub Actions, IAM, multi-account, ECS Fargate, CloudFront, GuardDuty, Security Hub, FinOps.

Table des matières

Dédicace i

Remerciements ii

Acknowledgements iii

Résumé iv

Abstract v

Table des matières xi

Liste des figures xv

Liste des tableaux xvi

Liste des listings xvii

Liste des acronymes xviii

Introduction générale 1

1 Contexte du projet : le cas Atlas FinTech 3

1.1 Présentation du cas d'usage 3

1.1.1 Le cas Atlas FinTech 3

1.1.2 Enjeux de sécurité et de gouvernance 4

1.1.3 Périmètre du projet 6

1.2 État de l'art des fondations cloud sur AWS 7

1.2.1 Référentiels et cadres méthodologiques 7

1.2.2 Solutions clé-en-main 7

1.2.3 Positionnement du projet 8
Conclusion du chapitre 9

2 Étude préalable et analyse des besoins 11
2.1 Besoins fonctionnels et non fonctionnels 11
2.1.1 Besoins fonctionnels 11
2.1.2 Besoins non fonctionnels 12
2.1.3 Synthèse 13
2.2 Identification des acteurs 13
2.2.1 Acteurs humains 13
2.2.2 Acteurs techniques 14
2.2.3 Synthèse des acteurs 14
2.2.4 Diagramme de contexte 14
2.3 Cas d'utilisation 15
2.3.1 Recensement des cas d'utilisation 16
2.3.2 Diagramme de cas d'utilisation 16
2.3.3 Description des cas critiques 16
2.4 Contraintes du projet 17
2.4.1 Contraintes techniques 18
2.4.2 Contraintes réglementaires et de conformité 18
2.4.3 Contraintes budgétaires et organisationnelles 19
2.5 Priorisation des besoins 19
2.6 Analyse des risques 20
2.6.1 Identification des risques principaux 20
2.6.2 Matrice probabilité / impact 21
2.6.3 Matrice de traçabilité risques → mitigations 21
Conclusion du chapitre 22

3 Fondements des infrastructures cloud sécurisées 24
3.1 Le Cloud Computing 24
3.1.1 Du modèle traditionnel au modèle Cloud 24
3.1.2 Avantages et limites du Cloud Computing 25

3.1.3 Architecture et caractéristiques du Cloud Computing 25
3.1.4 Modèles de services Cloud 26
3.1.5 Modèles de déploiement du Cloud 27
3.1.6 Modèle de responsabilité partagée et sécurité du Cloud 28
3.1.7 Cas d'utilisation du Cloud Computing 29
3.2 Gouvernance et architecture cloud multi-comptes 29
3.2.1 Principe de la segmentation par comptes (landing zone) 29
3.2.2 Organisation hiérarchique et unités organisationnelles 30
3.2.3 Politiques organisationnelles et délégation administrative 30
3.2.4 Gestion centralisée des identités et principe du moindre privilège 31
3.3 Sécurité des architectures cloud 31
3.3.1 Défense en profondeur et isolation réseau 31
3.3.2 Segmentation réseau et contrôle du trafic 32
3.3.3 Chiffrement des données au repos et en transit 32
3.3.4 Traçabilité, audit et détection des menaces 32
3.4 Conteneurisation et architectures distribuées 33
3.4.1 Principes de la conteneurisation 33
3.4.2 Orchestration de conteneurs et modèles d'exécution 33
3.4.3 Scalabilité automatique (auto-scaling) 33
3.4.4 Répartition de charge et haute disponibilité 33
3.5 Observabilité des systèmes distribués 34
3.5.1 Les trois piliers de l'observabilité 34
3.5.2 Alerting, notification et supervision proactive 34
3.5.3 Tableaux de bord et visualisation 35
3.6 DevSecOps et intégration continue 35
3.6.1 Principe du *Shift Left Security* 35
3.6.2 Automatisation des pipelines de déploiement 35
3.6.3 Authentification fédérée et gestion des secrets 36
3.6.4 Analyse de vulnérabilités et contrôle qualité 36
3.7 Architectures événementielles et découplage 36
3.7.1 Principe du découplage producteur/consommateur 36

3.7.2 Files de messages et gestion des échecs . . . 37
3.8 Distribution de contenu et performance . . . 37
3.8.1 Réseaux de diffusion de contenu (CDN) . . . 37
3.8.2 Mise en cache et réduction de latence . . . 38
3.9 Résilience et continuité d'activité . . . 38
3.9.1 Stratégies de reprise après sinistre . . . 38
3.9.2 Haute disponibilité multi-zone et multi-région . . . 39
3.10 Gestion financière du cloud (FinOps) . . . 40
3.10.1 Modèles de tarification cloud . . . 40
3.10.2 Gouvernance des coûts et visibilité budgétaire . . . 40
3.11 Infrastructure as Code (IaC) . . . 40
3.11.1 De la gestion manuelle à l'automatisation déclarative . . . 40
3.11.2 Principes et avantages de l'IaC . . . 40
3.11.3 Gestion de l'état et collaboration . . . 41
3.12 Conclusion du chapitre . . . 41

4 Technologies et outils utilisés 43
4.1 Infrastructure as Code . . . 43
4.2 Gouvernance et identité . . . 44
4.3 Réseau et exposition . . . 45
4.4 Calcul et conteneurs . . . 46
4.5 Stockage et données . . . 47
4.6 Sécurité opérationnelle . . . 48
4.7 Observabilité et messagerie . . . 49
4.8 Outils DevSecOps . . . 50
4.9 Synthèse . . . 51
Conclusion du chapitre . . . 51

5 Conception de la landing zone Atlas FinTech 53
5.1 Vision globale et principe de conception en couches . . . 53
5.2 Gouvernance et organisation multi-comptes . . . 55
5.2.1 Choix de l'unité d'isolation : le compte AWS . . . 55

5.2.2 Encadrement par Service Control Policies . . . 56
5.2.3 Fédération d'identité humaine via IAM Identity Center . . . 56
5.3 Architecture réseau . . . 57
5.3.1 Structure du VPC et segmentation en trois niveaux . . . 57
5.3.2 Multi-AZ et haute disponibilité . . . 58
5.3.3 Contrôle des flux par Security Groups . . . 58
5.4 Architecture applicative . . . 59
5.4.1 Choix de Fargate contre EC2 et Kubernetes . . . 59
5.4.2 Deux rôles IAM distincts pour ECS . . . 60
5.4.3 Chiffrement au repos et en transit . . . 60
5.4.4 Distribution CloudFront et OAC . . . 61
5.5 Pipeline DevSecOps . . . 61
5.5.1 Fédération OIDC avec GitHub Actions . . . 61
5.5.2 Security gate Trivy bloquant . . . 62
5.6 Sécurité opérationnelle . . . 63
5.6.1 Journalisation d'audit centralisée . . . 63
5.6.2 Délégation de GuardDuty et Security Hub . . . 64
5.6.3 Découplage GuardDuty → SQS → Lambda . . . 64
5.7 Traçabilité besoins → conception . . . 65
Conclusion du chapitre . . . 65

6 Mise en œuvre de la fondation cloud Atlas FinTech 68
6.1 Préparation de l'environnement de travail . . . 69
6.1.1 Phase 00 – Bootstrap du compte AWS . . . 69
6.1.2 Phase 01 – Backend Terraform . . . 71
6.2 Fondations de la landing zone . . . 73
6.2.1 Phase 02 – Landing zone multi-comptes . . . 74
6.2.2 Phase 03 – Réseau à trois couches . . . 81
6.3 Plateforme applicative et sa maturation . . . 84
6.3.1 Phase 04 – Compute serverless (ECS Fargate et ALB) . . . 84
6.3.2 Phase 05 – Durcissement sécurité . . . 87

6.3.3 Phase 06 – Observabilité 91
6.3.4 Phase 07 – Pipeline d'intégration et de déploiement continus 94
6.4 Extensions et gouvernance des coûts 97
6.4.1 Phase 08 – Gouvernance des coûts 97
6.4.2 Phase 09 – Stockage objet 99
6.4.3 Phase 10 – Distribution de contenu à échelle mondiale 100
6.4.4 Phase 11 – Découplage asynchrone des événements de sécurité 104
Conclusion du chapitre 106

Conclusion générale **107**

Table des figures

Figure 1.1	Synthèse du scénario d'étude Atlas FinTech.	4
Figure 1.2	Les quatre enjeux structurants d'une fondation cloud : sécurité, isolation, audit et coût.	5
Figure 1.3	Déploiement manuel versus fondation gouvernée : l'objectif du projet.	6
Figure 1.4	Périmètre du projet : éléments inclus et exclus.	6
Figure 2.1	Diagramme de contexte de la solution Atlas FinTech	15
Figure 2.2	Diagramme de cas d'utilisation de la solution Atlas FinTech	16
Figure 2.3	Flux des deux cas d'utilisation critiques	17
Figure 2.4	Matrice probabilité / impact des risques identifiés	21
Figure 3.1	Cloud versus On-Premise	25
Figure 3.2	Modèles de services Cloud (IaaS, PaaS, SaaS)	27
Figure 3.3	Modèles de déploiement du Cloud	28
Figure 3.4	Modèle de responsabilité partagée dans le Cloud	29
Figure 3.5	Architecture de référence d'une landing zone multi-comptes	30
Figure 3.6	Principe de la défense en profondeur	31
Figure 3.7	Les trois piliers de l'observabilité	34
Figure 3.8	Pipeline CI/CD générique intégrant un contrôle de sécurité	35
Figure 3.9	Pattern de découplage	37
Figure 3.10	Architecture simplifiée d'un CDN	38
Figure 3.11	Comparaison des stratégies de reprise après sinistre selon le RTO/RPO	39
Figure 5.1	Vue d'ensemble de l'architecture Atlas FinTech en cinq couches.	54
Figure 5.2	Structure organisationnelle et flux d'authentification humaine.	57
Figure 5.3	Architecture réseau du VPC Atlas FinTech en trois niveaux.	59
Figure 5.4	Pipeline DevSecOps : de la modification du code au déploiement ECS.	63

Figure 5.5 Flux de traitement asynchrone des findings GuardDuty. 65

Figure 6.1 Utilisateur IAM `terraform-deployer` créé sans accès console, destiné exclusivement à l'automatisation. 70

Figure 6.2 Alerte budgétaire configurée à 85 % du seuil de 50 USD, avec notification par courrier électronique. 70

Figure 6.3 Table DynamoDB `atlas-fintech-tfstate-lock` active, prête à sérialiser les opérations Terraform concurrentes. 72

Figure 6.4 Sortie de `terraform state list` après migration réussie de l'état local vers le backend S3. 73

Figure 6.5 Hiérarchie d'unités organisationnelles créée dans AWS Organizations. 78

Figure 6.6 Les huit SCPs déployées via le module réutilisable, dont six attachées à la racine de l'organisation. 79

Figure 6.7 Portail SSO donnant accès aux comptes de l'organisation après authentification MFA. 79

Figure 6.8 Constatations GuardDuty réelles remontées sur le compte de gestion (usage de l'utilisateur *root*), matérialisant la détection effective en conditions de production. 80

Figure 6.9 Standards de conformité activés dans Security Hub à l'échelle organisationnelle. 80

Figure 6.10 VPC `atlas-fintech-vpc` déployé avec le bloc CIDR `10.0.0.0/16`. 82

Figure 6.11 Les neuf sous-réseaux répartis sur trois zones de disponibilité. . . . 83

Figure 6.12 Trois passerelles NAT déployées, une par zone de disponibilité, matérialisant le choix de haute disponibilité au niveau du trafic sortant. 83

Figure 6.13 Tables de routage déployées, illustrant l'asymétrie entre la table publique partagée et les tables privées applicatives dédiées par zone. . 83

Figure 6.14 Deux tâches ECS Fargate en cours d'exécution, placées dans des sous-réseaux privés distincts. 86

Figure 6.15 Groupe cible affichant deux instances saines, réparties sur deux zones de disponibilité distinctes. 87

Figure 6.16 Page nginx servie à travers l'URL publique de l'ALB, confirmant le fonctionnement de bout en bout de la chaîne Internet $\rightarrow$ ALB $\rightarrow$ ECS. 87

Figure 6.17 Clé KMS gérée par le client avec rotation annuelle activée. 90
Figure 6.18 Dépôt ECR reconfiguré avec chiffrement KMS et immutabilité des tags. 90
Figure 6.19 Stratégie Auto Scaling Target Tracking associée au service ECS. . . 90
Figure 6.20 Historique d'une alarme CloudWatch matérialisant le cycle complet `OK` → `ALARM` → `OK` lors du test manuel. 92
Figure 6.21 Courrier électronique de notification effectivement reçu à la suite du déclenchement forcé de l'alarme. 93
Figure 6.22 Tableau de bord CloudWatch agrégeant les quatre indicateurs opérationnels de la plateforme applicative. 93
Figure 6.23 Exécution du workflow GitHub Actions bloquée par le point de contrôle des vulnérabilités : quatre CVEs critiques détectées entraînent un code de sortie non nul et l'annulation du déploiement. . 95
Figure 6.24 Résultat de l'analyse de vulnérabilités du dépôt ECR détaillant les quatre vulnérabilités CRITICAL identifiées sur l'image nginx de base. 96
Figure 6.25 Relations de confiance du rôle IAM après application du correctif sur le motif `StringLike` de la revendication `sub`. 96
Figure 6.26 Console AWS Budgets affichant le budget mensuel et ses deux seuils de notification. 98
Figure 6.27 Étiquettes d'allocation de coûts activées via la console, matérialisant l'action manuelle complémentaire au déploiement Terraform. . 99
Figure 6.28 Contenu du compartiment matérialisant la coexistence des deux préfixes conventionnels. 100
Figure 6.29 Règles de cycle de vie configurées côte à côte, avec la chaîne complète de transitions pour le préfixe froid. 100
Figure 6.30 Configuration des *cache behaviors* de la distribution, matérialisant le routage par chemin entre l'origine S3 (préfixe `/assets/*`) et l'origine ALB (comportement par défaut). 102
Figure 6.31 Contenu servi depuis l'origine S3 via l'OAC, accessible à travers la distribution CloudFront. 103

Figure 6.32 En-têtes de réponse HTTP indiquant un `x-cache: Hit from cloudfront` depuis le point de présence de Madrid, confirmant le fonctionnement de la mise en cache et la couverture géographique européenne. 103
Figure 6.33 Cinquième énoncé ajouté à la *key policy* KMS pour autoriser CloudFront à déchiffrer les objets S3. 103
Figure 6.34 Configuration *Dead-letter* de la file d'attente principale, matérialisant le lien avec la DLQ et la valeur du `maxReceiveCount`. . . . 105
Figure 6.35 Journaux CloudWatch de la fonction Lambda affichant successivement un traitement réussi et les tentatives infructueuses ayant conduit au rebasculement. 105
Figure 6.36 Message volontairement mal formé retrouvé dans la DLQ après trois échecs de traitement, confirmant le fonctionnement effectif de la politique de rebasculement. 105

Liste des tableaux

2.1 Synthèse des acteurs de la solution Atlas FinTech 15
2.2 Cas d'utilisation de la solution Atlas FinTech 16
2.3 Priorisation MoSCoW des besoins du projet 20
2.4 Matrice de traçabilité risques → mitigations 22

3.1 Comparaison des modèles de services Cloud 26
3.2 Comparaison des mécanismes de filtrage réseau 32
3.3 Synthèse des stratégies de reprise après sinistre 39

4.1 Synthèse des technologies et outils du projet Atlas FinTech 51

5.1 Permission sets définis dans IAM Identity Center 56
5.2 Comparaison des plateformes de conteneurs sur AWS 60
5.3 Traçabilité besoins → décisions de conception 65

6.1 Correspondance entre rôles conceptuels (chapitre 5) et comptes réellement déployés 77

Listings

Listing 6.1 Provisionnement du backend d'état Terraform. 71
Listing 6.2 Extrait de la SCP interdisant l'usage hors région Union européenne. . . 75
Listing 6.3 Chaîne de confiance ALB → App → Base de données via référence de groupes de sécurité. 81
Listing 6.4 Déclaration du service ECS avec placement multi-AZ et enregistrement automatique auprès du groupe cible. 84
Listing 6.5 Extrait de la *key policy* scopée autorisant CloudWatch Logs à chiffrer les journaux uniquement dans le compte propriétaire. 88
Listing 6.6 Alarme CloudWatch sur le taux d'erreurs 5xx de l'ALB, avec traitement explicite de l'absence de trafic. 91
Listing 6.7 Cinquième énoncé de la *key policy* KMS, autorisant CloudFront à déchiffrer les objets S3 servis via la distribution. 101

Liste des acronymes

ACM	AWS Certificate Manager
ALB	Application Load Balancer
API	Application Programming Interface
ARN	Amazon Resource Name
AWS	Amazon Web Services
AZ	Availability Zone
BF	Besoin Fonctionnel
BNF	Besoin Non Fonctionnel
CapEx	Capital Expenditure
CDN	Content Delivery Network
CIDR	Classless Inter-Domain Routing
CI/CD	Continuous Integration / Continuous Deployment
CIS	Center for Internet Security
CLI	Command Line Interface
CMK	Customer Managed Key
CVE	Common Vulnerabilities and Exposures
DDoS	Distributed Denial of Service
DLQ	Dead Letter Queue
DNS	Domain Name System
DR	Disaster Recovery
EC2	Elastic Compute Cloud
ECR	Elastic Container Registry
ECS	Elastic Container Service

EIP Elastic IP

FinOps Financial Operations

FSBP Foundational Security Best Practices

HTTPS HyperText Transfer Protocol Secure

IaaS Infrastructure as a Service

IaC Infrastructure as Code

IAM Identity and Access Management

ISO 27001 International Organization for Standardization 27001

JSON JavaScript Object Notation

KMS Key Management Service

LZA Landing Zone Accelerator

MFA Multi-Factor Authentication

MoSCoW Must have, Should have, Could have, Won't have

NAT Network Address Translation

OAC Origin Access Control

OIDC OpenID Connect

OpEx Operational Expenditure

OU Organizational Unit

PaaS Platform as a Service

PCI-DSS Payment Card Industry Data Security Standard

RGPD Règlement Général sur la Protection des Données

RPO Recovery Point Objective

RSSI Responsable de la Sécurité des Systèmes d'Information

RTO Recovery Time Objective

S3 Simple Storage Service

SaaS Software as a Service

SCP Service Control Policy

SG Security Group

SigV4 Signature Version 4

SNS Simple Notification Service

SOC 2 Security Operations Center 2

SQS Simple Queue Service

SSO Single Sign-On

STS Security Token Service

TLS Transport Layer Security

TTL Time To Live

UML Unified Modeling Language

URL Uniform Resource Locator

VPC Virtual Private Cloud

WAF Well-Architected Framework

Introduction générale

L'adoption massive du cloud computing a profondément transformé la manière dont les organisations conçoivent et exploitent leurs systèmes d'information. Cette évolution s'accompagne d'une exigence croissante en matière de sécurité, de gouvernance et de maîtrise opérationnelle : un environnement déployé sans socle préalable expose durablement l'entreprise à la dérive des privilèges, à l'absence de traçabilité et à une dépense non maîtrisée.

Le présent rapport s'inscrit dans le cadre d'un projet auto-dirigé mené au sein de la filière Génie Cyber Défense et Systèmes de Télécommunications Embarqués (GCDSTE) de l'ENSA Marrakech. Retenu en substitution à un stage conventionnel, il aborde un problème complet de fondation cloud, depuis les principes structurants jusqu'à la livraison d'une infrastructure opérationnelle.

Le sujet revêt un intérêt particulier pour une formation en cybersécurité : il permet de traiter la sécurité non comme un correctif ajouté a posteriori, mais comme une propriété intrinsèque du socle sur lequel reposent les charges de travail — identité, isolation, journalisation et contrôle des changements y étant pensés dès la conception.

La problématique retenue concerne l'établissement d'une base AWS sécurisée, automatisée et reproductible, en l'absence de toute infrastructure préexistante. Il ne s'agit pas de rendre un service fonctionnel, mais de poser les principes d'identité, de cloisonnement et de responsabilité qui permettront ensuite d'accueillir des charges métier en confiance. L'objectif est donc de concevoir et de mettre en œuvre une landing zone AWS orientée sécurité, en s'appuyant sur l'approche Infrastructure as Code.

Le rapport comprend cinq chapitres (contexte, état de l'art, technologies, conception, mise en œuvre) suivis d'une conclusion générale.

Chapitre 1

Contexte du projet : le cas Atlas FinTech

1 Contexte du projet : le cas Atlas FinTech

Ce chapitre pose le cadre du projet en présentant le cas d'usage retenu, Atlas FinTech, ainsi que le contexte réglementaire et technique qui en découle. Il expose ensuite l'état de l'art des approches de fondation cloud sur AWS et justifie le positionnement adopté. La plupart des incidents de sécurité observés dans le cloud ne proviennent pas d'une faille technique isolée, mais d'une fondation d'infrastructure mal conçue ou absente, sur laquelle les équipes empilent progressivement des correctifs ponctuels. Poser correctement ce cadre avant toute implémentation constitue donc un prérequis méthodologique, et non une étape accessoire du projet. C'est pourquoi ce chapitre s'attache à justifier chaque orientation par un besoin métier ou réglementaire concret, plutôt que par une bonne pratique générique détachée du contexte d'Atlas FinTech.

1.1 Présentation du cas d'usage

1.1.1 Le cas Atlas FinTech

Le projet prend appui sur un cas d'usage fictif mais construit dans un souci de vraisemblance sectorielle : Atlas FinTech, une jeune entreprise de technologie financière basée à Casablanca et opérant dans plusieurs marchés européens, ce qui l'assujettit au RGPD. Le choix du secteur financier n'est pas anodin. Il combine une exigence réglementaire forte (protection des données personnelles au titre du RGPD, obligations de rétention pluriannuelle des journaux d'audit inspirées des cadres SOC 2 et PCI-DSS), une sensibilité opérationnelle à la disponibilité des services, et une attention marquée à la maîtrise des coûts d'infrastructure dans une phase de croissance.



FIGURE 1.1 – Synthèse du scénario d'étude Atlas FinTech.

Atlas FinTech ne dispose d'aucune infrastructure cloud préexistante, ce qui place le projet dans une situation de fondation pure : chaque choix structurant peut être posé sans contrainte d'héritage. Cette situation, favorable à la démonstration pédagogique, est également celle que rencontrent effectivement les organisations en phase d'amorçage ou de migration *lift-and-shift* depuis un hébergement on-premise vers le cloud.

1.1.2 Enjeux de sécurité et de gouvernance

Un environnement AWS déployé sans socle préalable expose durablement son propriétaire à quatre catégories de risques, qui reviennent transversalement dans les référentiels de sécurité cloud.

Dérive des privilèges. Sans mécanisme d'authentification fédérée ni politiques de contrôle centralisées, les utilisateurs finissent par accumuler des droits résiduels dont plus personne ne connaît la justification.

Absence de traçabilité. Sans journalisation d'audit centralisée, aucune reconstitution a posteriori d'un incident n'est possible.

Difficulté d'audit. Les référentiels de conformité (CIS, ISO 27001, SOC 2) exigent la démonstration continue du respect de règles précises, ce qui suppose une capacité de mesure et non de simple déclaration.

Dépense non maîtrisée. La facturation à l'usage du cloud transforme la moindre erreur de configuration en surcoût réel.

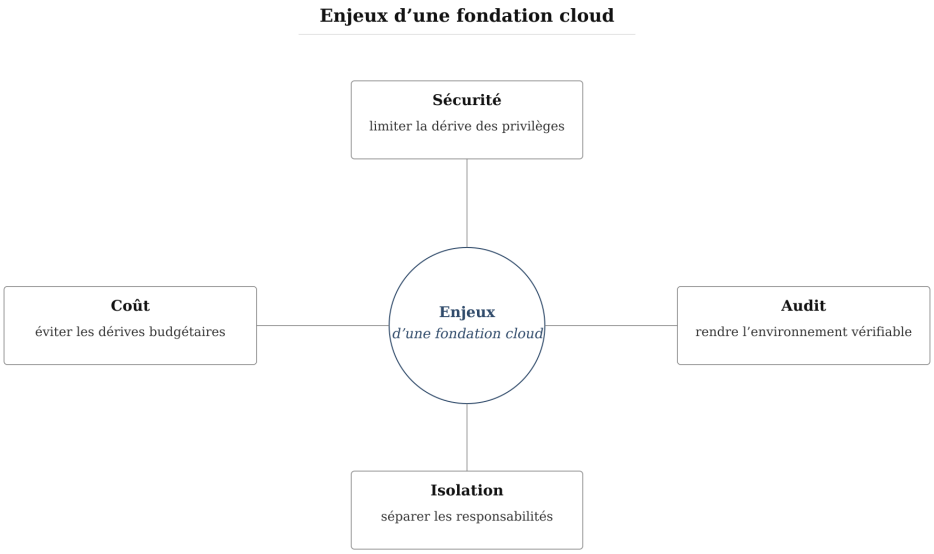


FIGURE 1.2 – Les quatre enjeux structurants d'une fondation cloud : sécurité, isolation, audit et coût.

Ces quatre risques ne sont pas indépendants : ils se renforcent mutuellement en l'absence d'un socle explicitement conçu pour les prévenir. La démarche proposée dans ce rapport consiste à traiter ces exigences dès la construction de la fondation, plutôt qu'à les adresser après coup par des correctifs successifs.

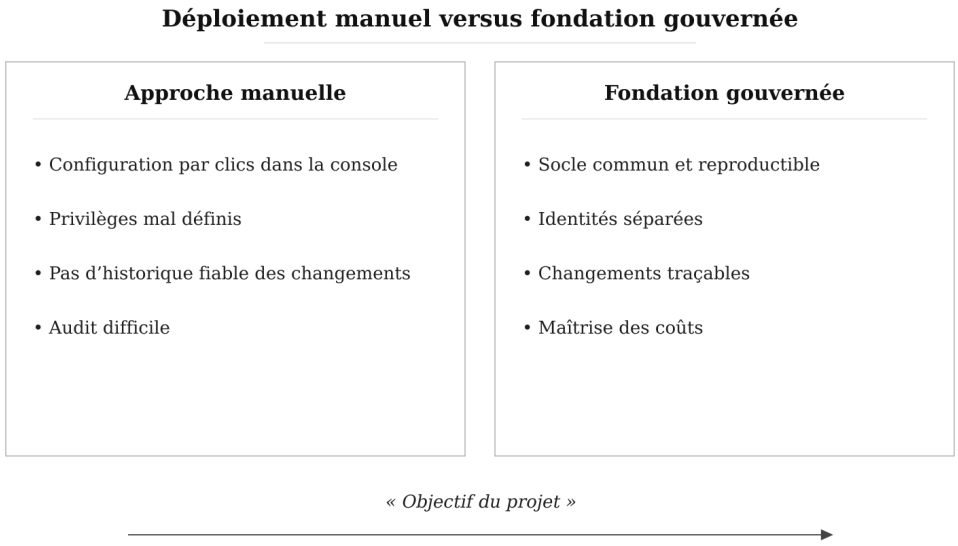


FIGURE 1.3 – Déploiement manuel versus fondation gouvernée : l'objectif du projet.

1.1.3 Périmètre du projet

Le projet couvre la conception et la mise en œuvre du socle d'infrastructure sur lequel Atlas FinTech pourra ensuite déployer ses applications métier. La figure 1.4 synthétise les éléments inclus et exclus de ce périmètre.

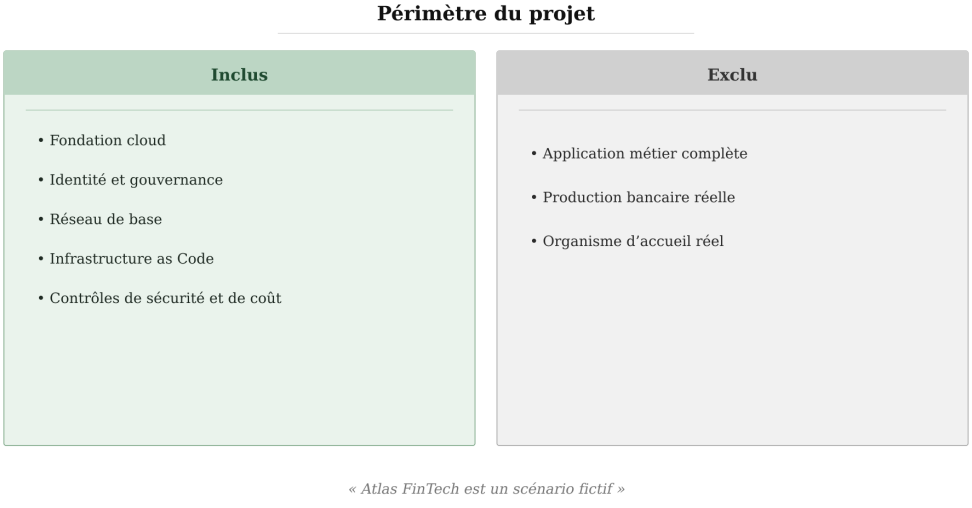


FIGURE 1.4 – Périmètre du projet : éléments inclus et exclus.

La conception d'un plan de reprise inter-région complet, explicitement exclue du périmètre, fera partie des perspectives d'évolution présentées en conclusion générale.

1.2 État de l'art des fondations cloud sur AWS

La question de la fondation cloud sécurisée fait l'objet d'une littérature riche, à la fois académique et professionnelle, qu'il importe de situer avant de justifier les choix retenus pour le projet Atlas FinTech. On distingue trois grandes familles de références : les cadres méthodologiques généralistes, les référentiels de conformité opérationnelle, et les guides prescriptifs d'implémentation.

1.2.1 Référentiels et cadres méthodologiques

La première famille est constituée par les cadres méthodologiques généralistes, au premier rang desquels *AWS Well-Architected Framework* [22], qui articule la conception d'une charge de travail selon six piliers : excellence opérationnelle, sécurité, fiabilité, efficacité des performances, optimisation des coûts et durabilité. Ce cadre ne fournit pas de configuration prête à l'emploi mais impose une grille de lecture appliquée systématiquement aux décisions d'architecture.

La deuxième famille rassemble les référentiels de conformité et de sécurité opérationnelle. *CIS AWS Foundations Benchmark v1.4.0* [**cis-aws**] et *AWS Foundational Security Best Practices* [**aws-fsbp**] définissent des règles précises et mesurables, à partir desquelles un service comme AWS Security Hub peut évaluer en continu la conformité d'un compte. Ces référentiels constituent le socle commun sur lequel s'appuient de nombreuses politiques internes d'entreprises régulées.

La troisième famille regroupe les guides opérationnels de type *AWS Prescriptive Guidance*, qui documentent des patterns concrets de mise en œuvre [**aws-prescriptive**]. Ces guides constituent une réponse à la question récurrente : comment traduire en configuration effective les principes énoncés dans le *Well-Architected Framework* et les référentiels CIS ?

1.2.2 Solutions clé-en-main

Face à la complexité de construire manuellement une fondation cloud, plusieurs solutions clé-en-main ont émergé. **AWS Control Tower** est le service géré d'AWS pour l'établissement au-

tomatisé d'une landing zone : il provisionne une organisation multi-comptes, active un jeu de garde-fous prédéfinis (*guardrails*) et maintient une conformité de base. **AWS Landing Zone Accelerator** (LZA) est une variante plus flexible, distribuée sous forme de code CloudFormation et destinée aux organisations aux besoins réglementaires étendus (secteur public, défense, finance régulée). Enfin, plusieurs éditeurs tiers proposent des solutions équivalentes construites sur Terraform.

Ces solutions présentent l'avantage d'une mise en route rapide et d'une conformité par défaut, mais elles présentent également une limite structurelle : la logique de configuration qui les sous-tend reste largement opaque à leurs utilisateurs. Un incident, une évolution réglementaire ou un besoin de personnalisation non prévu par le service peut alors se traduire par une dépendance forte et coûteuse à l'éditeur.

1.2.3 Positionnement du projet

Le projet Atlas FinTech ne se substitue pas à ces solutions ; il en adopte les principes fondamentaux tout en construisant la fondation depuis ses composants primitifs. Ce choix répond à une double finalité. La première est pédagogique : construire soi-même impose de comprendre chaque décision, là où l'usage d'une solution clé-en-main laisse en boîte noire des pans entiers de la configuration. La seconde est méthodologique : la maîtrise complète du code Terraform permet une personnalisation fine sans concession sur la conformité, et prépare l'ingénieur à traiter des situations dans lesquelles aucune solution clé-en-main n'est disponible.

Ce positionnement présente en outre l'avantage pédagogique de couvrir en profondeur les services et principes évalués par la certification AWS Solutions Architect Associate (SAA-C03), sur laquelle le projet s'aligne. Le projet s'inspire donc explicitement des principes du *Well-Architected Framework*, adopte les règles opérationnelles du CIS AWS Foundations Benchmark, s'appuie sur les patterns d'AWS Prescriptive Guidance, mais en produit une réalisation entièrement maîtrisée en Terraform, sans recours à un service géré de type Control Tower ni à un accélérateur externe.

Conclusion du chapitre

Ce chapitre a présenté le cas d'usage Atlas FinTech, entreprise fictive du secteur financier qui sert de fil conducteur au projet, ainsi que les enjeux structurants de sécurité, de gouvernance, d'auditabilité et de maîtrise des coûts qui en découlent. Il a également situé le travail au regard de la littérature et des solutions existantes, en explicitant les raisons qui conduisent à construire la fondation depuis ses composants primitifs plutôt qu'à recourir à un service géré de type Control Tower ou à un accélérateur tiers.

Le chapitre suivant traduit ce contexte en une analyse formelle des besoins fonctionnels et non fonctionnels, précise les acteurs et cas d'usage du système, et hiérarchise les priorités de conception. La conception architecturale proprement dite est ensuite exposée au chapitre 5, avant que sa mise en œuvre effective ne fasse l'objet du chapitre **??**.

Chapitre 2

ÉTUDE PRÉALABLE ET ANALYSE DES BESOINS

2 Étude préalable et analyse des besoins

Le chapitre 1 a introduit le cas d'usage Atlas FinTech, ses enjeux structurants et le positionnement du projet au regard des solutions clé-en-main existantes. Le présent chapitre formalise l'analyse qui en découle selon cinq angles complémentaires : la traduction du contexte en besoins fonctionnels et non fonctionnels, l'identification des acteurs qui interagissent avec la solution, le recensement des cas d'utilisation qui matérialisent ces besoins, l'inventaire des contraintes techniques, réglementaires et budgétaires qui bornent la conception, et enfin la priorisation des besoins ainsi qu'une première analyse des risques associés.

Cette analyse constitue le pivot entre l'énoncé du problème posé au chapitre 1 et les décisions de conception présentées au chapitre 5. Elle vise à ce que chaque choix technique ultérieur puisse être rattaché à un besoin explicite, un acteur identifié et un risque adressé.

2.1 Besoins fonctionnels et non fonctionnels

L'analyse des besoins traduit les enjeux exposés au chapitre 1 en exigences précises auxquelles la landing zone construite dans la suite du rapport devra répondre. Ces exigences sont formalisées selon la distinction classique entre besoins fonctionnels, qui décrivent ce que l'infrastructure doit permettre, et besoins non fonctionnels, qui décrivent les propriétés que cette infrastructure doit posséder.

2.1.1 Besoins fonctionnels

Les besoins fonctionnels retenus sont au nombre de cinq.

BF1 — Structurer une organisation multi-comptes AWS. L'infrastructure doit reposer sur une organisation AWS structurée par unités organisationnelles et comptes séparés, chaque compte accueillant un périmètre fonctionnel distinct.

BF2 — Fédérer les identités humaines. L'accès des utilisateurs à l'ensemble des comptes doit passer par un point d'authentification centralisé, protégé par une authentification à facteur multiple.

BF3 — Exposer les applications de manière sécurisée. Les charges applicatives doivent être accessibles depuis Internet via un équilibreur de charge placé en amont, sans exposition directe des instances de calcul.

BF4 — Automatiser le cycle de vie applicatif. La publication et le déploiement des mises à jour applicatives doivent être pilotés par un pipeline d'intégration continue, authentifié par fédération OpenID Connect et exempt de tout identifiant AWS à longue durée de vie.

BF5 — Superviser en continu la sécurité et l'exploitation. L'infrastructure doit produire de manière automatique les métriques, journaux et constatations de sécurité nécessaires à la détection d'incidents et au diagnostic opérationnel.

2.1.2 Besoins non fonctionnels

Les besoins non fonctionnels décrivent les propriétés transversales attendues du socle.

BNF1 — Sécurité par défaut. Toute donnée manipulée par le socle doit être chiffrée au repos, l'accès à toute ressource doit procéder du principe de moindre privilège, et aucun service ne doit être exposé publiquement en l'absence de justification explicite.

BNF2 — Haute disponibilité intra-région. Toute couche applicative doit tolérer la défaillance complète d'une zone de disponibilité sans interruption du service.

BNF3 — Traçabilité et auditabilité. Toute action effectuée dans l'infrastructure doit être journalisée de manière inaltérable et centralisée, pour permettre reconstitution d'incident et démonstration de conformité.

BNF4 — Automatisation et reproductibilité. L'ensemble de l'infrastructure doit être décrit comme code, versionné, et reproductible à l'identique dans un compte AWS vierge à partir du seul code source.

BNF5 — Maîtrise des coûts. Le socle doit intégrer des mécanismes d'alerte budgétaire déclenchés avant tout dépassement significatif, et permettre la ventilation des dépenses selon des axes métier grâce à des étiquettes d'allocation.

BNF6 — Contention des rayons d'action. L'organisation du code d'infrastructure doit garantir qu'une opération de déploiement ou de destruction ne peut affecter que le périmètre fonctionnel du composant modifié, sans risque de propagation involontaire aux autres composants.

2.1.3 Synthèse

L'ensemble de ces onze besoins constitue le référentiel auquel les choix de conception présentés au chapitre 5 et les déploiements décrits au chapitre **??** devront répondre. Chaque décision d'architecture prise dans la suite du rapport sera reliée à l'un ou plusieurs de ces besoins, ce qui permettra une évaluation objective du projet à son terme.

2.2 Identification des acteurs

L'identification des acteurs distingue les intervenants humains, qui manipulent la solution directement ou indirectement, des acteurs techniques, c'est-à-dire les systèmes externes qui interagissent avec la landing zone de manière automatisée.

2.2.1 Acteurs humains

Quatre rôles humains sont identifiés au regard du périmètre du projet.

L'**administrateur cloud** conçoit, déploie et fait évoluer la landing zone. Il opère depuis le compte de gestion pour la configuration organisationnelle, et depuis les comptes membres pour la gestion des ressources. Il est le principal utilisateur de la chaîne Terraform et le destinataire des alertes d'observabilité.

Le **développeur applicatif** intervient sur le code des charges de travail hébergées dans les

comptes *Workloads*. Il n'accède pas directement à la console AWS : ses interactions passent exclusivement par le dépôt Git, dont chaque *push* déclenche la pipeline CI/CD.

L'**auditeur** ou **RSSI** consulte les journaux d'audit, vérifie la conformité des configurations et suit les *findings* de sécurité. Ses accès sont en lecture seule et transitent par IAM Identity Center.

L'**utilisateur final** consomme l'application depuis un navigateur, sans conscience de l'infrastructure sous-jacente. Il interagit exclusivement avec la distribution CloudFront exposée publiquement.

2.2.2 Acteurs techniques

Trois systèmes externes interagissent de manière automatisée avec la landing zone.

GitHub Actions héberge la définition de la pipeline CI/CD et exécute les *jobs* de *build*, *scan* et déploiement. La fédération OIDC lui permet d'assumer un rôle IAM sans credentials stockés.

Amazon GuardDuty constitue une source d'événements de sécurité générés en continu par l'analyse des journaux CloudTrail, VPC Flow Logs et DNS. Les *findings* qu'il produit alimentent la chaîne de traitement asynchrone décrite au chapitre 5.

Trivy intervient depuis GitHub Actions comme scanner de vulnérabilités des images de conteneurs, en amont de leur publication dans le registre ECR.

2.2.3 Synthèse des acteurs

Le tableau 2.1 synthétise l'ensemble des acteurs et leur mode d'interaction avec la solution.

2.2.4 Diagramme de contexte

La figure 2.1 situe le système Atlas FinTech au centre des interactions avec l'ensemble des acteurs identifiés, humains et techniques. Contrairement au diagramme de cas d'utilisation présenté en section 2.3, ce diagramme ne détaille pas les actions possibles mais donne une vue

TABLE 2.1 – Synthèse des acteurs de la solution Atlas FinTech

Acteur	Nature	Mode d'interaction principal
Administrateur cloud	Humain	Console AWS via IAM Identity Center, Terraform
Développeur applicatif	Humain	Dépôt Git (*push* sur `main`)
Auditeur / RSSI	Humain	CloudWatch Logs, tableaux de bord CloudTrail
Utilisateur final	Humain	Navigateur web via CloudFront
GitHub Actions	Technique	Fédération OIDC vers AWS STS
Amazon GuardDuty	Technique	Publication d'*events* sur EventBridge
Trivy	Technique	Scan des images depuis GitHub Actions

d'ensemble de *qui* interagit avec le système et par quel canal.

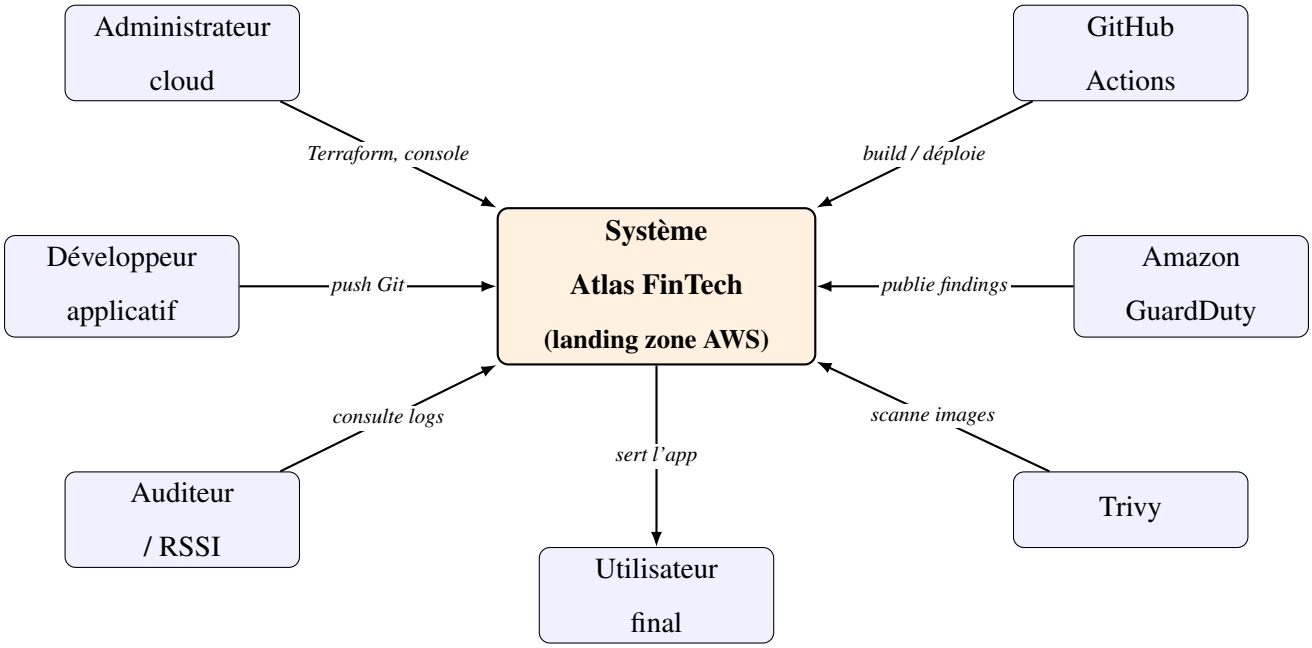


FIGURE 2.1 – Diagramme de contexte de la solution Atlas FinTech

2.3 Cas d'utilisation

Les interactions entre les acteurs et la solution se traduisent en un ensemble de cas d'utilisation, qui matérialisent les besoins fonctionnels énoncés en section 2.1.

2.3.1 Recensement des cas d'utilisation

Sept cas d'utilisation principaux sont retenus, présentés dans le tableau 2.2. Chaque cas est rattaché à l'acteur qui en est à l'origine et au besoin fonctionnel qu'il satisfait.

TABLE 2.2 – Cas d'utilisation de la solution Atlas FinTech

Cas d'utilisation	**Acteur**	**Besoin**
Déployer une charge de travail	Administrateur	BF1
Réviser une politique SCP	Administrateur	BF1
Publier une nouvelle version	Développeur	BF4
Consommer l'application	Utilisateur final	BF3
Consulter les journaux d'audit	Auditeur	BNF3
Traiter un *finding* de sécurité	GuardDuty (auto)	BF5
Scanner une image de conteneur	Trivy (auto)	BF4

2.3.2 Diagramme de cas d'utilisation

La figure 2.2 présente le diagramme UML de cas d'utilisation, articulant les acteurs autour du système Atlas FinTech. Le cas *Publier une nouvelle version* inclut systématiquement le cas *Scanner une image de conteneur*, conformément au *Security Gate* décrit ci-après.

FIGURE 2.2 – Diagramme de cas d'utilisation de la solution Atlas FinTech

2.3.3 Description des cas critiques

Deux cas d'utilisation présentent un caractère critique pour la sécurité globale de la solution et méritent une description détaillée.

Publier une nouvelle version applicative. Ce cas est déclenché par un *push* sur la branche `main` du dépôt applicatif. La chaîne exécute successivement la construction de l'image, le scan Trivy, la publication ECR et le déploiement ECS. Le point critique réside dans la présence d'un *Security Gate* bloquant : si Trivy détecte au moins une vulnérabilité de sévérité CRITICAL, la publication est refusée et le développeur est notifié. Ce cas satisfait le BF4 (automatisation du

cycle de vie applicatif), dont l'absence de secrets à longue durée de vie fait partie intégrante de la définition formulée en section 2.1.1.

Traiter un *finding* de sécurité. Ce cas est déclenché de manière asynchrone lorsqu'Amazon GuardDuty publie un *finding*. L'événement est routé par EventBridge vers une file SQS, consommée par une fonction Lambda dédiée qui écrit l'événement enrichi dans CloudWatch Logs. En cas d'échec répété du traitement, le *finding* est transféré vers une *Dead Letter Queue* pour investigation manuelle. Ce cas satisfait le BF5 (supervision continue de la sécurité et de l'exploitation).

La figure 2.3 synthétise ces deux flux, en mettant en évidence les points de décision qui conditionnent leur issue.

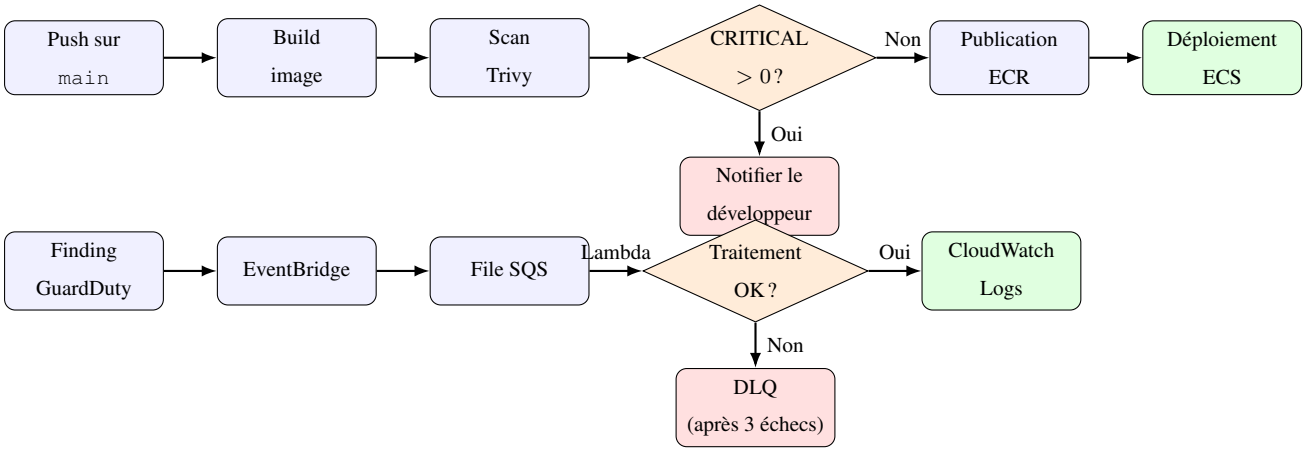


FIGURE 2.3 – Flux des deux cas d'utilisation critiques

2.4 Contraintes du projet

Trois catégories de contraintes bornent l'espace des solutions envisageables : les contraintes techniques héritées des choix de plateforme, les contraintes réglementaires inspirées des cadres du secteur financier, et les contraintes budgétaires et organisationnelles propres au cadre académique du projet.

2.4.1 Contraintes techniques

Les contraintes techniques découlent des choix stratégiques de la plateforme cible et des pratiques d'ingénierie retenues.

- **Infrastructure as Code exclusive.** Toute ressource AWS déployée doit l'être via Terraform, versionné dans Git. Aucune modification manuelle (*ClickOps*) n'est tolérée en dehors des configurations initiales explicitement documentées.
- **Region européenne unique.** Le déploiement retient la région *eu-west-1*. Le compte de journalisation est configuré de manière à pouvoir accueillir un futur trail multi-régions sans refonte structurelle.
- **Services managés privilégiés.** Les services managés par AWS (Fargate, RDS, Lambda, SQS) sont systématiquement préférés aux alternatives auto-gérées, afin de réduire la charge opérationnelle et de bénéficier des mises à jour de sécurité gérées par le fournisseur.
- **Absence de credentials long-lived.** Aucun secret statique ne doit être stocké dans le dépôt Git, dans les Secrets GitHub, ni dans les fichiers de configuration Terraform.

2.4.2 Contraintes réglementaires et de conformité

L'inspiration réglementaire du projet repose sur trois cadres, sans qu'aucun ne soit imposé de manière formelle dans le contexte académique.

Le Règlement Général sur la Protection des Données (RGPD) impose la localisation des données à caractère personnel dans l'espace économique européen. Le choix de la région *eu-west-1* et l'absence de service opéré hors UE respectent cette exigence.

Les CIS AWS Foundations Benchmarks [**cis-aws**] définissent un socle de contrôles de configuration que la landing zone vise à satisfaire, notamment sur les volets CloudTrail, IAM et VPC.

Les AWS Foundational Security Best Practices [**aws-fsbp**] fournissent un référentiel opérationnel aligné sur AWS Security Hub. La conformité à ce référentiel est mesurable automatique-

ment, ce qui en fait un critère de qualité vérifiable dans la durée.

2.4.3 Contraintes budgétaires et organisationnelles

Le projet s'inscrit dans un cadre académique auto-financé, ce qui impose deux limites structurelles.

D'une part, les ressources générant un coût continu (NAT Gateways, tâches Fargate, base RDS) ne sont maintenues actives que pendant les phases de validation. Après validation, elles sont détruites via `terraform destroy` et redéployables à la demande. Cette contrainte a orienté le choix de patterns *serverless* lorsque possible (Lambda, SQS, EventBridge) pour la couche de sécurité opérationnelle.

D'autre part, l'unicité du contributeur limite la bande passante disponible pour l'exploitation continue. Cette contrainte a écarté d'emblée la construction d'une véritable astreinte, remplacée par une observabilité active par notification mail.

2.5 Priorisation des besoins

Face à la contrainte de temps du projet, une priorisation des besoins selon la méthode MoSCoW a été conduite. Cette méthode répartit les besoins en quatre catégories : *Must have* (indispensables), *Should have* (fortement souhaitables), *Could have* (souhaitables si le temps le permet) et *Won't have this time* (hors périmètre de cette itération).

Le tableau 2.3 présente cette répartition. Elle a directement guidé l'ordre de mise en œuvre des phases décrites au chapitre **??**.

Les besoins classés *Must have* et *Should have* constituent le périmètre effectivement mis en œuvre dans le cadre du projet. Les besoins *Could have* sont mentionnés comme perspectives dans la conclusion générale du rapport.

TABLE 2.3 – Priorisation MoSCoW des besoins du projet

Catégorie	Besoins concernés
Must have	Structure multi-comptes (BF1), fédération d'identités (BF2), exposition applicative sécurisée (BF3), traçabilité et auditabilité (BNF3), sécurité par défaut incluant le chiffrement au repos (BNF1), automatisation et reproductibilité par IaC (BNF4)
Should have	Pipeline CI/CD avec *Security Gate* (BF4), supervision continue et traitement asynchrone des *findings* (BF5), haute disponibilité multi-AZ (BNF2), maîtrise des coûts par alertes budgétaires (BNF5), contention des rayons d'action (BNF6)
Could have	*Custom domain* via Route 53, Web Application Firewall (WAF) applicatif, tableaux de bord CloudWatch enrichis
Won't have	Réplication multi-régions (Disaster Recovery), astreinte opérationnelle, charge de travail applicative en production réelle

2.6 Analyse des risques

L'analyse des risques identifie les menaces pesant sur la solution et met en regard les mesures de mitigation intégrées à la conception. Cette analyse ne vise pas l'exhaustivité mais une couverture des scénarios les plus structurants pour l'architecture retenue.

2.6.1 Identification des risques principaux

Sept risques structurants sont retenus.

- **R1 — Compromission d'un principal IAM.** Un attaquant obtient des credentials d'un utilisateur ou d'un rôle et les exploite pour agir sur l'infrastructure.
- **R2 — Perte d'une zone de disponibilité.** Une panne affecte simultanément l'ensemble des ressources hébergées dans une AZ.
- **R3 — Déploiement d'une image vulnérable.** Une image contenant une CVE critique est publiée en production sans détection préalable.
- **R4 — Altération du journal d'audit.** Un attaquant désactive CloudTrail ou supprime des événements pour effacer la trace de son intrusion.
- **R5 — Fuite de données depuis S3.** Un bucket contenant des données sensibles est rendu public par erreur de configuration.

- **R6 — Vol de credentials de la chaîne CI/CD.** Un secret AWS statique stocké dans la CI est exfiltré via une vulnérabilité du dépôt.
- **R7 — Saturation du pipeline de sécurité.** Un pic d'événements GuardDuty sature la capacité de traitement et provoque la perte de *findings*.

2.6.2 Matrice probabilité / impact

La figure 2.4 positionne chaque risque selon sa probabilité d'occurrence et son impact estimé, avant application des mesures de mitigation. Ce positionnement complète le tableau de traçabilité (tableau 2.4) en mettant en évidence les risques les plus critiques d'un simple coup d'œil.

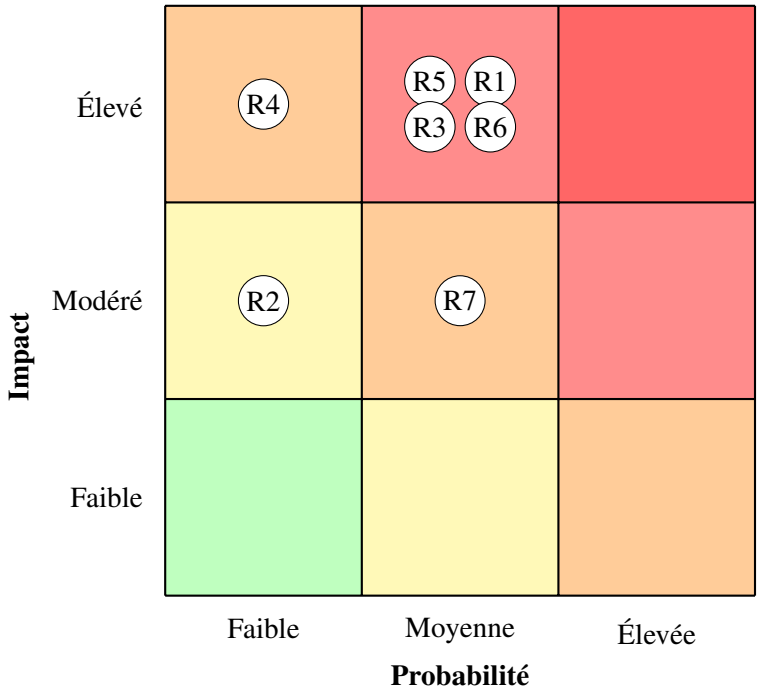


FIGURE 2.4 – Matrice probabilité / impact des risques identifiés

2.6.3 Matrice de traçabilité risques → mitigations

Le tableau 2.4 met en regard chaque risque identifié avec la mesure de mitigation intégrée à la conception, et renvoie à la section du chapitre 5 qui la détaille.

Cette traçabilité garantit qu'aucun risque identifié ne reste sans réponse dans la conception présentée au chapitre suivant.

TABLE 2.4 – Matrice de traçabilité risques → mitigations

Risque	Mesure de mitigation	Mécanisme technique	Section
R1	Isolation multi-comptes, SCPs restrictives	Organizations, SCPs	5.2
R2	Déploiement multi-AZ complet	3 AZ, NAT par zone	5.3
R3	*Security Gate* bloquant sur Trivy	GitHub Actions	5.5
R4	SCP interdisant la modification de CloudTrail	SCP racine	5.2
R5	Bucket privé avec OAC + SigV4	CloudFront, S3, OAC	5.4
R6	Fédération OIDC sans credentials statiques	AWS STS, GitHub OIDC	5.5
R7	Découplage par SQS avec DLQ	EventBridge, SQS	5.6

Conclusion du chapitre

Ce chapitre a approfondi l'énoncé du problème posé au chapitre 1 en formalisant onze besoins fonctionnels et non fonctionnels, en identifiant sept acteurs de la solution, en recensant les cas d'utilisation qui structurent leurs interactions, en dressant l'inventaire des contraintes techniques, réglementaires et budgétaires, et en priorisant les besoins selon la méthode MoSCoW. Une première analyse des risques a également été conduite, mettant en regard chaque menace identifiée avec la mesure de mitigation retenue.

L'ensemble constitue le socle d'analyse à partir duquel les décisions de conception, présentées au chapitre 5, pourront être justifiées et rattachées à un besoin, un acteur et un risque explicitement identifiés. Le chapitre suivant expose les fondements théoriques mobilisés pour ces décisions, avant que le chapitre 4 n'introduise les services AWS concrètement retenus.

Chapitre 3

FONDEMENTS DES INFRASTRUCTURES CLOUD SÉCURISÉES

3 Fondements des infrastructures cloud sécurisées

Toute architecture cloud repose sur un ensemble de concepts fondamentaux dont la maîtrise conditionne la pertinence des choix de conception. Avant de décrire l'infrastructure spécifique déployée dans le cadre de ce projet, il est nécessaire de revenir sur les principes qui régissent le fonctionnement du Cloud Computing, les mécanismes de gouvernance permettant de structurer un environnement multi-comptes, ainsi que les bonnes pratiques associées à la sécurité, à la conteneurisation et à l'automatisation des déploiements. Ces notions constituent un socle commun, indépendant de tout fournisseur cloud particulier, sur lequel s'appuient l'ensemble des décisions architecturales présentées ultérieurement.

Ce chapitre aborde successivement les principes du Cloud Computing, la gouvernance et l'architecture multi-comptes, la sécurité des architectures cloud, la conteneurisation, l'observabilité des systèmes distribués, les pratiques DevSecOps, les architectures événementielles, la distribution de contenu, la résilience, la gestion financière du cloud, ainsi que l'Infrastructure as Code. Chacune de ces notions sera reprise et illustrée de manière concrète dans le chapitre consacré à l'implémentation, où les choix technologiques spécifiques retenus pour ce projet seront détaillés et justifiés.

3.1 Le Cloud Computing

3.1.1 Du modèle traditionnel au modèle Cloud

Avant l'émergence du cloud, les organisations devaient investir dans des serveurs physiques, des équipements de stockage et des infrastructures réseau. Ce modèle nécessitait un investissement initial élevé (Capital Expenditure (CapEx)) et un dimensionnement pour les pics d'activité,

entraînant souvent un gaspillage de capacité.

Avec le Cloud Computing, les ressources sont louées à la demande auprès de fournisseurs tels qu'AWS, reposant sur des dépenses opérationnelles (Operational Expenditure (OpEx)) plutôt que sur un investissement initial.

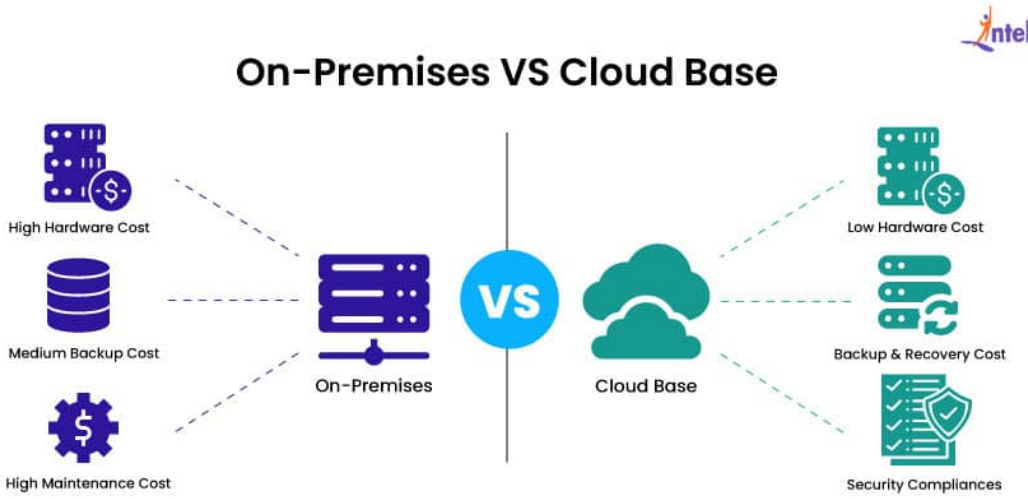


FIGURE 3.1 – Cloud versus On-Premise

3.1.2 Avantages et limites du Cloud Computing

- **Réduction des coûts** : facturation à l'usage, sans achat de matériel.
- **Flexibilité et élasticité** : adaptation rapide des ressources à la demande.
- **Disponibilité renforcée** : infrastructures réparties et résilientes.
- **Dépendance au fournisseur** : risque de *vendor lock-in*.
- **Confidentialité des données** : enjeu accru dans un environnement mutualisé.

3.1.3 Architecture et caractéristiques du Cloud Computing

Le Cloud Computing repose sur cinq caractéristiques essentielles :

1. **Libre-service à la demande** : provisionnement automatique sans intervention humaine du fournisseur.

2. **Accès réseau étendu** : disponibilité depuis différents types d'appareils.
3. **Mutualisation des ressources** : modèle multi-locataire.
4. **Élasticité rapide** : ajustement dynamique selon la demande.
5. **Service mesuré** : facturation transparente basée sur l'usage réel.

3.1.4 Modèles de services Cloud

Les services cloud sont proposés selon plusieurs modèles, se distinguant par le niveau de responsabilité laissé au client dans la gestion de l'infrastructure sous-jacente. Le tableau 3.1 présente une comparaison de ces modèles, ainsi que leur usage dans le cadre de ce projet.

TABLE 3.1 – Comparaison des modèles de services Cloud

Modèle	**Description**	**Exemple d'usage dans ce projet**
Infrastructure as a Service (IaaS)	Ressources virtualisées (calcul, stockage, réseau)	Réseau VPC
Platform as a Service (PaaS)	Environnement géré, montée en charge automatisée	Amazon ECS Fargate
FaaS! (**FaaS!**)	Exécution de code à la demande, sans gestion serveur	AWS Lambda
Software as a Service (SaaS)	Application accessible via Internet, gérée intégralement	Non utilisé dans ce projet

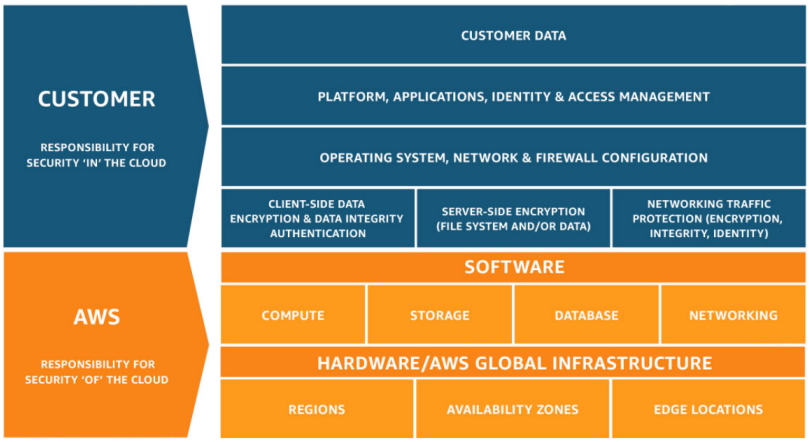


FIGURE 3.2 – Modèles de services Cloud (IaaS, PaaS, SaaS)

Comme le montre la Figure 3.2, le niveau de contrôle exercé par le client diminue progressivement du modèle IaaS au modèle SaaS, à mesure que le fournisseur prend en charge une part croissante de la pile technologique.

3.1.5 Modèles de déploiement du Cloud

Quatre modèles de déploiement coexistent : le cloud privé, dédié à une seule organisation ; le cloud public, partagé et accessible via Internet ; le cloud hybride, combinant environnements privés et publics ; et le multi-cloud, reposant sur plusieurs fournisseurs. Ce projet repose exclusivement sur un modèle de cloud public, choix justifié par l'objectif de préparation à une certification spécifique à un fournisseur (AWS).

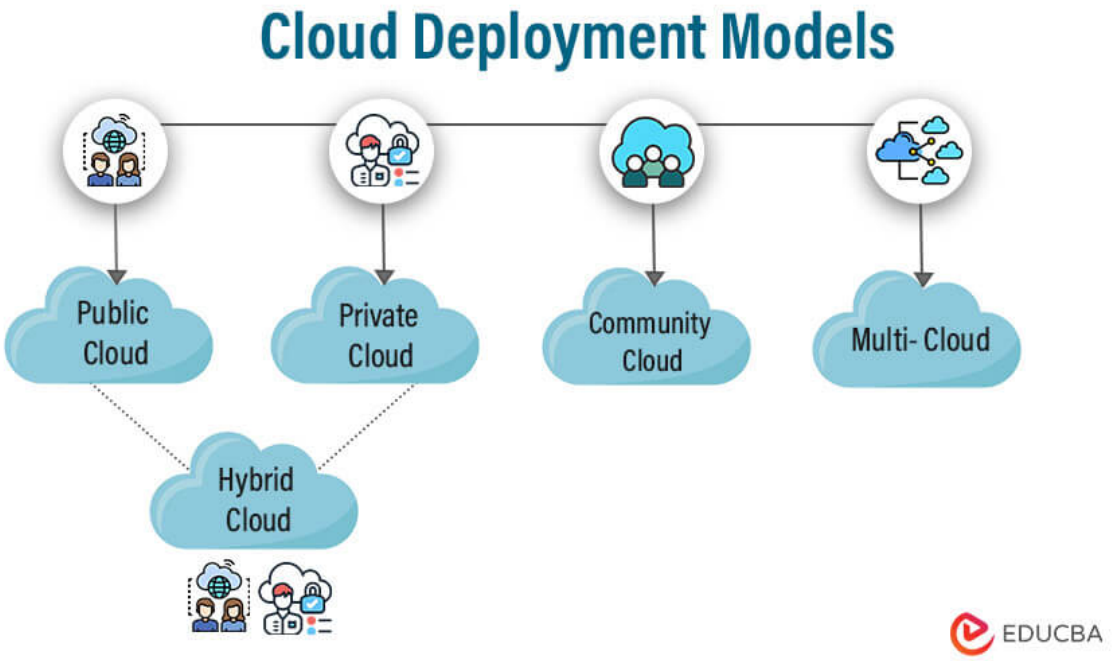


FIGURE 3.3 – Modèles de déploiement du Cloud

Le choix d'un modèle de déploiement dépend principalement du niveau de contrôle recherché, des contraintes réglementaires et du budget disponible. Ce projet repose exclusivement sur le modèle de cloud public.

3.1.6 Modèle de responsabilité partagée et sécurité du Cloud

Définition clé

Le modèle de responsabilité partagée répartit les obligations de sécurité entre le fournisseur cloud, responsable de la sécurité *du* cloud (infrastructure physique, réseau sous-jacent), et le client, responsable de la sécurité *dans* le cloud (données, configuration, gestion des accès) [**aws-shared-responsibility**].

Le niveau de responsabilité du client varie selon le modèle de service utilisé : il est maximal en IaaS, intermédiaire en PaaS, et minimal en SaaS.

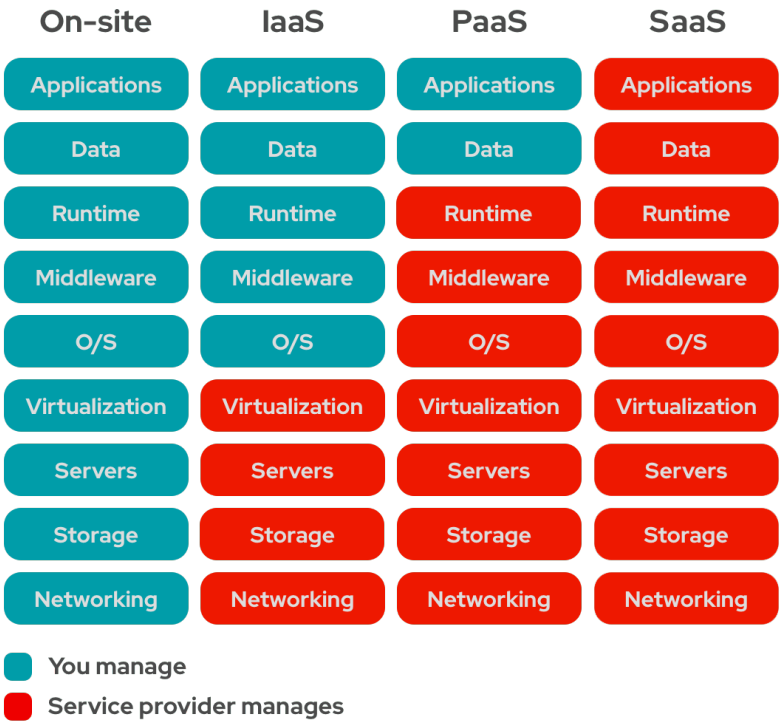


FIGURE 3.4 – Modèle de responsabilité partagée dans le Cloud

3.1.7 Cas d'utilisation du Cloud Computing

Le Cloud Computing est couramment utilisé pour l'hébergement d'applications web, le stockage et la sauvegarde de données, le développement et les tests logiciels, ainsi que pour des domaines avancés tels que l'analyse Big Data et l'intelligence artificielle.

3.2 Gouvernance et architecture cloud multi-comptes

3.2.1 Principe de la segmentation par comptes (landing zone)

Une *landing zone* désigne une architecture de référence permettant de structurer un environnement cloud multi-comptes dès sa conception, en appliquant des règles de sécurité, de gouvernance et de conformité de manière cohérente à l'ensemble des comptes gérés.

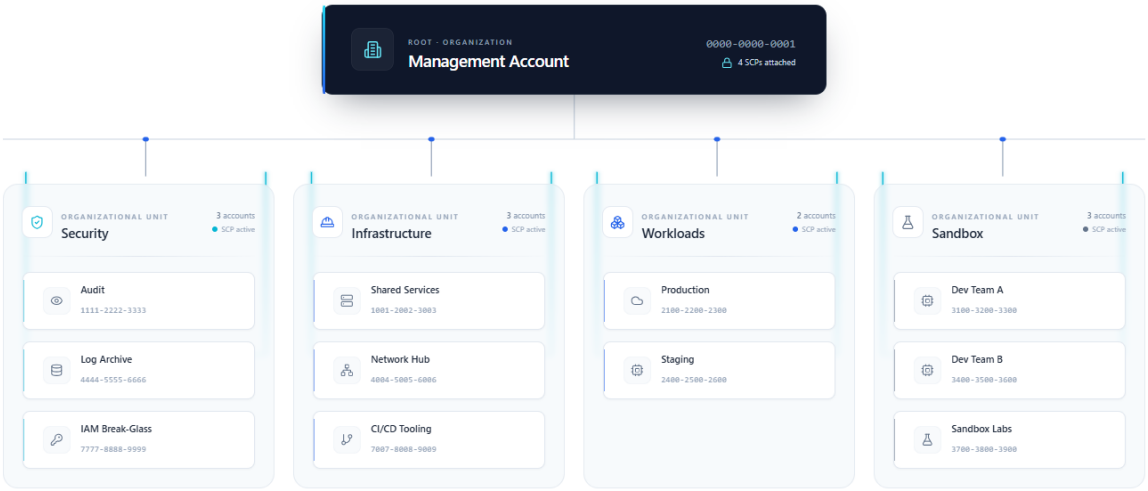


FIGURE 3.5 – Architecture de référence d'une landing zone multi-comptes

Il convient de préciser que ce schéma illustre un modèle de référence générique, distinct de l'architecture spécifique déployée dans le cadre de ce projet, laquelle sera présentée en détail dans le chapitre consacré à l'implémentation.

3.2.2 Organisation hiérarchique et unités organisationnelles

La structuration hiérarchique d'une organisation cloud repose généralement sur :

- un **compte de gestion (management account)**, point d'entrée administratif ;
- des **unités organisationnelles (OU)**, regroupant les comptes selon leur fonction (sécurité, infrastructure, charges de travail) ;
- des **comptes membres**, hébergeant les ressources opérationnelles.

3.2.3 Politiques organisationnelles et délégation administrative

Les politiques organisationnelles (*Service Control Policies*) permettent de définir des restrictions applicables à l'ensemble des comptes d'une organisation, indépendamment des permissions accordées au niveau individuel. Elles agissent comme un plafond de permissions : une action explicitement refusée au niveau organisationnel ne peut être autorisée par aucune politique locale, même en cas de privilèges administratifs complets sur le compte concerné.

Par ailleurs, certains services de sécurité peuvent être gérés de manière centralisée par un **compte administrateur délégué**, distinct du compte de gestion, afin de séparer les responsabilités de facturation et de sécurité opérationnelle.

3.2.4 Gestion centralisée des identités et principe du moindre privilège

Le principe du moindre privilège consiste à n'accorder à chaque identité (utilisateur, rôle, service) que les permissions strictement nécessaires à l'accomplissement de sa fonction, sans excès. Ce principe est renforcé par une gestion centralisée des identités, permettant d'authentifier les utilisateurs une seule fois pour accéder à plusieurs comptes selon leurs autorisations.

3.3 Sécurité des architectures cloud

3.3.1 Défense en profondeur et isolation réseau

La défense en profondeur repose sur la superposition de plusieurs couches de protection indépendantes, de sorte que la défaillance d'un mécanisme unique ne compromette pas l'ensemble du système.

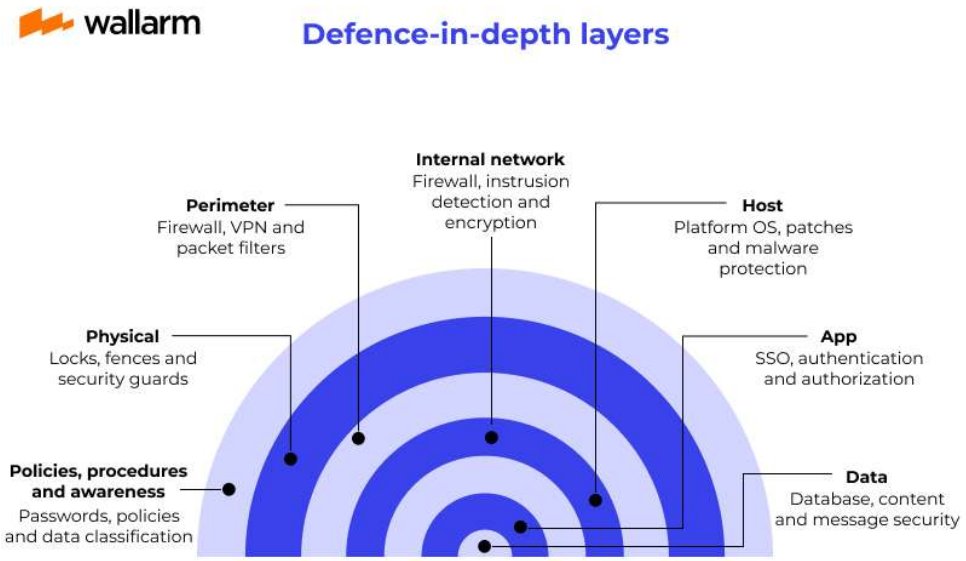


FIGURE 3.6 – Principe de la défense en profondeur

Ce principe se traduit concrètement par la superposition de plusieurs mécanismes complémentaires : segmentation réseau, contrôle d'accès, chiffrement et journalisation, chacun agissant comme une barrière indépendante.

3.3.2 Segmentation réseau et contrôle du trafic

La segmentation réseau consiste à isoler les ressources selon leur niveau de sensibilité, généralement à travers des sous-réseaux publics et privés. Le contrôle du trafic entre ces segments repose sur des règles de filtrage appliquées au niveau des ressources ou des sous-réseaux, selon deux logiques complémentaires :

TABLE 3.2 – Comparaison des mécanismes de filtrage réseau

Mécanisme	**Niveau d'application**	**Comportement**
Groupes de sécurité	Ressource / interface réseau	Stateful, règles d'autorisation uniquement
Listes de contrôle d'accès réseau	Sous-réseau	Stateless, règles d'autorisation et de refus

3.3.3 Chiffrement des données au repos et en transit

Le chiffrement des données au repos protège les informations stockées contre tout accès non autorisé, tandis que le chiffrement en transit protège les données lors de leur transmission sur le réseau. La gestion des clés de chiffrement peut être déléguée entièrement au fournisseur, ou confiée au client à travers des clés dites *customer-managed*, offrant un contrôle accru sur les politiques d'accès et de rotation des clés.

3.3.4 Traçabilité, audit et détection des menaces

La traçabilité des actions effectuées sur une infrastructure cloud constitue un pilier essentiel de la sécurité, permettant de répondre aux questions *qui*, *quoi*, *quand* pour chaque opération réalisée. Cette traçabilité est complétée par des mécanismes de détection automatisée des me-

naces, capables d'identifier des comportements anormaux (accès inhabituels, activité de reconnaissance, communication avec des infrastructures malveillantes connues) sans intervention humaine préalable.

3.4 Conteneurisation et architectures distribuées

3.4.1 Principes de la conteneurisation

La conteneurisation consiste à empaqueter une application avec l'ensemble de ses dépendances dans une unité isolée et portable, appelée conteneur, garantissant un comportement identique quel que soit l'environnement d'exécution.

3.4.2 Orchestration de conteneurs et modèles d'exécution

L'orchestration de conteneurs automatise leur déploiement, leur mise à l'échelle et leur supervision. Deux modèles d'exécution principaux coexistent : un modèle reposant sur des instances de calcul gérées par l'utilisateur, et un modèle *serverless*, où l'infrastructure sous-jacente est entièrement gérée par le fournisseur cloud.

3.4.3 Scalabilité automatique (auto-scaling)

La scalabilité automatique ajuste dynamiquement le nombre d'instances d'une application en fonction de métriques observées (utilisation processeur, mémoire, nombre de requêtes), garantissant un équilibre entre performance et maîtrise des coûts.

3.4.4 Répartition de charge et haute disponibilité

Un répartiteur de charge distribue le trafic entrant entre plusieurs instances d'une application, tout en effectuant des vérifications de santé (*health checks*) permettant de retirer automatiquement une instance défaillante de la rotation.

3.5 Observabilité des systèmes distribués

3.5.1 Les trois piliers de l'observabilité

L'observabilité d'un système distribué repose sur trois types de données complémentaires [**sridharan-observab**

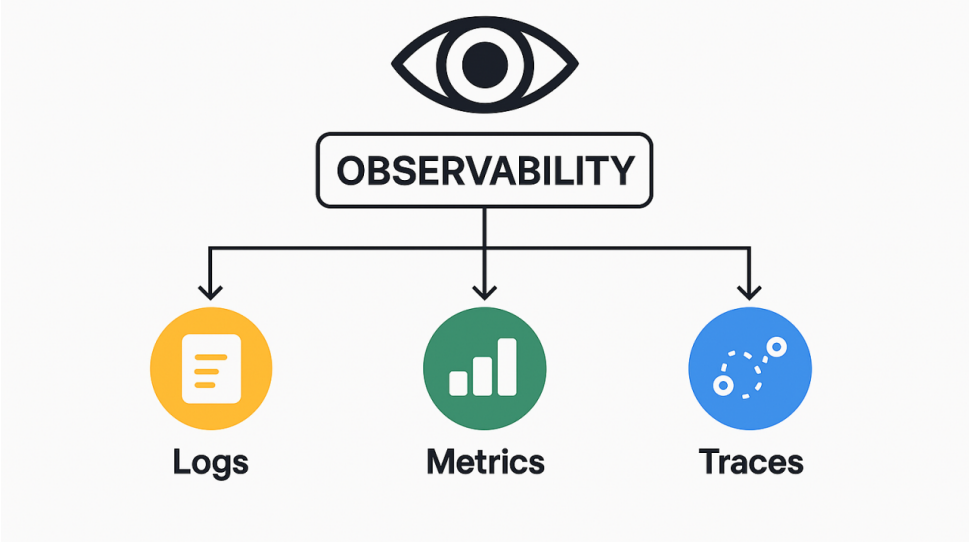


FIGURE 3.7 – Les trois piliers de l'observabilité

Il est à noter que le suivi par traces distribuées constitue un principe théorique important, mais n'a pas été mis en œuvre dans le cadre de ce projet, celui-ci reposant principalement sur les métriques et les journaux d'événements.

- **Métrique** : valeur numérique mesurée dans le temps (ex. utilisation processeur).
- **Log** : enregistrement d'un événement isolé (ex. erreur applicative).
- **Trace** : suivi d'une requête à travers plusieurs composants d'un système distribué.

3.5.2 Alerting, notification et supervision proactive

La supervision proactive repose sur la définition de seuils d'alerte appliqués aux métriques collectées, déclenchant automatiquement une notification lorsque ces seuils sont franchis, sans nécessiter de surveillance humaine continue.

3.5.3 Tableaux de bord et visualisation

Les tableaux de bord consolident plusieurs métriques et indicateurs dans une interface unique, facilitant l'identification rapide de l'état de santé global d'un système.

3.6 DevSecOps et intégration continue

3.6.1 Principe du *Shift Left Security*

Le principe du *Shift Left Security* consiste à intégrer les contrôles de sécurité le plus tôt possible dans le cycle de développement, plutôt que de les réserver à une phase finale de validation, réduisant ainsi le coût et la complexité de correction des vulnérabilités détectées.

3.6.2 Automatisation des pipelines de déploiement

Un pipeline d'intégration et de déploiement continu automatise l'enchaînement des étapes de construction, de test et de déploiement d'une application, réduisant les interventions manuelles et les risques d'erreur associés.

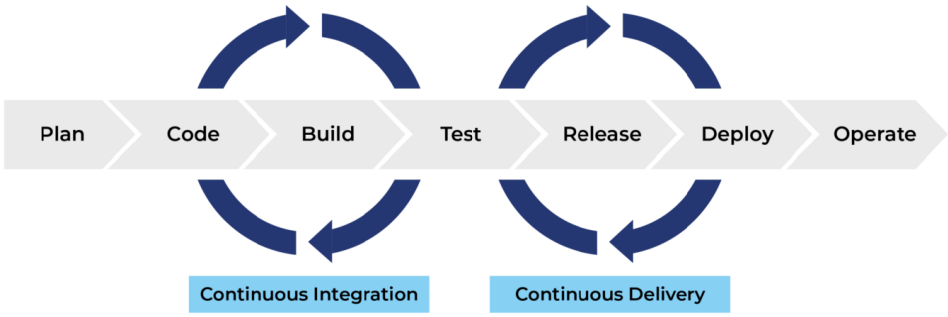


FIGURE 3.8 – Pipeline CI/CD générique intégrant un contrôle de sécurité

L'intégration d'un contrôle de sécurité au sein même du pipeline, plutôt qu'après le déploie-

ment, permet de bloquer automatiquement toute mise en production présentant une vulnérabilité critique.

3.6.3 Authentification fédérée et gestion des secrets

L'authentification fédérée permet à un système externe (par exemple, un outil d'intégration continue) d'obtenir des accès temporaires à une infrastructure cloud, sans recourir à des identifiants stockés de façon permanente, réduisant ainsi le risque de compromission de secrets à long terme.

3.6.4 Analyse de vulnérabilités et contrôle qualité

L'analyse automatisée de vulnérabilités permet d'identifier, au sein d'une image applicative ou d'un artefact de déploiement, des failles de sécurité connues avant leur mise en production. Un contrôle qualité intégré au pipeline peut être configuré pour bloquer automatiquement un déploiement en cas de détection d'une vulnérabilité jugée critique.

3.7 Architectures événementielles et découplage

3.7.1 Principe du découplage producteur/consommateur

Le découplage consiste à séparer la production d'un événement de son traitement, en insérant un intermédiaire (file de messages) entre les deux, permettant à chaque composant de fonctionner indépendamment et à des rythmes différents.

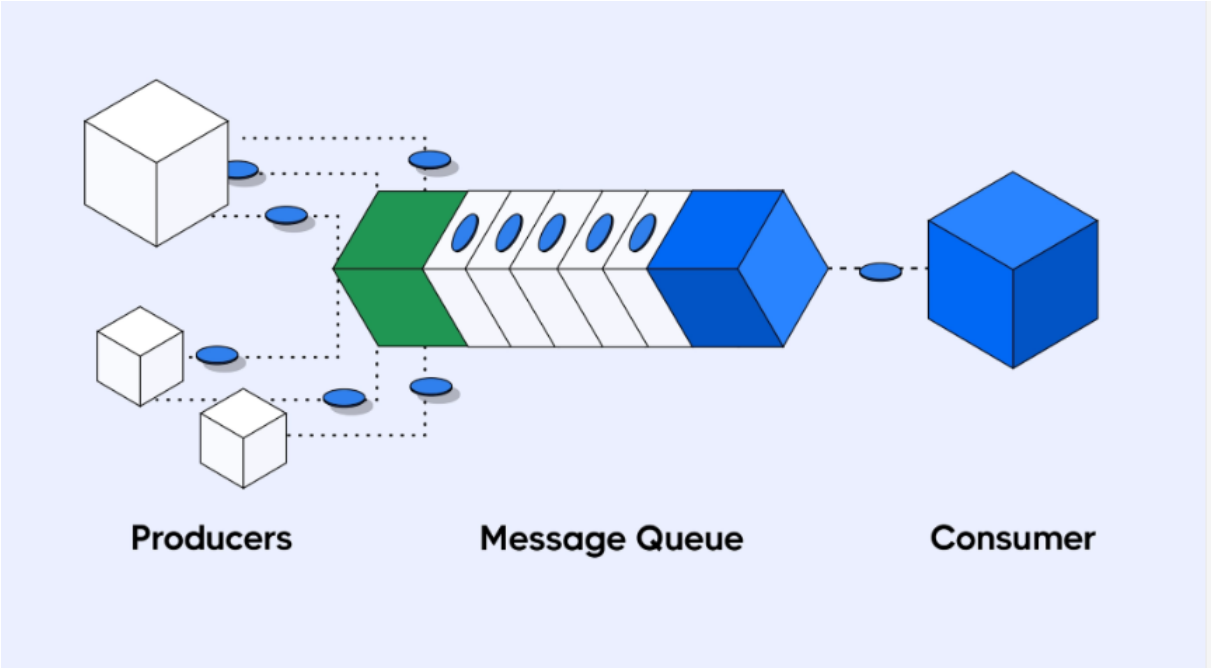


FIGURE 3.9 – Pattern de découplage

Ce découplage renforce la résilience globale du système : une indisponibilité temporaire du consommateur n'entraîne pas la perte des messages produits, ceux-ci restant disponibles dans la file d'attente.

3.7.2 Files de messages et gestion des échecs

Une file de messages garantit la livraison des messages même en cas d'indisponibilité temporaire du consommateur. Une file de messages non distribués (*Dead Letter Queue*) permet d'isoler les messages ayant échoué à plusieurs reprises, évitant qu'ils ne bloquent indéfiniment le traitement des messages suivants.

3.8 Distribution de contenu et performance

3.8.1 Réseaux de diffusion de contenu (CDN)

Un réseau de diffusion de contenu répartit des copies du contenu applicatif sur des serveurs géographiquement proches des utilisateurs finaux, réduisant la latence perçue.

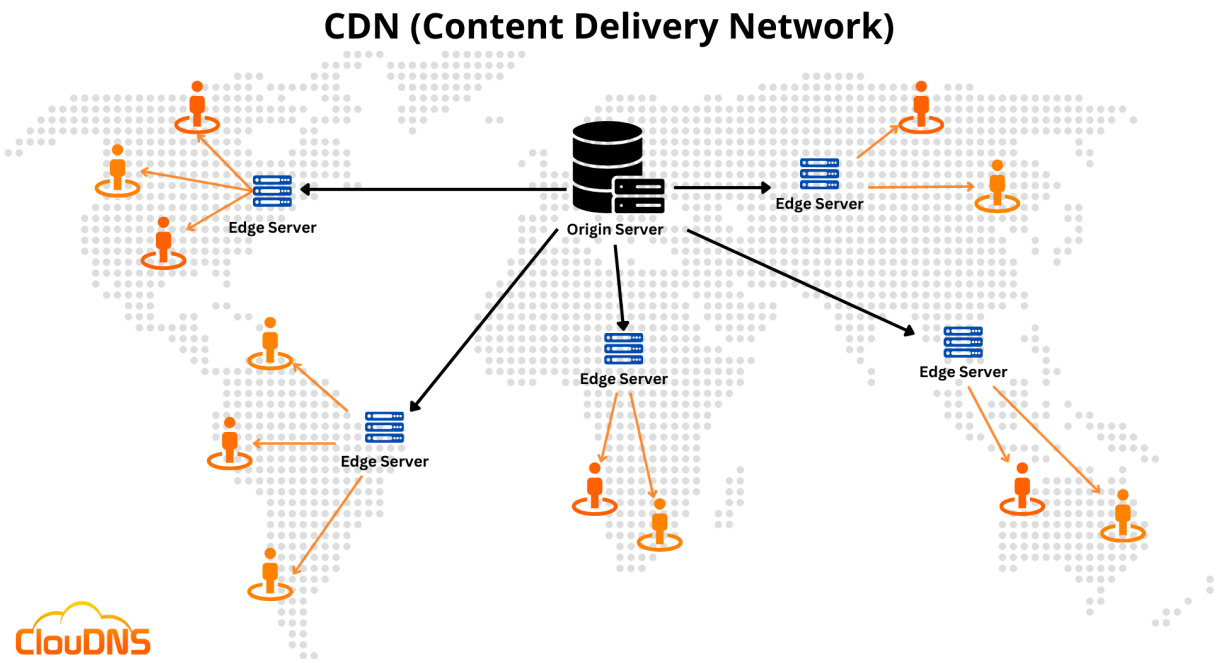


FIGURE 3.10 – Architecture simplifiée d'un CDN

En rapprochant le contenu des utilisateurs finaux, cette architecture réduit significativement les temps de chargement, en particulier pour les utilisateurs géographiquement éloignés du serveur d'origine.

3.8.2 Mise en cache et réduction de latence

La mise en cache consiste à conserver temporairement une copie d'une donnée fréquemment demandée, évitant de solliciter systématiquement la source d'origine et réduisant ainsi le temps de réponse global du système.

3.9 Résilience et continuité d'activité

3.9.1 Stratégies de reprise après sinistre

Quatre stratégies principales de reprise après sinistre sont généralement distinguées, se différenciant par leur objectif de temps de reprise (Recovery Time Objective (RTO)) et leur objectif

de point de reprise (Recovery Point Objective (RPO)) [26].

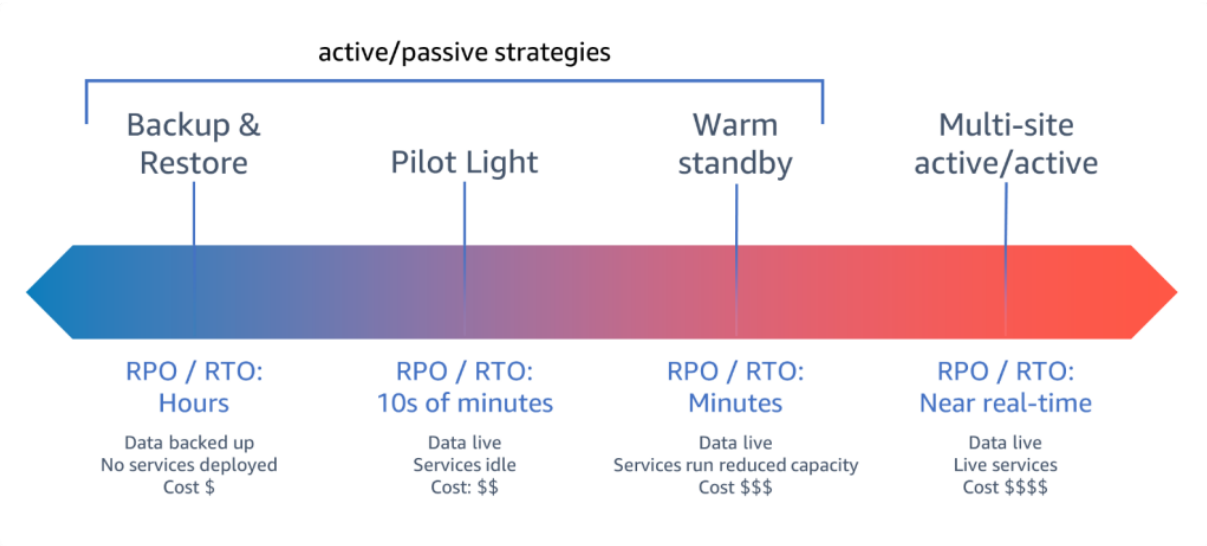


FIGURE 3.11 – Comparaison des stratégies de reprise après sinistre selon le RTO/RPO

Il convient de préciser que ce comparatif présente un cadre théorique général ; la stratégie Pilot Light n'a pas encore été déployée à ce stade du projet, mais constitue une évolution envisagée.

TABLE 3.3 – Synthèse des stratégies de reprise après sinistre

Stratégie	**RTO**	**RPO**	**Principe**
Backup & Restore	Élevé	Élevé	Restauration à partir de sauvegardes
Pilot Light	Moyen	Moyen	Environnement minimal maintenu en veille
Warm Standby	Faible	Faible	Environnement réduit actif en continu
Multi-Site Actif-Actif	Très faible	Très faible	Deux environnements pleinement actifs

3.9.2 Haute disponibilité multi-zone et multi-région

La haute disponibilité repose sur la répartition des ressources critiques sur plusieurs zones de disponibilité au sein d'une même région, voire sur plusieurs régions géographiques distinctes, afin de limiter l'impact d'une défaillance localisée.

3.10 Gestion financière du cloud (FinOps)

3.10.1 Modèles de tarification cloud

Les fournisseurs cloud proposent généralement plusieurs modèles de tarification : un paiement à l'usage sans engagement, adapté aux charges variables ; un tarif réduit en contrepartie d'un engagement sur une durée déterminée ; et une tarification opportuniste reposant sur des capacités disponibles à moindre coût, mais soumises à interruption.

3.10.2 Gouvernance des coûts et visibilité budgétaire

La gouvernance des coûts repose sur la définition de seuils budgétaires, associés à des mécanismes d'alerte automatique, ainsi que sur l'attribution systématique de métadonnées (*tags*) aux ressources déployées, permettant une répartition précise des coûts par projet, équipe ou composant.

3.11 Infrastructure as Code (IaC)

3.11.1 De la gestion manuelle à l'automatisation déclarative

La gestion manuelle d'une infrastructure via une interface graphique devient rapidement limitée à mesure que le nombre de ressources augmente, en raison de l'absence de traçabilité fiable des modifications et du risque élevé d'erreur humaine lors d'opérations répétitives.

3.11.2 Principes et avantages de l'IaC

- **Approche déclarative** : description de l'état cible, l'outil calcule les actions nécessaires.
- **Reproductibilité** : possibilité de recréer un environnement identique à volonté.
- **Versionnement** : suivi des modifications via un système de contrôle de version.

- **Modularité** : réutilisation de composants standardisés.

3.11.3 Gestion de l'état et collaboration

Un outil d'Infrastructure as Code maintient généralement un fichier d'état représentant la correspondance entre les ressources déclarées et les ressources réellement déployées. Dans un contexte comportant plusieurs composants indépendants ou plusieurs comptes cloud, cet état est généralement fragmenté en plusieurs unités distinctes (une par composant ou par compte), plutôt qu'en une séparation classique par environnement de développement.

3.12 Conclusion du chapitre

Ce chapitre a permis de poser les bases conceptuelles nécessaires à la compréhension de l'architecture développée dans ce projet. Les principes du Cloud Computing, les mécanismes de gouvernance multi-comptes, ainsi que les pratiques de sécurité, de conteneurisation et d'observabilité ont été présentés de manière générale, indépendamment de toute implémentation spécifique. Cette approche a permis d'établir un vocabulaire commun et une compréhension partagée des enjeux, préalable indispensable à toute décision architecturale éclairée.

Les notions abordées dans ce chapitre, notamment les pratiques DevSecOps, les architectures événementielles, la résilience, la gestion financière et l'Infrastructure as Code, seront désormais mobilisées de manière concrète dans le chapitre suivant. Celui-ci détaillera les technologies et outils effectivement retenus pour la mise en œuvre du projet, en justifiant chaque choix par rapport aux besoins identifiés au chapitre précédent et aux principes théoriques exposés ici.

Chapitre 4

TECHNOLOGIES ET OUTILS UTILISÉS

4 Technologies et outils utilisés

Le chapitre **??** a présenté les fondements théoriques qui structurent la démarche de conception de la landing zone Atlas FinTech. Ce chapitre présente les technologies et outils concrets qui ont permis de matérialiser ces fondements. Il ne s'agit pas de dresser un catalogue exhaustif mais d'expliciter, pour chaque brique retenue, son rôle dans l'architecture et la raison de son choix. Les technologies sont regroupées en six familles complémentaires : l'infrastructure as code, la gouvernance et l'identité, le réseau et l'exposition, le calcul et les conteneurs, le stockage et les données, la sécurité opérationnelle, l'observabilité et la messagerie, et enfin les outils DevSecOps.

4.1 Infrastructure as Code

L'ensemble des ressources déployées dans le cadre du projet est décrit sous forme de code, versionné et reproductible. Trois outils concourent à ce socle.

Terraform



Terraform (HashiCorp) est l'outil d'*Infrastructure as Code* déclaratif retenu pour l'ensemble des déploiements. Il permet de décrire les ressources cibles, d'en planifier les changements avec `terraform plan`, puis d'appliquer ces changements de manière transactionnelle avec `terraform apply`.

Git et GitHub



Git assure le versionnement de l'intégralité du code Terraform et applicatif. **GitHub** héberge les dépôts, trace l'historique des modifications, et sert de point d'entrée au pipeline CI/CD via GitHub Actions.

Backend d'état distant

Le backend d'état Terraform s'appuie sur deux services AWS combinés : un bucket **Amazon S3** pour le stockage du fichier `terraform.tfstate`, chiffré au repos et versionné, et une table **Amazon DynamoDB** pour le verrouillage transactionnel qui empêche deux exécutions concurrentes de `apply` sur le même état. Cette combinaison est décrite en détail au chapitre **??**.

4.2 Gouvernance et identité

AWS Organizations



AWS Organizations est le service de gestion centralisée d'une organisation multi-comptes AWS. Il permet la création de comptes, leur regroupement en unités organisationnelles, et l'application de *Service Control Policies* (SCP) qui définissent le périmètre d'actions autorisées.

AWS IAM



AWS Identity and Access Management (IAM) est le service fondamental de gestion des identités et des autorisations sur AWS. Il fournit les rôles, politiques et fournisseurs d'identité utilisés pour la fédération OIDC avec GitHub Actions et pour les rôles techniques des services (ECS, Lambda, etc.).

AWS IAM Identity Center



AWS IAM Identity Center (anciennement AWS SSO) constitue le point unique de connexion humaine à l'ensemble des comptes AWS de l'organisation. Il centralise l'authentification et permet l'attribution de *permission sets* par utilisateur et par compte.

4.3 Réseau et exposition

Amazon VPC



Amazon Virtual Private Cloud (VPC) fournit un réseau isolé dans lequel les ressources sont déployées. La segmentation en sous-réseaux publics, privés applicatifs et privés de données, répartis sur plusieurs zones de disponibilité, constitue la base de l'architecture réseau du projet.

Amazon Route 53



Amazon Route 53 est le service DNS managé d'AWS. Il est prévu comme point d'entrée pour l'exposition future de l'application via un nom de domaine personnalisé, ainsi que pour la résolution des noms internes de la landing zone.

Elastic Load Balancing



Elastic Load Balancing (ELB), plus précisément l'*Application Load Balancer* (ALB), assure la répartition du trafic HTTP entrant vers les tâches ECS. Il constitue le seul point d'exposition public de la charge de travail applicative.

Amazon CloudFront



Amazon CloudFront est le réseau de diffusion de contenu (CDN) d'AWS. Il sert les ressources statiques depuis un bucket S3 privé, protégé par *Origin Access Control* (OAC) et signature SigV4, et route le trafic dynamique vers l'ALB en amont.

AWS Certificate Manager



AWS Certificate Manager (ACM) provisionne, renouvelle et stocke automatiquement les certificats TLS utilisés pour le chiffrement en transit des flux applicatifs, notamment sur l'ALB et sur CloudFront.

4.4 Calcul et conteneurs

Amazon EC2



Amazon Elastic Compute Cloud (EC2) est le service fondamental de machines virtuelles d'AWS. Il n'est pas utilisé directement dans le projet, la couche de calcul reposant sur Fargate qui en abstrait entièrement la gestion. Il est mentionné ici à titre conceptuel comme substrat implicite des services de conteneurisation retenus.

Amazon ECS avec Fargate





Amazon Elastic Container Service (ECS) associé au mode de lancement **Fargate** permet d'exécuter des conteneurs sans provisionner ni gérer d'instances EC2. La charge opérationnelle liée aux systèmes d'exploitation est ainsi éliminée, en cohérence avec le principe de préférence des services managés.

Amazon ECR



Amazon Elastic Container Registry (ECR) héberge les images de conteneurs publiées par le pipeline CI/CD. Le registre est configuré en mode *immutable tags* et chiffré via une clé KMS gérée par le client.

AWS Lambda



AWS Lambda exécute du code de manière serverless, sans provisionnement de serveurs. Il est utilisé dans le pipeline de traitement asynchrone des *findings* de sécurité pour consommer les messages SQS et écrire dans CloudWatch Logs.

4.5 Stockage et données

Amazon S3



Amazon Simple Storage Service (S3) fournit un stockage objet durable et scalable. Il est utilisé pour le backend d'état Terraform, pour la journalisation centralisée CloudTrail, et pour l'hébergement des ressources statiques servies par CloudFront.

Amazon DynamoDB



Amazon DynamoDB est une base de données NoSQL managée. Elle est utilisée exclusivement pour le verrouillage transactionnel du fichier d'état Terraform, garantissant qu'aucune exécution concurrente de `terraform apply` ne peut corrompre l'état de l'infrastructure.

4.6 Sécurité opérationnelle

AWS KMS



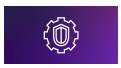
AWS Key Management Service (KMS) gère les clés cryptographiques utilisées pour chiffrer les données au repos. Le projet utilise des *Customer Managed Keys* (CMK) pour ECR, S3, CloudWatch Logs et les composants sensibles, offrant ainsi une politique de clé finement contrôlée.

AWS CloudTrail



AWS CloudTrail enregistre l'ensemble des appels d'API effectués sur l'organisation AWS et livre ces journaux dans un bucket S3 dédié, chiffré et versionné. Il constitue la source primaire de traçabilité et satisfait directement au besoin BNF3.

Amazon GuardDuty



Amazon GuardDuty est le service de détection de menaces qui analyse en continu les journaux CloudTrail, VPC Flow Logs et DNS. Ses *findings* alimentent le pipeline asynchrone de traitement des alertes de sécurité.

AWS Security Hub



AWS Security Hub agrège les constatations de sécurité provenant de GuardDuty, AWS Config et d'autres services, et mesure la conformité aux référentiels CIS et AWS Foundational Security Best Practices. Il fournit une vue centralisée de la posture de sécurité de l'organisation.

AWS Config



AWS Config enregistre les changements de configuration des ressources AWS et évalue leur conformité par rapport à des règles prédéfinies. Il fournit la piste d'audit sur laquelle repose Security Hub.

4.7 Observabilité et messagerie

Amazon CloudWatch



Amazon CloudWatch collecte les métriques et les journaux de l'ensemble des services AWS. Il déclenche des alarmes selon des seuils configurables et alimente les tableaux de bord d'observabilité opérationnelle.

Amazon SNS



Amazon Simple Notification Service (SNS) diffuse les notifications déclenchées par les alarmes CloudWatch et les alertes budgétaires vers les abonnés (courrier électronique principalement).

Amazon EventBridge



Amazon EventBridge est le service de routage d'événements. Il capture les *findings* publiés par GuardDuty et les transmet à la file SQS pour un traitement asynchrone, découplant ainsi la source d'événements de son consommateur.

Amazon SQS



Amazon Simple Queue Service (SQS) fournit une file de messages managée qui découple les producteurs des consommateurs. Le projet utilise une file principale associée à une *Dead Letter Queue* pour les messages n'ayant pu être traités après plusieurs tentatives.

AWS Budgets



AWS Budgets définit des seuils budgétaires au niveau de l'organisation et déclenche des notifications SNS lorsque ces seuils sont atteints ou approchés. Il constitue le pilier de la démarche FinOps du projet.

4.8 Outils DevSecOps

GitHub Actions



GitHub Actions est la plateforme de CI/CD intégrée à GitHub. Elle héberge la pipeline qui construit les images Docker, exécute les scans de sécurité, publie les artefacts sur ECR et déclenche le déploiement sur ECS. L'authentification auprès d'AWS repose sur la fédération OIDC, éliminant ainsi tout besoin de credentials statiques.

Docker



Docker est utilisé pour la conteneurisation des applications. Les images construites lors du pipeline CI/CD sont signées, scannées, puis publiées sur ECR avant déploiement sur ECS Fargate.

Trivy



Trivy est un scanner de vulnérabilités open-source utilisé dans la pipeline CI/CD pour analyser les images Docker avant leur publication. Il constitue le *Security Gate* bloquant : toute vulnérabilité de sévérité CRITICAL détectée interrompt le pipeline et notifie le développeur, empêchant la publication d'une image compromise sur ECR.

4.9 Synthèse

Le tableau 4.1 récapitule l'ensemble des technologies et outils retenus, en précisant pour chacun le rôle joué dans le projet et le chapitre où sa mise en œuvre est détaillée.

TABLE 4.1 – Synthèse des technologies et outils du projet Atlas FinTech

Technologie / outil	**Rôle dans le projet**	**Phase**
Terraform, Git, GitHub	IaC déclarative, versionnement	1–11
S3 + DynamoDB	Backend d'état Terraform	1
AWS Organizations	Structure multi-comptes, SCP	2
IAM, IAM Identity Center	Identités humaines et machine	2
VPC, Route 53, ELB, ACM	Réseau, DNS, exposition sécurisée	3, 10
ECS Fargate, ECR	Calcul conteneurisé, registre d'images	4
Lambda	Traitement asynchrone des findings	11
KMS	Chiffrement au repos	5
CloudTrail, GuardDuty, Security Hub, Config	Sécurité opérationnelle	5, 11
CloudWatch, SNS	Observabilité, alertes	6
EventBridge, SQS	Découplage des flux d'événements	11
CloudFront	CDN pour les ressources statiques	10
S3 (stockage objet)	Hébergement des artefacts statiques	9
AWS Budgets	Gouvernance FinOps	8
GitHub Actions, Docker, Trivy	Pipeline CI/CD et scan de sécurité	7

Conclusion du chapitre

Ce chapitre a présenté l'ensemble des technologies et outils mobilisés dans la construction de la landing zone Atlas FinTech, en les regroupant par grande famille fonctionnelle : infrastructure as code, gouvernance et identité, réseau et exposition, calcul et conteneurs, stockage et données, sécurité opérationnelle, observabilité et messagerie, outils DevSecOps. Cette présentation n'a pas vocation à se substituer à la documentation officielle des outils, mais à situer chaque brique dans son rôle spécifique au sein du projet et à justifier son inclusion au regard des besoins formalisés au chapitre 2.

L'articulation entre ces briques, les décisions d'architecture qui en tirent parti, et les patterns qui les combinent constituent l'objet du chapitre 5, qui présente la conception détaillée de la landing zone.

Chapitre 5

Conception de la landing zone Atlas FinTech

5 Conception de la landing zone Atlas FinTech

Les chapitres précédents ont posé le cadre du projet (chapitre 1), formalisé les besoins fonctionnels et non fonctionnels (chapitre 2), exposé les fondements théoriques mobilisés (chapitre **??**) et présenté les technologies et services retenus (chapitre 4). Le présent chapitre traduit ces éléments en une architecture cohérente, organisée en couches successives dont chacune apporte une réponse concrète à un ensemble de besoins identifiés.

La démarche adoptée consiste à justifier chaque décision de conception en explicitant les alternatives considérées, les critères de choix et les référentiels mobilisés, plutôt qu'à décrire mécaniquement les composants retenus. Cette approche vise à faire du chapitre une véritable étude de conception et non un simple catalogue d'architecture. La section 5.1 présente la vision d'ensemble et le principe de conception en couches. Les sections 5.2 à 5.6 détaillent chaque couche : gouvernance et organisation multi-comptes, architecture réseau, architecture applicative, pipeline DevSecOps, et sécurité opérationnelle. La section 5.7 présente la matrice de traçabilité entre les besoins formulés au chapitre 2 et les décisions de conception retenues.

5.1 Vision globale et principe de conception en couches

La conception d'une landing zone ne peut être abordée comme un problème monolithique. Chaque décision de sécurité, de coût ou de disponibilité prise à un niveau conditionne les degrés de liberté disponibles aux niveaux inférieurs. Il est donc essentiel de structurer la conception selon une progression descendante, du plus général au plus spécifique.

L'architecture retenue s'organise en cinq couches complémentaires, chacune traitant une préoccupation distincte :

Gouvernance et organisation. Cette couche définit la structure multi-comptes, les frontières d'autorité et les mécanismes d'authentification humaine. Elle répond aux besoins BF1, BF2, BNF3 et BNF6.

Architecture réseau. Cette couche organise le VPC en sous-réseaux publics, privés applicatifs et privés de données, répartis sur trois zones de disponibilité. Elle répond aux besoins BF3, BNF1 et BNF2.

Architecture applicative. Cette couche définit l'exposition sécurisée de la charge de travail via un ALB, ECS Fargate et CloudFront, et satisfait les besoins BF3, BNF1 et BNF2.

Pipeline DevSecOps. Cette couche traite l'automatisation du cycle de vie applicatif via GitHub Actions et un *security gate* bloquant. Elle satisfait le besoin BF4.

Sécurité opérationnelle. Cette couche assure la détection continue, la supervision et le traitement asynchrone des *findings*. Elle répond aux besoins BF5, BNF3 et BNF5.

Cette organisation en couches respecte le principe de séparation des préoccupations formulé dans le pilier *Security* du Well-Architected Framework [22], et permet de restreindre la propagation d'un défaut de conception à la couche concernée.

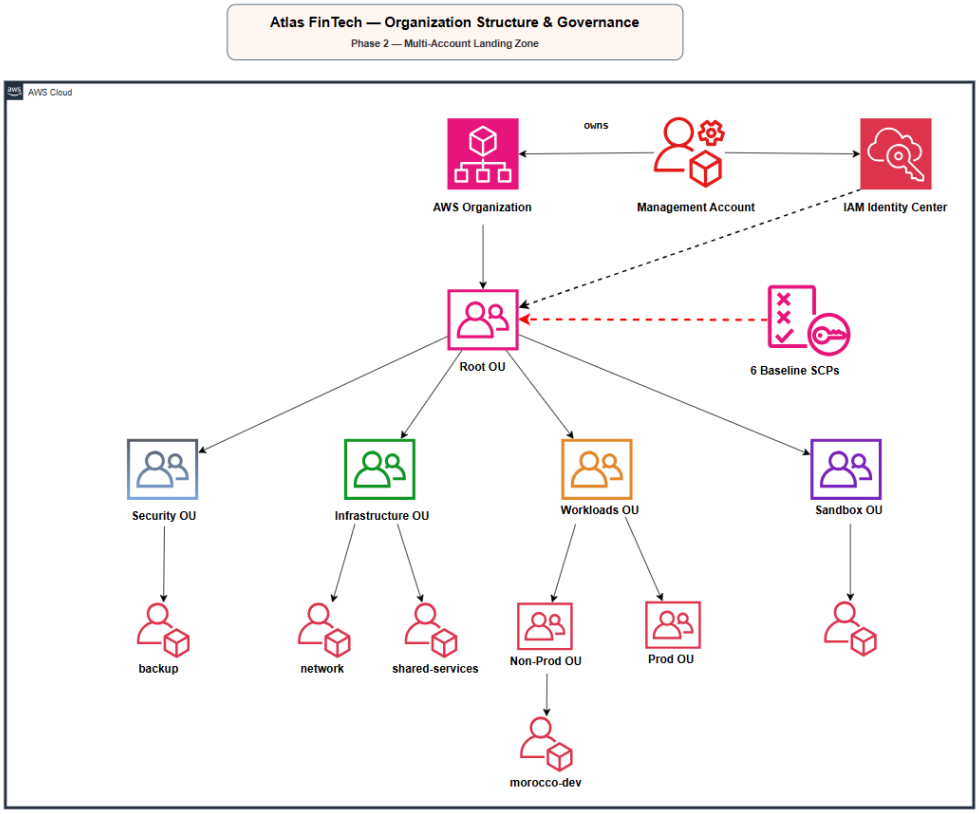


FIGURE 5.1 – Vue d'ensemble de l'architecture Atlas FinTech en cinq couches.

5.2 Gouvernance et organisation multi-comptes

La couche de gouvernance constitue le socle sur lequel reposent toutes les autres. Elle définit l'unité fondamentale d'isolation retenue et la manière dont les identités humaines et machine accèdent aux ressources.

5.2.1 Choix de l'unité d'isolation : le compte AWS

La première décision structurante consiste à choisir l'unité d'isolation. Trois options ont été considérées :

- Utiliser un compte AWS unique et cloisonner par IAM et par tags.
- Utiliser un compte par environnement (dev, prod).
- Utiliser un compte par fonction, regroupé en unités organisationnelles (OU).

La première option est explicitement déconseillée par AWS et par le CIS AWS Foundations Benchmark [**cis-aws**], car un incident IAM peut alors compromettre l'ensemble du périmètre. La seconde offre une isolation acceptable entre environnements mais ne sépare pas les fonctions transverses (audit, sécurité, réseau) de la charge de travail elle-même.

La troisième option, retenue, consiste à organiser les comptes selon leur fonction :

- **Compte de gestion.** Il centralise la facturation, héberge AWS Organizations et applique les SCP. Aucune charge applicative ne s'y exécute.
- **Compte des services partagés.** Il héberge les services d'audit et de sécurité opérationnelle (CloudTrail, GuardDuty, Security Hub, AWS Config), délégués depuis le compte de gestion.
- **Compte de la charge de travail.** Il héberge l'ALB, ECS Fargate et les ressources applicatives.
- **Compte de journalisation.** Il centralise les journaux CloudTrail dans un bucket S3 dédié.

Cette structure satisfait BF1 (structure multi-comptes), BNF3 (traçabilité par centralisation des

journaux) et BNF6 (contention des rayons d'action, puisqu'une erreur dans un compte ne peut affecter les autres).

5.2.2 Encadrement par Service Control Policies

Les SCP constituent le mécanisme par lequel le compte de gestion impose des limites d'autorisation aux comptes membres, indépendamment des politiques IAM locales. Elles n'accordent aucun droit ; elles en restreignent l'exercice.

Trois SCP principales ont été retenues :

- Interdire la désactivation de CloudTrail dans tous les comptes membres (mitigation directe de R4).
- Interdire l'usage de régions autres qu'*eu-west-1* (respect du RGPD).
- Interdire la création de ressources publiques par défaut sur S3 (mitigation directe de R5).

5.2.3 Fédération d'identité humaine via IAM Identity Center

L'accès humain à l'organisation transite exclusivement par IAM Identity Center. Cette centralisation présente trois avantages : elle évite la création d'utilisateurs IAM dans chaque compte, elle impose un point unique de gestion des cycles de vie des accès, et elle permet l'application uniforme du MFA.

Deux *permission sets* ont été définis :

TABLE 5.1 – Permission sets définis dans IAM Identity Center

Permission set	**Portée d'action**	**Compte**
AdministratorAccess	Gestion complète des ressources	Tous
ReadOnlyAccess	Consultation des logs, findings, ressources	Services partagés, journalisation

Cette architecture satisfait BF2 (fédération d'identités) et contribue à BNF3 (traçabilité, chaque action étant attribuable à un utilisateur nommé).

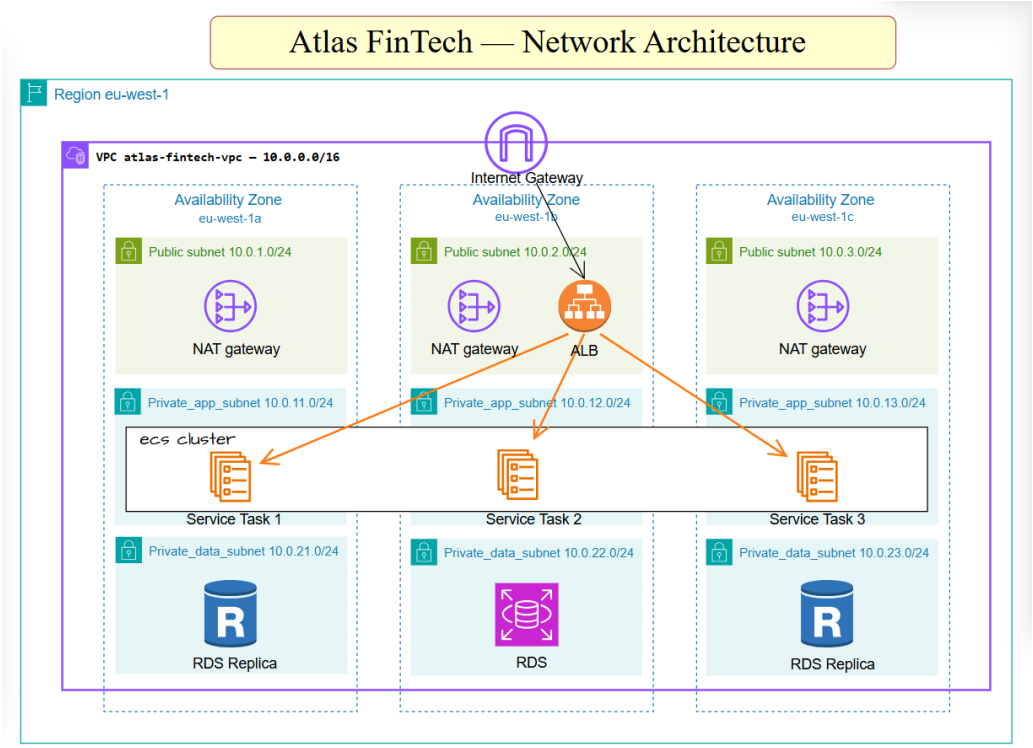


FIGURE 5.2 – Structure organisationnelle et flux d'authentification humaine.

5.3 Architecture réseau

L'architecture réseau se déploie dans le compte de la charge de travail. Elle vise à concilier l'exposition contrôlée d'une charge applicative accessible depuis Internet avec l'isolation stricte des composants sensibles.

5.3.1 Structure du VPC et segmentation en trois niveaux

Le VPC retenu segmente l'espace d'adressage en trois niveaux, chacun répliqué sur trois zones de disponibilité :

- **Sous-réseaux publics.** Ils hébergent les NAT Gateways et l'ALB. Seuls ces sous-réseaux disposent d'une route par défaut vers l'Internet Gateway.
- **Sous-réseaux privés applicatifs.** Ils hébergent les tâches ECS Fargate. Leur trafic sortant vers Internet transite obligatoirement par les NAT Gateways.

- **Sous-réseaux privés de données.** Ils sont réservés à une future base RDS et ne disposent d'aucune route vers Internet.

Cette segmentation matérialise le principe de *defense in depth* en imposant un chemin de compromission croissant pour atteindre les données depuis l'extérieur.

5.3.2 Multi-AZ et haute disponibilité

Chaque niveau de sous-réseau est répliqué sur *eu-west-1a*, *eu-west-1b* et *eu-west-1c*. Une NAT Gateway est déployée dans chacune des trois AZ, plutôt qu'une NAT unique routant tout le trafic sortant, afin d'éviter qu'une panne d'AZ ne rende toutes les tâches privées incapables de communiquer avec l'extérieur.

Cette décision a un coût réel (trois NAT Gateways facturées en continu) mais constitue la seule manière de satisfaire pleinement le BNF2 (haute disponibilité intra-région).

5.3.3 Contrôle des flux par Security Groups

L'exposition des composants est encadrée par une chaîne de Security Groups en cascade :

$$\text{Internet} \rightarrow \text{ALB SG} \rightarrow \text{ECS SG} \rightarrow \text{RDS SG}$$

Chaque SG accepte du trafic entrant uniquement en provenance du SG immédiatement en amont, jamais depuis un CIDR ouvert. Cette approche, recommandée par l'AWS FSBP [**aws-fsbp**], présente l'avantage de rendre la chaîne d'autorisation lisible à la simple inspection des SG.

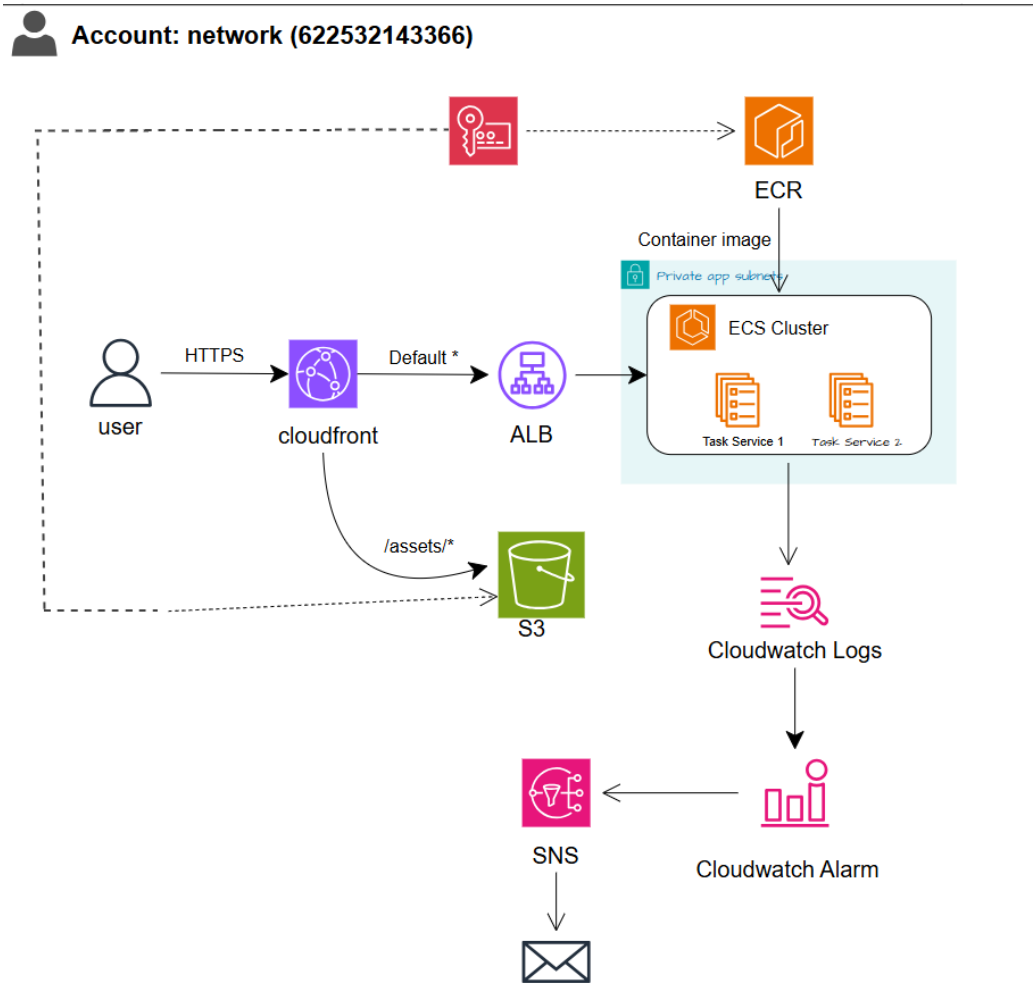


FIGURE 5.3 – Architecture réseau du VPC Atlas FinTech en trois niveaux.

5.4 Architecture applicative

La couche applicative articule les composants qui traitent le trafic utilisateur. Elle repose sur trois éléments principaux : CloudFront comme point d'entrée global, l'ALB comme équilibreur de charge intra-région, et ECS Fargate comme couche d'exécution.

5.4.1 Choix de Fargate contre EC2 et Kubernetes

Trois options de plateforme de conteneurs ont été considérées : EC2 avec ECS classique, Amazon EKS (Kubernetes managé), et ECS Fargate.

TABLE 5.2 – Comparaison des plateformes de conteneurs sur AWS

	ECS + EC2	**EKS**	**ECS + Fargate**
Charge opérationnelle	Élevée (patch OS)	Élevée (control plane)	Nulle
Courbe d'apprentissage	Modérée	Très élevée	Faible
Coût à faible charge	Modéré	Élevé (control plane)	Faible
Intégration IAM native	Complète	Partielle (IRSA)	Complète

Fargate a été retenu car il offre la meilleure adéquation avec les contraintes du projet : absence de charge opérationnelle, coût maîtrisé à faible charge, et intégration IAM native via les rôles d'exécution et de tâche.

5.4.2 Deux rôles IAM distincts pour ECS

La séparation des rôles IAM d'ECS suit le principe de moindre privilège :

- Le **rôle d'exécution** est utilisé par l'agent Fargate pour tirer les images depuis ECR et écrire les journaux dans CloudWatch. Il n'a aucune permission métier.
- Le **rôle de tâche** est assumé par le conteneur lui-même et porte les permissions métier (accès aux buckets S3 applicatifs, aux paramètres SSM, etc.).

Cette séparation garantit qu'une compromission du conteneur ne donne pas accès aux permissions techniques d'orchestration.

5.4.3 Chiffrement au repos et en transit

Toutes les données au repos manipulées par la couche applicative sont chiffrées par des clés KMS gérées par le client (CMK) :

- ECR chiffre les images conteneurs avec une CMK dédiée.
- CloudWatch Logs chiffre les journaux avec une CMK dédiée.
- S3 chiffre les buckets applicatifs et le bucket d'artefacts CloudFront avec une CMK.

Le chiffrement en transit repose sur TLS 1.2 minimum via ACM, tant sur l'ALB que sur CloudFront.

5.4.4 Distribution CloudFront et OAC

CloudFront joue un double rôle : accélération de la distribution statique et masquage de l'origine ALB. Deux origines sont configurées :

- L'ALB pour le trafic dynamique (path `/*`).
- Un bucket S3 privé pour les ressources statiques (path `/assets/*`), accessible exclusivement via *Origin Access Control* (OAC) et signature SigV4.

Le choix de l'OAC plutôt que de l'ancienne OAI (*Origin Access Identity*) répond à sa recommandation actuelle par AWS et à sa compatibilité avec SSE-KMS, absente d'OAI.

Ce dispositif satisfait BF3 (exposition sécurisée) et mitige R5 (fuite depuis S3).

5.5 Pipeline DevSecOps

La pipeline CI/CD constitue la seule voie de mise à jour de la couche applicative. Sa conception vise trois objectifs : éliminer tout credential AWS statique, imposer un *security gate* bloquant avant publication, et garantir la reproductibilité du déploiement.

5.5.1 Fédération OIDC avec GitHub Actions

L'authentification de la pipeline auprès d'AWS repose sur la fédération OIDC. GitHub Actions est déclaré comme fournisseur d'identité auprès d'IAM ; un rôle IAM dédié définit les conditions sous lesquelles il peut être assumé (dépôt, branche, environnement).

Cette approche remplace le stockage de clés AWS dans les GitHub Secrets, qui présente deux inconvénients majeurs : la nécessité de rotation manuelle et le risque d'exfiltration en cas de compromission du dépôt. La fédération OIDC élimine ces deux risques (mitigation directe de R6).

5.5.2 Security gate Trivy bloquant

Le pipeline exécute successivement quatre étapes :

1. Construction de l'image Docker.
2. Scan Trivy de l'image construite.
3. Publication ECR (uniquement si aucun CRITICAL détecté).
4. Déclenchement du déploiement ECS.

Le *gate* Trivy est configuré pour bloquer sur toute vulnérabilité de sévérité CRITICAL. Cette configuration constitue un compromis explicite : trop stricte (bloquer sur HIGH également), elle générerait des faux positifs récurrents qui inciteraient à contourner la règle ; trop laxiste (bloquer uniquement sur les CVE exploitées), elle laisserait passer des images à risque. Le seuil CRITICAL correspond aux vulnérabilités disposant d'un exploit connu et constitue le point d'équilibre le plus courant dans les guides AWS Prescriptive Guidance [**aws-prescriptive**].

Ce mécanisme satisfait BF4 (automatisation du cycle de vie applicatif) et mitige R3 (déploiement d'une image vulnérable).

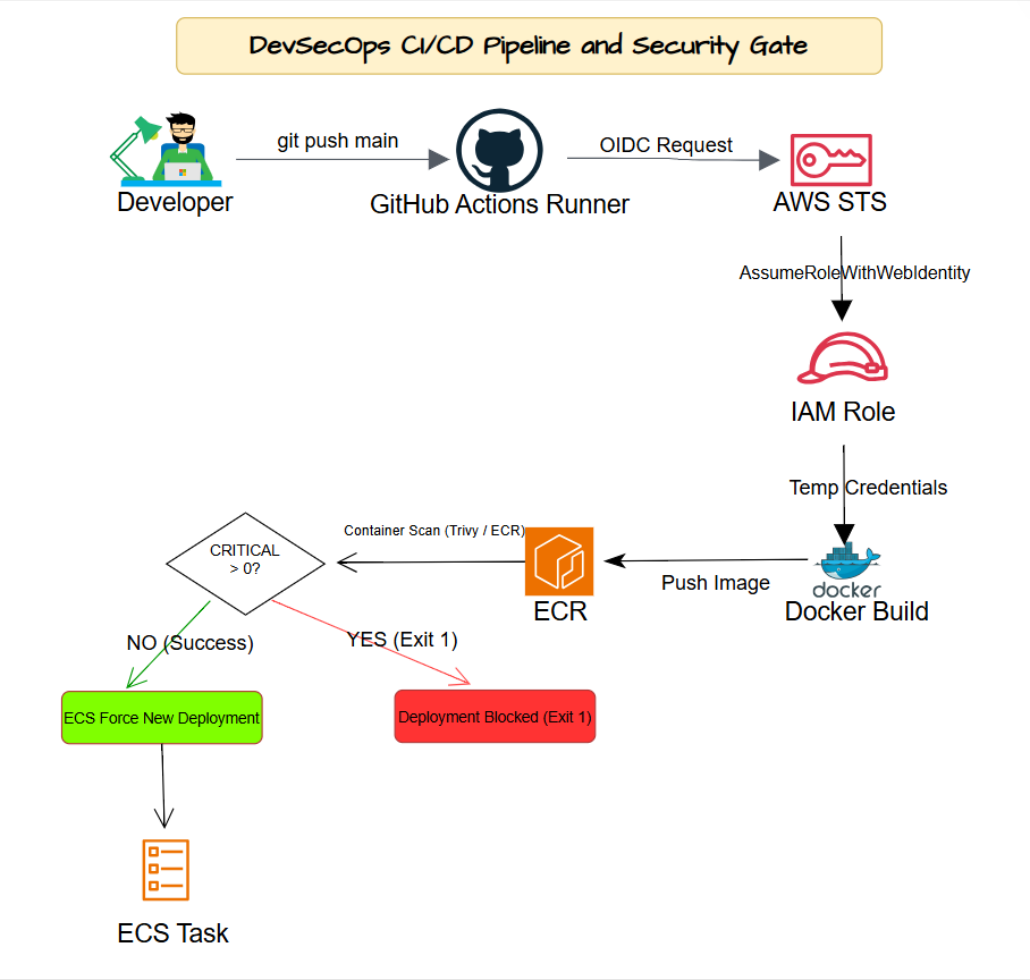


FIGURE 5.4 – Pipeline DevSecOps : de la modification du code au déploiement ECS.

5.6 Sécurité opérationnelle

La couche de sécurité opérationnelle assure la détection continue des menaces, la centralisation des *findings* et leur traitement asynchrone. Elle repose sur trois piliers : la journalisation d'audit centralisée, l'agrégation des *findings*, et le découplage du traitement.

5.6.1 Journalisation d'audit centralisée

CloudTrail est activé au niveau de l'organisation et livre l'ensemble de ses journaux dans un bucket S3 dédié, hébergé dans le compte de journalisation. Ce bucket applique trois protections :

- Chiffrement au repos avec une CMK dédiée.

- Versioning activé pour empêcher toute suppression logique.
- Politique bloquant explicitement toute suppression, même par l'administrateur du compte.

Ce dispositif satisfait BNF3 (traçabilité et auditabilité) et mitige R4 (altération du journal d'audit).

5.6.2 Délégation de GuardDuty et Security Hub

Plutôt que d'activer GuardDuty et Security Hub dans le compte de gestion, ces services sont délégués au compte des services partagés, qui devient *delegated administrator*. Cette délégation présente deux avantages : elle sépare le rôle d'administration organisationnelle du rôle d'administration sécurité, et elle concentre les *findings* de tous les comptes dans un point unique.

Cette architecture applique le principe de séparation des responsabilités du pilier *Security* du Well-Architected Framework [22].

5.6.3 Découplage GuardDuty → SQS → Lambda

Les *findings* de GuardDuty sont routés par EventBridge vers une file SQS, consommée par une fonction Lambda dédiée. Cette architecture asynchrone présente deux avantages majeurs par rapport à une invocation directe de Lambda depuis EventBridge :

- Elle absorbe les pics d'événements sans perte (mitigation de R7).
- Elle permet de retraiter automatiquement les *findings* ayant échoué, via une *Dead Letter Queue*.

Cette conception satisfait BF5 (supervision continue) et applique le principe de découplage asynchrone recommandé pour la résilience opérationnelle.

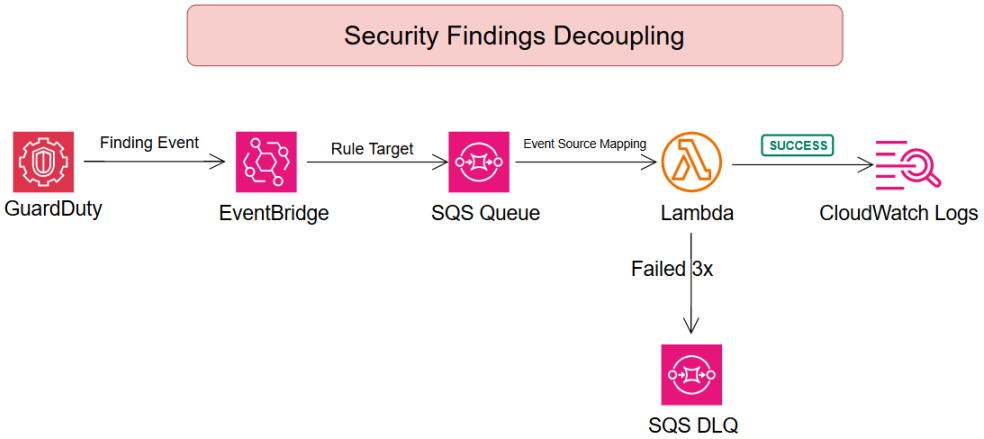


FIGURE 5.5 – Flux de traitement asynchrone des findings GuardDuty.

5.7 Traçabilité besoins → conception

La table 5.3 met en regard chaque besoin formulé au chapitre 2 avec les décisions de conception qui y répondent dans le présent chapitre. Cette traçabilité permet de vérifier qu'aucun besoin n'est laissé sans réponse et que chaque décision est rattachée à une exigence formelle.

TABLE 5.3 – Traçabilité besoins → décisions de conception

Besoin	Décision de conception	Section
BF1	Organisation multi-comptes AWS avec OU et SCP	5.2
BF2	Fédération humaine via IAM Identity Center + MFA	5.2
BF3	Exposition via ALB, CloudFront, OAC	5.3, 5.4
BF4	Pipeline OIDC avec security gate Trivy	5.5
BF5	Découplage GuardDuty → SQS → Lambda	5.6
BNF1	Chiffrement KMS transversal, moindre privilège	5.3, 5.4
BNF2	Multi-AZ complet (subnets, NAT, ECS)	5.3
BNF3	CloudTrail organisation, bucket verrouillé	5.6
BNF4	Infrastructure entièrement décrite en Terraform	5.2 – 5.6
BNF5	Budget org-wide, cost allocation tags	5.6
BNF6	Séparation par comptes et modules Terraform	5.2

Conclusion du chapitre

Ce chapitre a exposé la conception détaillée de la landing zone Atlas FinTech, structurée en cinq couches complémentaires : gouvernance et organisation multi-comptes, architecture ré-

seau, architecture applicative, pipeline DevSecOps et sécurité opérationnelle. Pour chacune, les alternatives ont été explicitement considérées, les critères de choix ont été énoncés, et les décisions retenues ont été rattachées aux besoins formulés au chapitre 2 et aux référentiels mobilisés au chapitre **??**.

La matrice de traçabilité présentée en section 5.7 confirme que chacun des onze besoins identifiés dispose d'une réponse dans la conception. Cette conception constitue le plan à partir duquel le chapitre **??** présente la mise en œuvre effective, phase par phase, en documentant les décisions d'implémentation, les difficultés rencontrées et les vérifications menées à chaque étape.

Chapitre 6

Mise en œuvre de la fondation cloud Atlas FinTech

6 Mise en œuvre de la fondation cloud Atlas FinTech

Ce chapitre présente la mise en œuvre concrète de la landing zone AWS sécurisée Atlas FinTech Secure Landing Zone, depuis la préparation de l'environnement de travail jusqu'aux capacités additionnelles construites autour de la plateforme applicative. Contrairement au chapitre précédent, dédié à la conception, celui-ci se concentre sur la démarche de construction : outillage retenu, extraits de code Terraform représentatifs, difficultés techniques rencontrées et vérification systématique des résultats. Les onze phases du projet se répartissent en quatre blocs logiques : la préparation de l'environnement (§6.1), la construction des fondations — gouvernance, contrôle, audit, réseau (§6.2) —, la plateforme applicative et son automatisation CI/CD (§6.3), et enfin les capacités additionnelles — coûts, stockage, distribution de contenu, découplage asynchrone (§6.4).

Trois principes récurrents traversent l'ensemble de ces phases. Le code Terraform est organisé de manière modulaire, chaque capacité disposant de son propre état et consommant à la demande les sorties des précédentes via le *remote state*, ce qui isole les rayons d'action entre déploiements. À partir de la troisième phase, tous les déploiements s'effectuent via l'assomption du rôle `OrganizationAccountAccessRole`, évitant la multiplication d'identifiants à longue durée de vie. Enfin, la réussite d'une commande de déploiement ne suffit jamais à valider une capacité : plusieurs phases provoquent volontairement des conditions d'erreur pour prouver que les mécanismes de détection et de blocage se déclenchent réellement.

Chaque section adopte une structure interne uniforme — objectif, choix de conception, éléments d'implémentation, points de vigilance et vérification des résultats — et se referme sur un bref bilan assurant la transition vers le bloc suivant. La présentation débute avec la préparation de l'environnement de travail, socle sans lequel aucune phase ultérieure n'aurait pu être

automatisée.

6.1 Préparation de l'environnement de travail

Avant qu'une seule ressource ne puisse être déployée par Terraform, un ensemble de prérequis doit être satisfait au niveau du compte AWS lui-même : identités humaine et machine correctement séparées, garde-fou budgétaire actif, et infrastructure de stockage de l'état Terraform provisionnée de manière durable et concurrente. Ces prérequis, dont l'automatisation complète est structurellement impossible puisqu'ils constituent précisément les fondations sur lesquelles l'automatisation vient ensuite s'appuyer, font l'objet des deux premières phases du projet. Bien qu'aucune des deux phases suivantes ne réponde directement à un besoin fonctionnel ou non fonctionnel du chapitre 2, elles conditionnent BNF4 (automatisation et reproductibilité), qui ne peut être satisfait sans un backend d'état Terraform durable.

6.1.1 Phase 00 – Bootstrap du compte AWS

Objectif

Cette phase, entièrement manuelle, vise à retirer le compte AWS de son état par défaut, dans lequel toutes les opérations passent par l'utilisateur *root*, pour le placer dans une configuration compatible avec les bonnes pratiques de séparation des rôles et de contrôle des coûts. Elle produit les deux identités qui interviendront durant toute la suite du projet ainsi qu'un premier garde-fou budgétaire déclenché avant qu'un dépassement significatif ne survienne.

Mise en œuvre et vérification

L'ensemble des opérations de cette phase s'effectue via la console AWS et l'outil `aws configure`, aucun code Terraform n'étant encore utilisable à ce stade. Le profil CLI `terraform-deployer` est configuré sur le poste local, ce qui permet aux commandes Terraform ultérieures d'assumer cette identité machine par défaut.

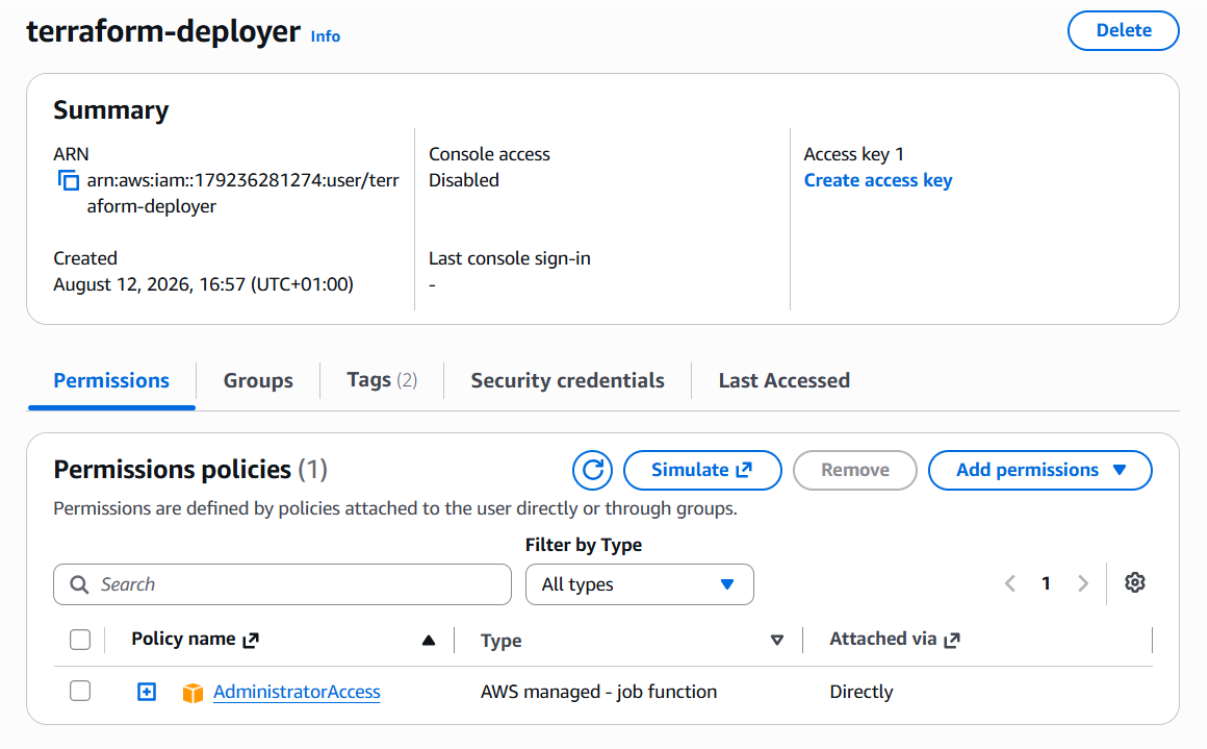


FIGURE 6.1 – Utilisateur IAM `terraform-deployer` créé sans accès console, destiné exclusivement à l'automatisation.

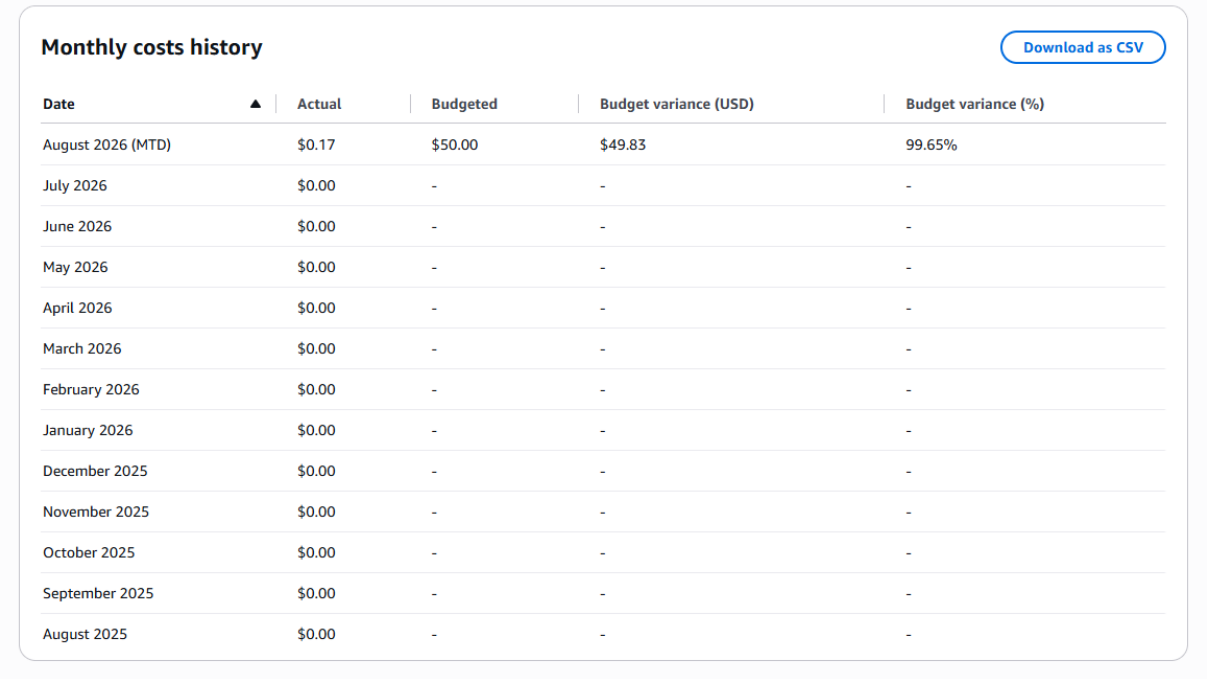


FIGURE 6.2 – Alerte budgétaire configurée à 85 % du seuil de 50 USD, avec notification par courrier électronique.

6.1.2 Phase 01 – Backend Terraform

Objectif

Le backend Terraform désigne l'emplacement où l'outil stocke son fichier d'état, c'est-à-dire la représentation persistante de l'infrastructure déployée. Le comportement par défaut de Terraform consiste à stocker cet état dans un fichier local, ce qui devient inopérant dès qu'un projet dépasse le poste d'un développeur unique. Cette phase provisionne un backend distant durable, chiffré et compatible avec un accès concurrent contrôlé.

Éléments d'implémentation

Le compartiment S3 et la table DynamoDB constituent l'ensemble des ressources créées durant cette phase. L'extrait ci-dessous en présente la déclaration Terraform :

```
resource "aws_s3_bucket" "tfstate" {
  bucket = "atlas-fintech-tfstate-${var.account_id}"
}

resource "aws_s3_bucket_versioning" "tfstate" {
  bucket = aws_s3_bucket.tfstate.id
  versioning_configuration {
    status = "Enabled"
  }
}

resource "aws_s3_bucket_public_access_block" "tfstate" {
  bucket                  = aws_s3_bucket.tfstate.id
  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}

resource "aws_dynamodb_table" "tfstate_lock" {
  name         = "atlas-fintech-tfstate-lock"
  billing_mode = "PAY_PER_REQUEST"
```

```
  hash_key      = "LockID"
  attribute {
    name = "LockID"
    type = "S"
  }
}
```

Listing 6.1 – Provisionnement du backend d'état Terraform.

Point de vigilance : séparation par dossiers

Le projet adopte dès cette phase une convention d'organisation qui prendra toute son importance dans les phases ultérieures : chaque capacité fonctionnelle fait l'objet d'un dossier Terraform distinct, avec son propre fichier d'état. Cette séparation limite le rayon d'action de toute opération `terraform apply` ou `terraform destroy` au périmètre du dossier concerné, ce qui protège les ressources déployées par les autres phases en cas d'erreur locale.

Vérification

Les figures ci-dessous confirment que la table DynamoDB de verrouillage est bien active et que le fichier d'état a été effectivement migré vers le backend S3 : la commande `terraform state list` lit désormais son état depuis le compartiment distant.

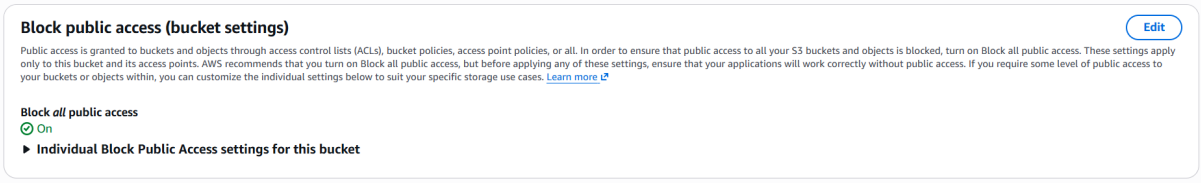


FIGURE 6.3 – Table DynamoDB `atlas-fintech-tfstate-lock` active, prête à sérialiser les opérations Terraform concurrentes.

```
PS C:\Users\hiba\Documents\AWS_SAA_projects\01-aws-secure-landing-zone\infra\bootstrap> terraform init -migrate-state
Initializing the backend...

Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v5.100.0


Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

FIGURE 6.4 – Sortie de `terraform state list` après migration réussie de l'état local vers le backend S3.

Bilan de la section

Ces deux phases préparatoires sont volontairement minimales : elles n'introduisent aucune capacité fonctionnelle au sens métier du terme, mais elles rendent possible tout ce qui suit. Elles matérialisent également deux principes qui structureront le reste du projet : la séparation stricte des identités humaine et machine, et la contention explicite des rayons d'action des opérations d'infrastructure. Sur ces bases, la section suivante peut aborder la construction des véritables fondations de la landing zone : la gouvernance multi-comptes et le socle réseau.

6.2 Fondations de la landing zone

Les fondations d'une landing zone AWS reposent sur deux couches indissociables : une couche organisationnelle, qui définit comment les comptes AWS sont structurés, comment les identités s'y authentifient, quelles opérations sont universellement interdites, et comment l'ensemble des activités est journalisé à des fins d'audit ; et une couche réseau, qui pose la topologie physique et logique dans laquelle les charges applicatives viendront s'exécuter. Ces deux couches font l'objet des deux phases suivantes. Ces deux phases matérialisent les décisions de conception relatives à BF1, BF2, BNF2, BNF3 et BNF6, telles que synthétisées dans la matrice de traçabilité (tableau 5.3).

6.2.1 Phase 02 – Landing zone multi-comptes

Objectif

Cette phase construit la structure organisationnelle complète du projet : création de l'organisation AWS, définition de la hiérarchie des unités organisationnelles (*Organizational Units*, ou OUs), provisionnement des comptes membres, déploiement des politiques de contrôle de service (*Service Control Policies*, ou SCPs), mise en place de la fédération d'identité via IAM Identity Center, activation de la journalisation d'audit à l'échelle de l'organisation via CloudTrail, et déploiement des services de détection de menaces GuardDuty et Security Hub. Il s'agit de la phase la plus dense du projet, tant par le nombre de sous-composants qu'elle réunit que par les décisions structurantes qu'elle fige pour la suite.

Hiérarchie organisationnelle et comptes membres

La hiérarchie d'unités organisationnelles adoptée reflète une séparation par nature de charge de travail : une OU *Security* destinée à héberger les comptes d'audit et de sauvegarde, une OU *Infrastructure* pour les comptes techniques partagés (réseau et services communs), une OU *Workloads* scindée en sous-OUs *Non-Prod* et *Prod* pour appliquer des politiques différenciées, et une OU *Sandbox* pour l'expérimentation. Cette hiérarchie applique le principe de tiering des SCPs, qui repose sur le mécanisme d'héritage des politiques : les règles définies au niveau du *Root* s'appliquent à tous les comptes, celles définies sur *Prod* durcissent l'ensemble sans en assouplir aucun aspect, et *Sandbox* reçoit une politique dédiée limitant l'expérimentation à un périmètre budgétaire acceptable.

Le provisionnement des comptes membres s'appuie sur un module Terraform réutilisable, dont l'usage encapsule les paramètres récurrents (nom du compte, adresse électronique du propriétaire, OU parente) et impose systématiquement l'usage du rôle `OrganizationAccountAccessRole`, qui constitue le point d'entrée cross-compte réutilisé dans toutes les phases suivantes.

Politiques de contrôle de service

Huit SCPs sont déployées via un second module réutilisable. Les six premières, attachées à la racine de l'organisation, définissent des interdictions universelles : le déni de toute opération en dehors de la région Union européenne (`DenyRegionOutsideEU`), l'interdiction pour un compte membre de quitter l'organisation, l'interdiction d'usage de l'utilisateur *root*, l'interdiction de désactivation des services de sécurité, l'interdiction de modification des trails CloudTrail, et l'interdiction de création d'utilisateurs IAM classiques dans les comptes membres (la fédération via Identity Center devenant l'unique voie d'accès). Deux SCPs supplémentaires, plus restrictives, sont attachées respectivement à l'OU *Prod* et à l'OU *Sandbox*.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "DenyAllOutsideEU",
      "Effect": "Deny",
      "Action": "*",
      "Resource": "*",
      "Condition": {
        "StringNotEquals": {
          "aws:RequestedRegion": ["eu-west-1"]
        },
        "ArnNotLike": {
          "aws:PrincipalARN": [
            "arn:aws:iam::*:role/OrganizationAccountAccessRole"
          ]
        }
      }
    }
  ]
}
```

Listing 6.2 – Extrait de la SCP interdisant l'usage hors région Union européenne.

L'exclusion du rôle `OrganizationAccountAccessRole` est essentielle : sans elle, les opérations Terraform elles-mêmes seraient bloquées, puisqu'elles transitent nécessairement par ce rôle lors des déploiements cross-compte.

Point de vigilance : quota implicite sur le nombre de comptes

Le plan initial du projet prévoyait le provisionnement de huit comptes membres. La quatrième opération de création s'est cependant soldée par une erreur `ConcurrentModificationException` puis, sur nouvel essai, par un déni explicite invoquant un quota implicite de cinq comptes par organisation nouvelle. Ce quota, non documenté sur la page principale de tarification, s'applique aux organisations récentes en tant que garde-fou contre les abus. Une demande d'augmentation à quinze comptes a été soumise au support AWS. En attendant sa validation, le projet a pragmatiquement continué avec quatre comptes membres effectivement créés (`network`, `shared-services`, `morocco-dev`, `backup`), les autres comptes du plan initial demeurant à l'état de structure organisationnelle non peuplée. Cette contrainte n'a pas remis en cause la validité de la démarche, l'ensemble des mécanismes étant démontré sur les comptes déjà créés.

Correspondance avec le modèle à quatre comptes du chapitre 5

La réalisation partielle décrite ci-dessus appelle une clarification par rapport au modèle conceptuel en quatre comptes fonctionnels présenté en section 5.2 : compte de gestion, compte des services partagés, compte de la charge de travail et compte de journalisation. Le tableau 6.1 établit la correspondance entre ces rôles conceptuels et les comptes membres effectivement provisionnés.

Cet écart entre le modèle conceptuel à quatre comptes et les quatre comptes effectivement disponibles illustre une contrainte réelle du projet plutôt qu'un abandon du principe : la logique de séparation par fonction reste posée dans la conception (chapitre 5), mais sa réalisation complète — en particulier la séparation entre le compte réseau et le compte de charge de travail, ainsi que l'isolement d'un compte de journalisation dédié — est différée à une levée du quota, mentionnée comme perspective en conclusion générale.

TABLE 6.1 – Correspondance entre rôles conceptuels (chapitre 5) et comptes réellement déployés

Rôle conceptuel (Ch. 5)	**Compte réel**	**Remarque**
Compte de gestion	Compte de gestion AWS Organizations	Héberge Organizations, SCPs, IAM Identity Center ; non compté comme compte membre
Compte des services partagés	`shared-services`	Administrateur délégué GuardDuty et Security Hub (Phase 02)
Compte de la charge de travail	`network`	Héberge à la fois le VPC (Phase 03) et la couche de calcul ECS Fargate / ALB (Phases 04–07) : réseau et charge de travail ne sont pas isolés dans deux comptes distincts
Compte de journalisation	*non isolé*	Faute de quota disponible, aucun compte dédié n'a été provisionné ; les journaux CloudTrail restent centralisés dans le bucket S3 décrit en Phase 02

Fédération d'identité et journalisation d'audit

L'AWS IAM Identity Center est déployé dans le compte de gestion et configuré avec un magasin d'identités interne (l'intégration à un annuaire externe étant hors périmètre pour un projet académique). Un utilisateur `hiba` est provisionné, protégé par une authentification à facteur multiple, et associé à deux *permission sets* : un jeu `Admin` pour la gestion active, et un jeu `ReadOnly` pour les opérations de consultation. Des *account assignments* attachent le *permission set* `Admin` de l'utilisateur `hiba` à chacun des comptes déployés, y compris le compte de gestion.

Un trail CloudTrail organisationnel, nommé `atlas-fintech-org-trail`, est déployé depuis le compte de gestion avec les options `is_multi_region_trail`, `include_global_service_e` et `enable_log_file_validation` activées. Les journaux sont stockés dans un compartiment S3 dédié dont la politique adopte un motif d'accès double, autorisant à la fois le principal du compte de gestion et le principal d'organisation via la condition `aws:SourceArn`. Cette configuration est nécessaire au bon fonctionnement d'un trail organisationnel, dont le service émet des requêtes d'écriture depuis un contexte distinct de celui du compte de gestion propriétaire.

Services de détection

Amazon GuardDuty et AWS Security Hub sont activés à l'échelle organisationnelle depuis le compte de gestion, chacun étant configuré pour couvrir automatiquement tout nouveau compte rejoignant l'organisation. Security Hub est aligné sur deux référentiels de conformité : *AWS Foundational Security Best Practices* et *CIS AWS Foundations Benchmark v1.4.0*. Postérieurement au déploiement initial, la fonction d'administration déléguée a été mise en place, le compte `network` devenant l'administrateur délégué pour GuardDuty et Security Hub. Cette délégation dissocie la fonction de gouvernance (compte de gestion) de la fonction de sécurité opérationnelle (compte dédié), conformément aux recommandations AWS Prescriptive Guidance.

Vérification

Les figures ci-dessous documentent les principaux artefacts déployés lors de cette phase.

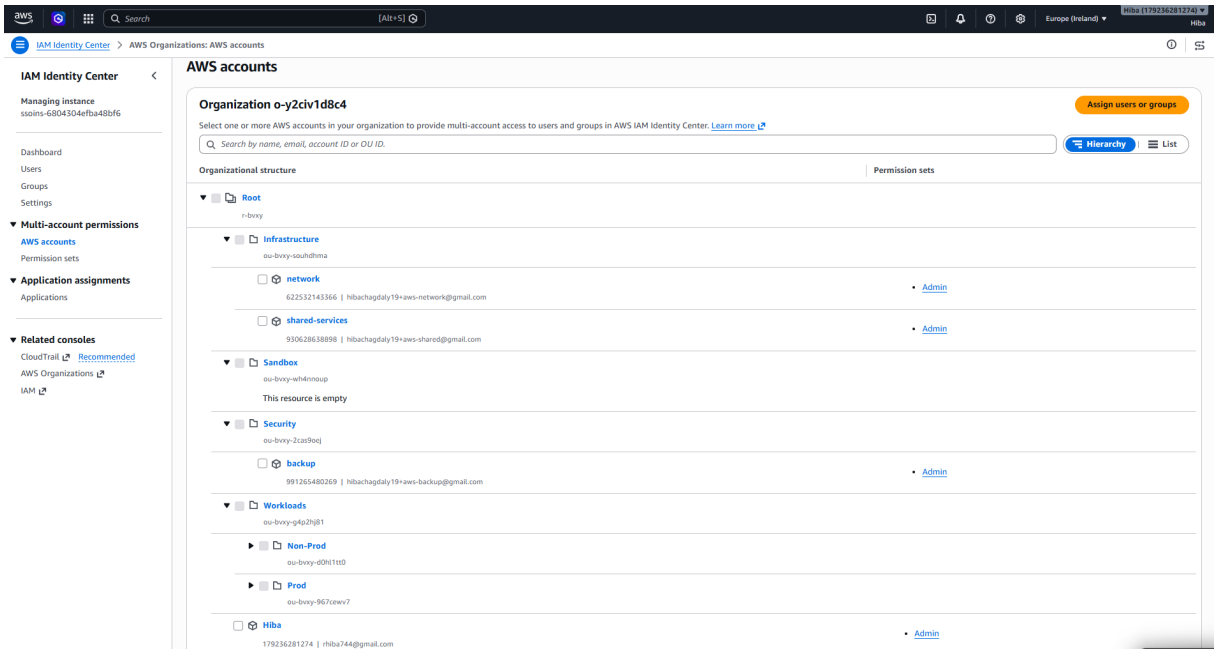


FIGURE 6.5 – Hiérarchie d'unités organisationnelles créée dans AWS Organizations.

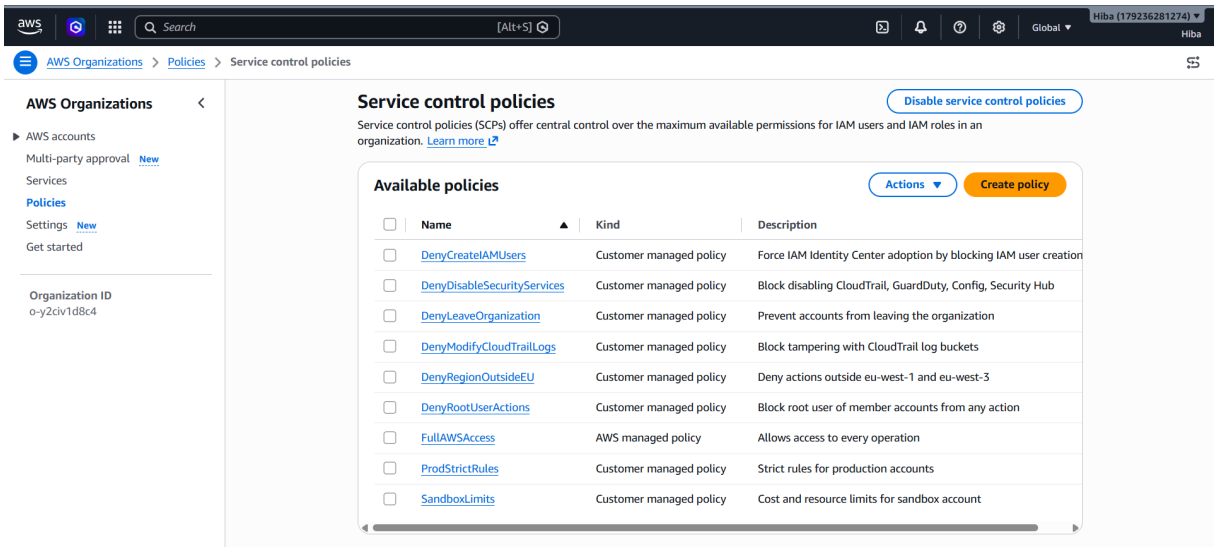


FIGURE 6.6 – Les huit SCPs déployées via le module réutilisable, dont six attachées à la racine de l'organisation.

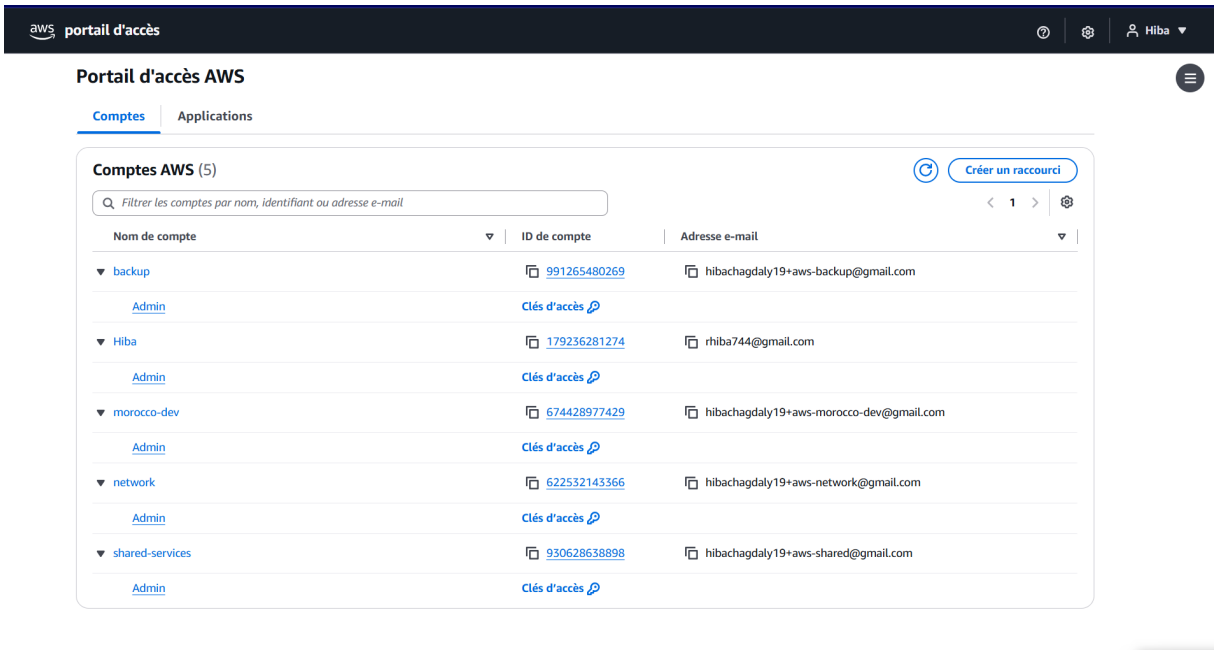


FIGURE 6.7 – Portail SSO donnant accès aux comptes de l'organisation après authentification MFA.

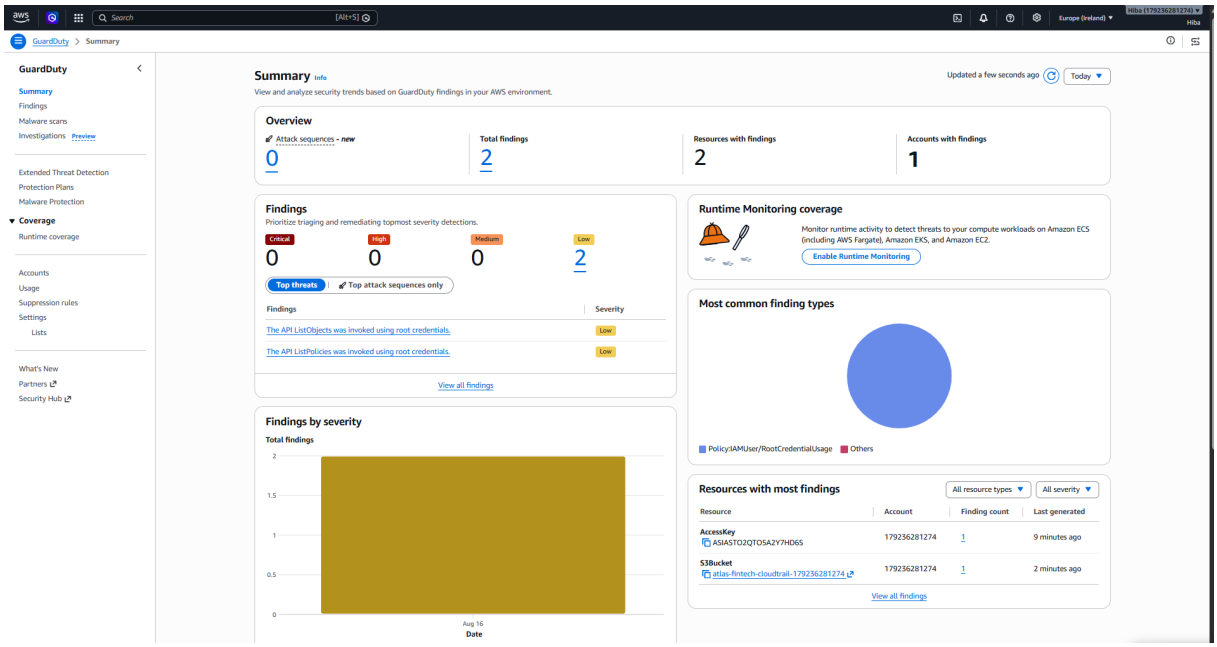


FIGURE 6.8 – Constatations GuardDuty réelles remontées sur le compte de gestion (usage de l'utilisateur *root*), matérialisant la détection effective en conditions de production.

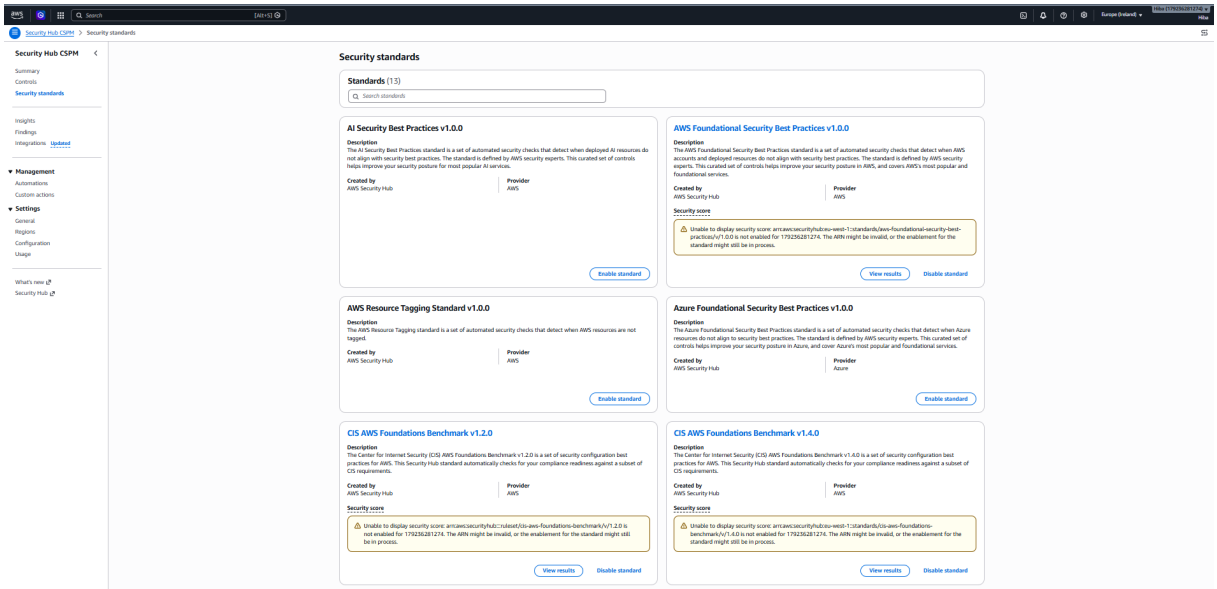


FIGURE 6.9 – Standards de conformité activés dans Security Hub à l'échelle organisationnelle.

6.2.2 Phase 03 – Réseau à trois couches

Objectif

Cette phase établit la topologie réseau qui accueillera l'ensemble des charges applicatives ultérieures. Elle constitue le premier déploiement effectué dans un compte membre plutôt que dans le compte de gestion : le fournisseur Terraform est configuré pour assumer le rôle `OrganizationAccountA` du compte `network` (identifiant `622532143366`) avant toute opération, ce qui matérialise le pattern d'authentification cross-compte introduit précédemment.

Chaîne de confiance entre couches

La configuration des groupes de sécurité constitue le point pédagogique le plus fort de cette phase. Trois groupes sont définis : `alb-sg` accepte le trafic Internet entrant sur les ports 80 et 443 ; `app-sg` accepte le trafic entrant uniquement en provenance du groupe `alb-sg` ; `db-sg` accepte le trafic entrant uniquement en provenance du groupe `app-sg`, sur le port PostgreSQL. Cette chaîne de confiance est rendue possible par la capacité des groupes de sécurité AWS à référencer un autre groupe comme source, plutôt qu'un bloc CIDR.

```
resource "aws_security_group" "app" {
  name   = "atlas-fintech-app-sg"
  vpc_id = aws_vpc.main.id

  ingress {
    from_port       = 80
    to_port         = 80
    protocol        = "tcp"
    security_groups = [aws_security_group.alb.id]  # source = groupe ALB
  }
}

resource "aws_security_group" "db" {
  name   = "atlas-fintech-db-sg"
  vpc_id = aws_vpc.main.id

  ingress {
```

```
    from_port        = 5432
    to_port          = 5432
    protocol         = "tcp"
    security_groups = [aws_security_group.app.id]  # source = groupe App
  }
  # Aucun egress declare : chaine stateful, reponses sortent
      automatiquement.
}
```

Listing 6.3 – Chaîne de confiance ALB → App → Base de données via référence de groupes de sécurité.

Cette approche est fondamentalement différente d'une configuration par CIDR, dans laquelle chaque nouvel équipement ajouté à une couche exigerait de modifier les règles des couches voisines. Ici, l'appartenance à un groupe suffit à établir la relation de confiance, et l'ajout de nouvelles instances hérite automatiquement des autorisations.

Un principe implicite mérite d'être explicité à ce stade : le fait que le routage rende une communication *possible* entre deux sous-réseaux ne signifie pas qu'elle soit *autorisée*. Les tables de routage définissent les chemins ; les groupes de sécurité définissent les permissions. La défense en profondeur repose précisément sur cette double couche.

Vérification

Les figures ci-dessous documentent la topologie déployée.

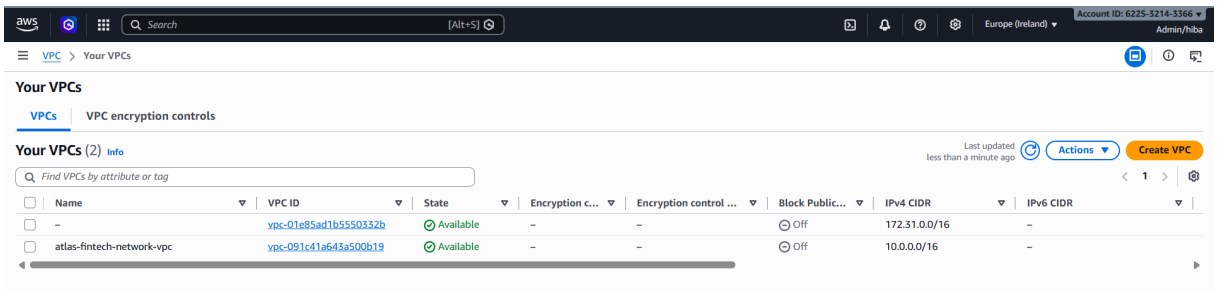


FIGURE 6.10 – VPC `atlas-fintech-vpc` déployé avec le bloc CIDR `10.0.0.0/16`.

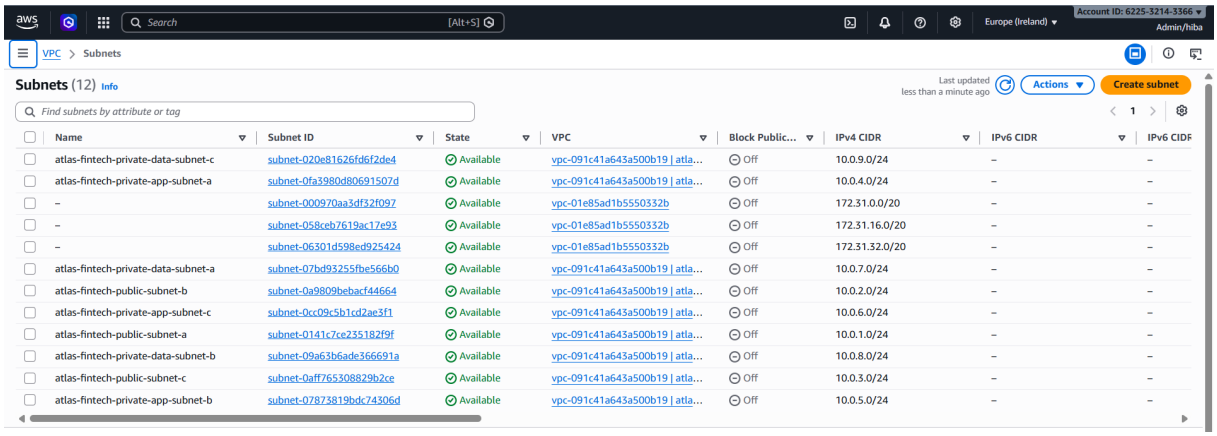


FIGURE 6.11 – Les neuf sous-réseaux répartis sur trois zones de disponibilité.

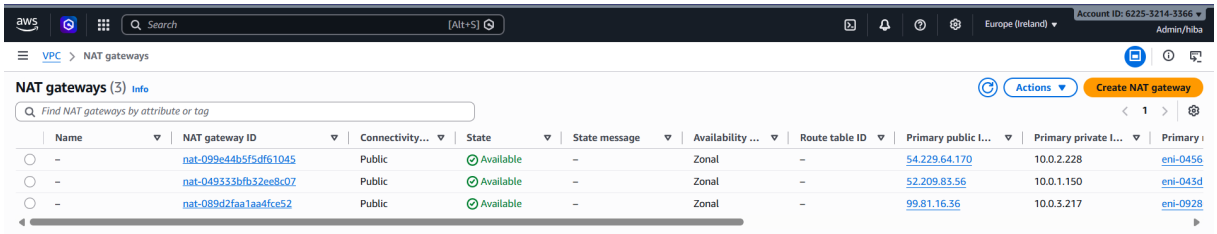


FIGURE 6.12 – Trois passerelles NAT déployées, une par zone de disponibilité, matérialisant le choix de haute disponibilité au niveau du trafic sortant.

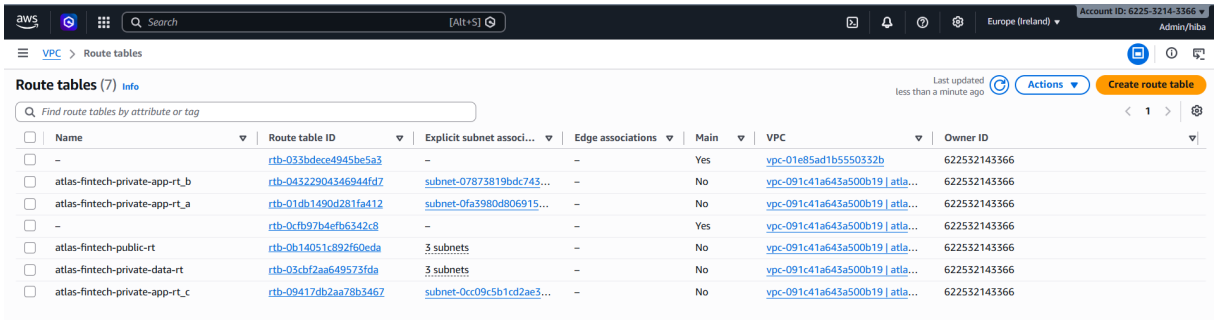


FIGURE 6.13 – Tables de routage déployées, illustrant l'asymétrie entre la table publique partagée et les tables privées applicatives dédiées par zone.

Bilan de la section

Ces deux phases posent l'architecture organisationnelle et réseau sur laquelle toutes les phases ultérieures viendront s'appuyer. Le pattern d'authentification cross-compte est désormais opérationnel, la topologie réseau est prête à accueillir des charges applicatives, et les mécanismes de gouvernance (SCPs, journalisation, détection de menaces) sont actifs à l'échelle de l'organisation. La section suivante peut alors aborder la construction de la plateforme applicative

proprement dite et sa maturation progressive au fil de quatre phases dédiées.

6.3 Plateforme applicative et sa maturation

Cette section couvre le cœur du cycle de vie d'une plateforme applicative : le déploiement initial des ressources de calcul, leur durcissement en matière de sécurité et de chiffrement, l'ajout d'une couche d'observabilité permettant de superviser leur comportement, et enfin l'automatisation sécurisée de leur mise à jour via un pipeline d'intégration et de déploiement continus. Chacune de ces quatre phases apporte une capacité qui vient enrichir la précédente, dans une progression qui reflète la maturation d'une plateforme depuis son état initial fonctionnel jusqu'à un état conforme aux exigences d'un environnement de production. Ces quatre phases traduisent en implémentation les décisions retenues pour BF3, BF4, BNF1 et BNF2 (tableau 5.3).

6.3.1 Phase 04 – Compute serverless (ECS Fargate et ALB)

Objectif

Cette phase déploie la couche de calcul de la plateforme sur Amazon ECS avec le mode de lancement Fargate, éliminant ainsi la gestion d'instances EC2 sous-jacentes. Elle introduit également l'équilibreur de charge applicatif (ALB) qui exposera la plateforme à Internet, ainsi que les deux rôles IAM associés aux tâches conteneurisées.

Éléments d'implémentation

Le service ECS agrège l'ensemble : il référence la définition de tâche, cible le nombre souhaité d'instances en cours d'exécution, place les tâches dans les sous-réseaux privés applicatifs, et les enregistre auprès du groupe cible.

```
resource "aws_ecs_service" "app" {
  name            = "atlas-fintech-app-svc"
  cluster         = aws_ecs_cluster.main.id
  task_definition = aws_ecs_task_definition.app.arn
```

```
  desired_count   = 2
  launch_type     = "FARGATE"

  network_configuration {
    subnets          = data.terraform_remote_state.network.outputs.
       private_app_subnets
    security_groups  = [data.terraform_remote_state.network.outputs.
       app_sg_id]
    assign_public_ip = false
  }

  load_balancer {
    target_group_arn = aws_lb_target_group.app.arn
    container_name   = "atlas-fintech-app"
    container_port   = 80
  }
}
```

Listing 6.4 – Déclaration du service ECS avec placement multi-AZ et enregistrement automatique auprès du groupe cible.

Le bloc `data "terraform_remote_state"` matérialise ici, pour la première fois dans le projet, la consommation par un dossier des sorties d'un autre : la définition du service ECS lit dynamiquement l'identifiant des sous-réseaux privés applicatifs et l'identifiant du groupe de sécurité produits par la phase précédente.

Points de vigilance rencontrés

Deux difficultés distinctes sont apparues durant cette phase.

Retour silencieux à un mode de lancement EC2. La première itération du service ECS omettait l'attribut `launch_type = "FARGATE"`, l'attribut étant optionnel dans le schéma Terraform. La commande `terraform apply` a rapporté un succès, mais aucune tâche n'a jamais démarré. L'investigation a révélé que le service défaultait alors sur le mode EC2, requérant un cluster peuplé d'instances EC2 enregistrées, absentes de la configuration. L'absence de message d'erreur explicite s'explique par le fait que le service considère l'attente d'instances EC2

comme un état d'attente légitime, non comme une erreur. L'ajout explicite du `launch_type` a résolu le problème.

Écoute HTTP bloquée par un groupe de sécurité HTTPS-only. La seconde difficulté a trait à un écho direct de la leçon posée en phase précédente. Le *listener* de l'ALB, configuré sur le port 80 pour cette étape (le certificat TLS n'ayant pas encore été mis en place), s'est trouvé bloqué par un groupe de sécurité `alb-sg` qui n'autorisait initialement que le port 443 en entrée. L'URL de l'ALB ne répondait pas, alors même que les tâches ECS s'exécutaient correctement en interne. L'ajout d'une règle d'entrée sur le port 80 a résolu le symptôme immédiat, mais surtout confirmé le principe posé en Phase 03 : le routage rendait le chemin possible, mais le groupe de sécurité en interdisait l'usage.

Vérification

Les figures ci-dessous documentent l'état opérationnel de la plateforme à l'issue de cette phase.

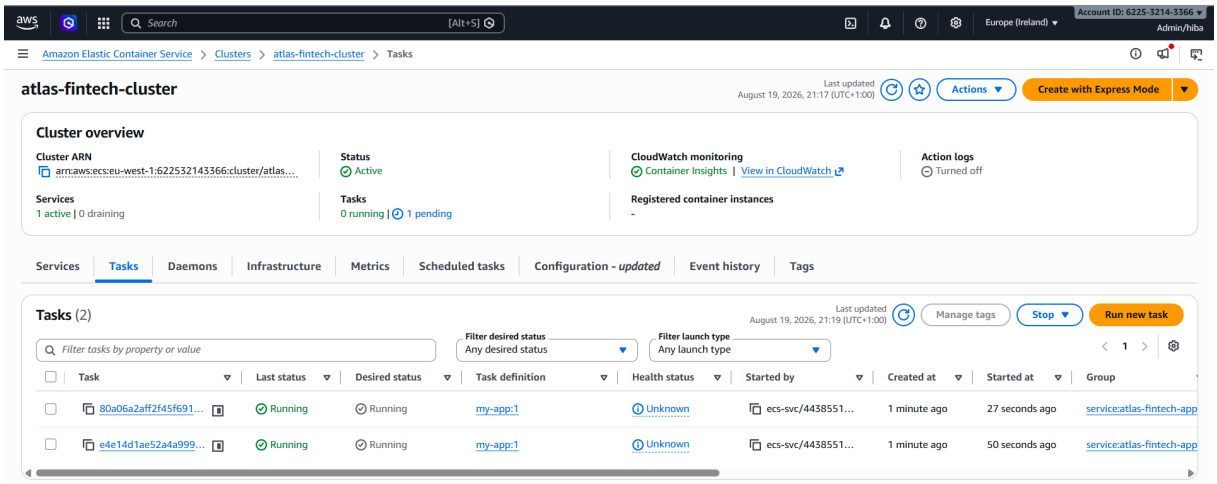


FIGURE 6.14 – Deux tâches ECS Fargate en cours d'exécution, placées dans des sous-réseaux privés distincts.

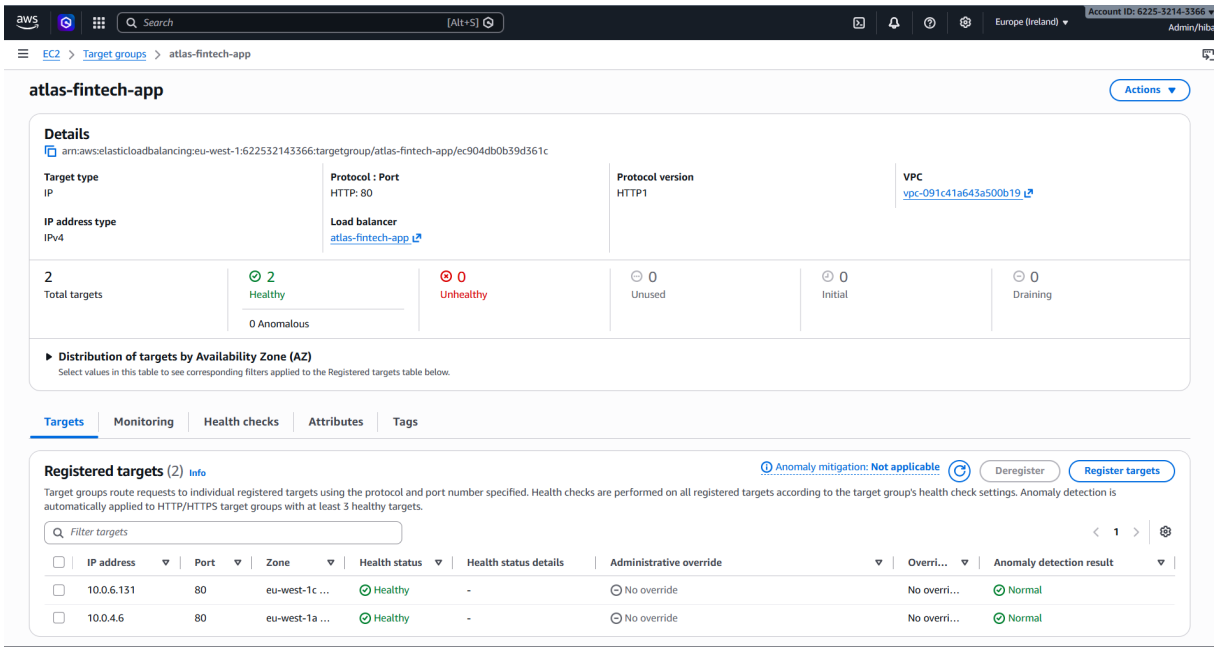


FIGURE 6.15 – Groupe cible affichant deux instances saines, réparties sur deux zones de disponibilité distinctes.

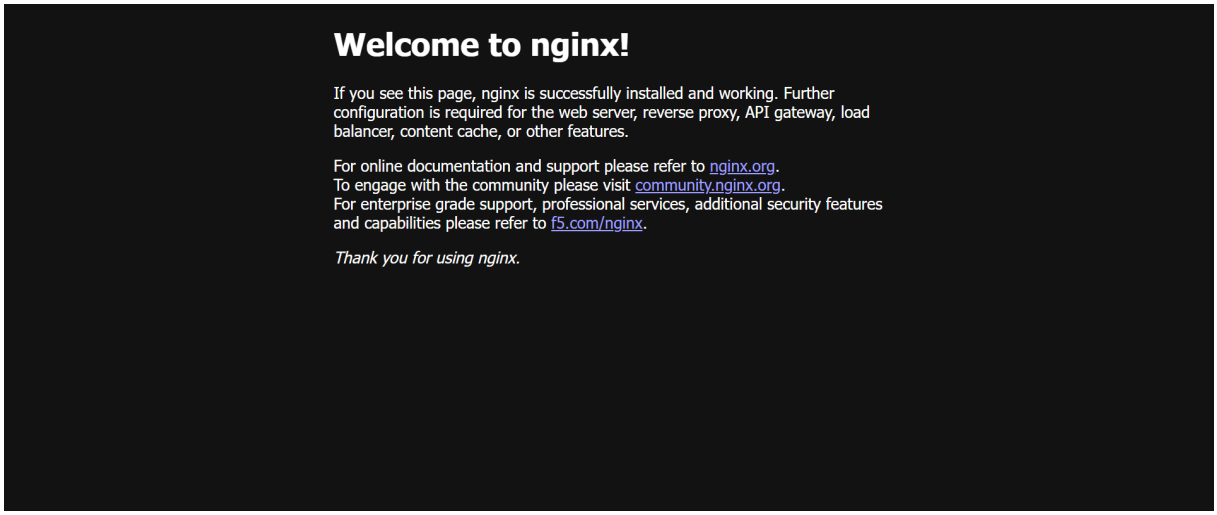


FIGURE 6.16 – Page nginx servie à travers l'URL publique de l'ALB, confirmant le fonctionnement de bout en bout de la chaîne Internet → ALB → ECS.

6.3.2 Phase 05 – Durcissement sécurité

Objectif

Cette phase transverse renforce la plateforme applicative sur trois axes : le chiffrement des données au repos avec une clé gérée par le client, la journalisation applicative dans CloudWatch

Logs, et l'élasticité automatique via l'ajustement dynamique du nombre de tâches ECS en fonction de la charge. Elle inaugure un nouveau dossier Terraform `infra/security/`, tout en modifiant les ressources déjà déployées dans `infra/compute/` pour les rendre consommatrices de ces nouveaux services.

Clé de chiffrement gérée par le client

Une clé KMS *customer-managed* (CMK) est provisionnée avec rotation annuelle automatique. Sa *key policy* adopte une approche de *scoping* explicite : elle autorise explicitement les services CloudWatch Logs et ECR à utiliser la clé, plutôt que d'accorder une autorisation générique. Cette approche relève d'un principe important : les *key policies* de KMS ne sont pas des IAM policies. Elles constituent le mécanisme d'autorisation racine de la clé ; sans l'énoncé initial autorisant le compte propriétaire à administrer la clé, celle-ci deviendrait irrécupérable après création.

```
{
  "Sid": "AllowCloudWatchLogsForAccount",
  "Effect": "Allow",
  "Principal": {
    "Service": "logs.eu-west-1.amazonaws.com"
  },
  "Action": [
    "kms:Encrypt",
    "kms:Decrypt",
    "kms:GenerateDataKey*",
    "kms:DescribeKey"
  ],
  "Resource": "*",
  "Condition": {
    "ArnLike": {
      "kms:EncryptionContext:aws:logs:arn": "arn:aws:logs:eu-west-1:${
          account_id}:*"
    }
  }
}
```

Listing 6.5 – Extrait de la *key policy* scopée autorisant CloudWatch Logs à chiffrer les journaux uniquement dans le compte propriétaire.

La condition `kms:EncryptionContext` constitue le mécanisme de *scoping* fin : elle n'autorise pas simplement le service CloudWatch Logs à utiliser la clé, mais uniquement pour chiffrer des groupes de journaux appartenant au compte propriétaire. Une future migration vers un compte différent ne pourrait pas exploiter cette clé.

Journalisation applicative

Un groupe de journaux CloudWatch chiffré avec la CMK est créé, et la définition de tâche ECS est modifiée pour envoyer ses sorties standard vers ce groupe via le pilote `awslogs`. Cette évolution nécessite un ajustement du rôle d'exécution ECS, qui doit obtenir les permissions `kms:GenerateDataKey` et `kms:Decrypt` pour manipuler les clés de données produites par la CMK.

Une action supplémentaire, `kms:CreateGrant`, s'est révélée nécessaire lors de la migration du dépôt ECR vers le chiffrement KMS : ECR utilise en effet des *grants* pour déléguer temporairement l'accès à la clé lors des opérations de récupération d'image. Cette dépendance n'est pas immédiatement documentée, mais son omission provoque un échec de démarrage de la tâche au moment où celle-ci tente de tirer son image du dépôt.

Mise à niveau du dépôt ECR

Le dépôt ECR est reconfiguré avec l'attribut `image_tag_mutability = "IMMUTABLE"` (empêchant la réutilisation d'un tag existant, ce qui protège contre les substitutions silencieuses d'images en production) et avec le chiffrement au repos assuré par la CMK. Cette modification force le remplacement de la ressource : le bloc `encryption_configuration` d'ECR ne peut pas être modifié après création. Le remplacement a été possible sans coût opérationnel puisque le dépôt ne contenait alors que l'image placeholder, mais le principe reste applicable à des migrations futures.

Élasticité automatique

L'*Application Auto Scaling* est configuré sur le service ECS selon une stratégie *Target Tracking* paramétrée sur l'utilisation CPU moyenne des tâches, avec une cible de 50 %. Le nombre de tâches peut varier entre deux et quatre, avec des périodes de refroidissement asymétriques : soixante secondes pour la mise à l'échelle vers le haut (réaction rapide à une montée en charge) et trois cents secondes pour la mise à l'échelle vers le bas (préservation contre une oscillation entre deux paliers).

Vérification

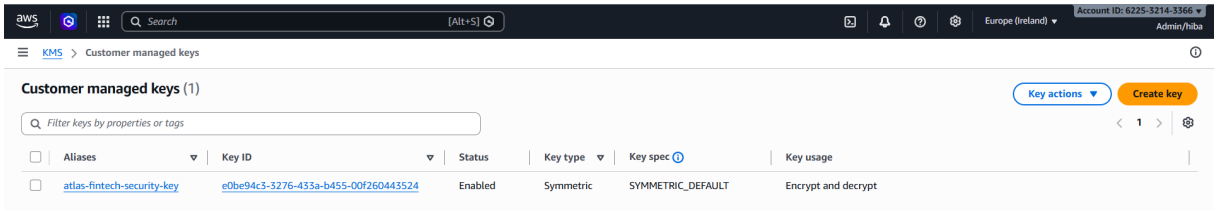


FIGURE 6.17 – Clé KMS gérée par le client avec rotation annuelle activée.

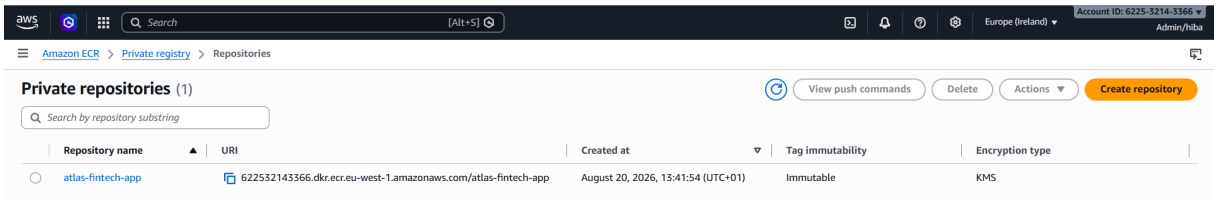


FIGURE 6.18 – Dépôt ECR reconfiguré avec chiffrement KMS et immutabilité des tags.

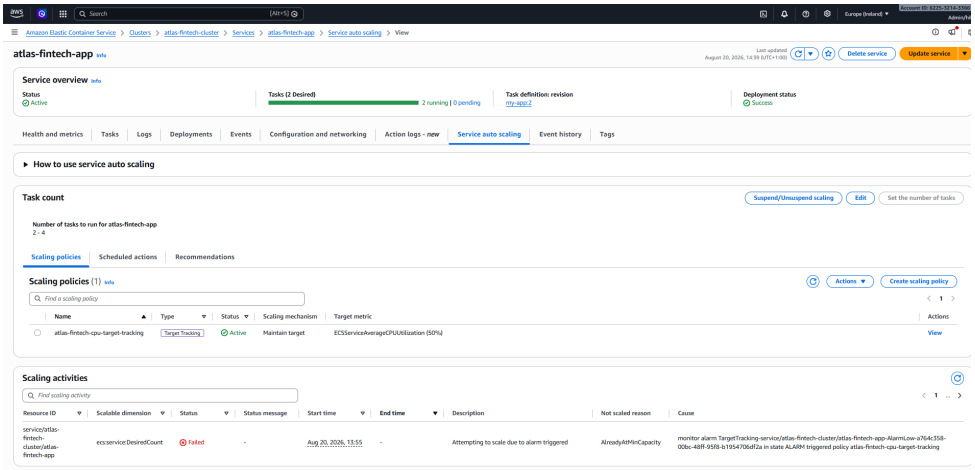


FIGURE 6.19 – Stratégie Auto Scaling Target Tracking associée au service ECS.

6.3.3 Phase 06 – Observabilité

Objectif

Cette phase introduit une couche d'observabilité opérationnelle permettant de superviser l'état de la plateforme applicative et de déclencher des alertes lorsque son comportement dévie d'un état de référence. Elle repose sur trois composants complémentaires : la publication de notifications via Amazon SNS, la définition d'alarmes CloudWatch surveillant des métriques précises, et l'agrégation de ces informations dans un tableau de bord visuel.

Éléments d'implémentation

Le sujet SNS et son abonnement par courriel constituent le canal de notification. L'exemple ci-dessous illustre la définition d'une alarme avec les nuances évoquées.

```
resource "aws_cloudwatch_metric_alarm" "alb_5xx" {
  alarm_name          = "atlas-fintech-alb-5xx"
  metric_name         = "HTTPCode_ELB_5XX_Count"
  namespace           = "AWS/ApplicationELB"
  statistic           = "Sum"
  period              = 300
  evaluation_periods  = 2
  threshold           = 5
  comparison_operator = "GreaterThanThreshold"
  treat_missing_data  = "notBreaching"

  dimensions = {
    LoadBalancer = data.terraform_remote_state.compute.outputs.
        alb_arn_suffix
  }

  alarm_actions = [aws_sns_topic.alerts.arn]
}
```

Listing 6.6 – Alarme CloudWatch sur le taux d'erreurs 5xx de l'ALB, avec traitement explicite de l'absence de trafic.

Vérification de bout en bout

Une caractéristique importante de cette phase est l'application concrète du principe posé en introduction du chapitre : la simple présence des alarmes ne suffit pas à garantir que la chaîne de notification est fonctionnelle. Un test manuel est donc effectué en forçant l'état d'une alarme via la commande `aws cloudwatch set-alarm-state`, ce qui déclenche la publication SNS et provoque effectivement la réception d'un courriel de notification. Après vérification, un second appel remet l'alarme dans l'état `OK`, confirmant que la sortie d'alarme génère également une notification.

Cette validation a nécessité d'obtenir des identifiants temporaires par assomption manuelle du rôle cross-compte (`aws sts assume-role`), le profil CLI `terraform-deployer` ne disposant pas nativement d'une configuration d'assomption de rôle. Cet aspect révèle une distinction subtile entre l'authentification utilisée par Terraform (qui gère elle-même la chaîne d'assomption via son fournisseur) et celle utilisée par l'AWS CLI pour des opérations ponctuelles, distinction qui deviendra source d'un incident secondaire lors de commandes Terraform ultérieures si les variables d'environnement issues de l'assomption manuelle ne sont pas correctement nettoyées.

Vérification

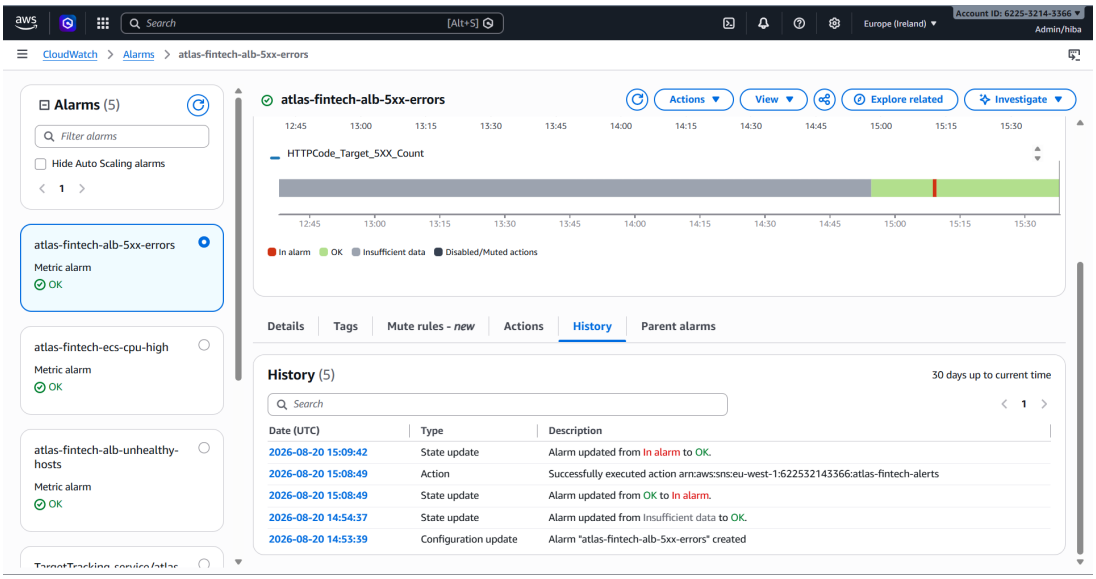
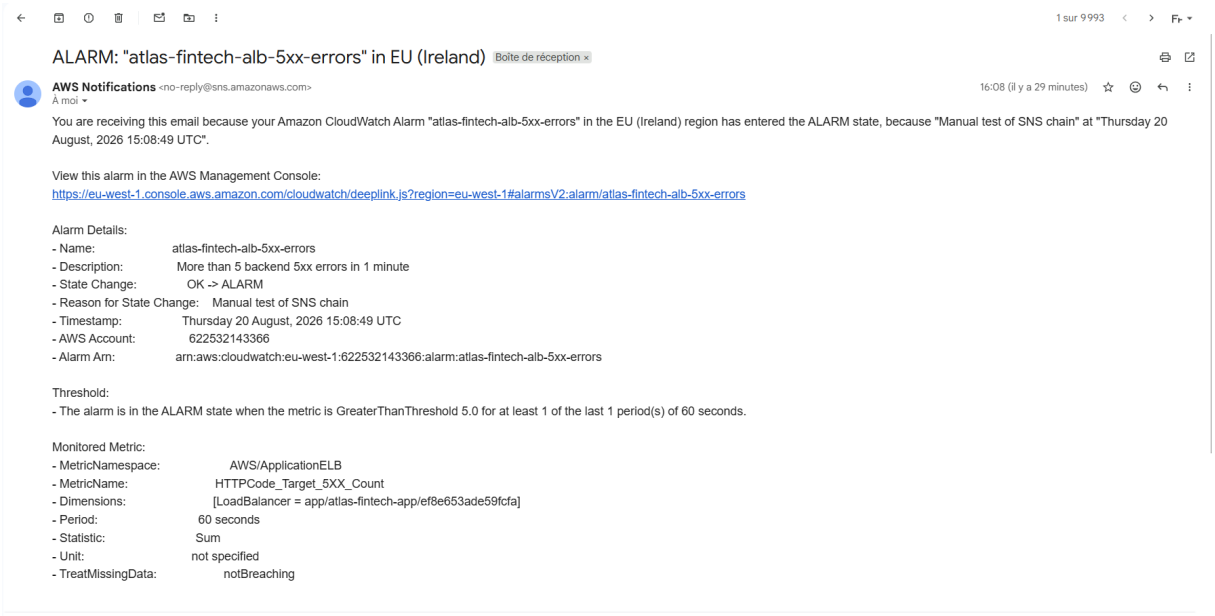


FIGURE 6.20 – Historique d'une alarme CloudWatch matérialisant le cycle complet `OK` → `ALARM` → `OK` lors du test manuel.



ALARM: "atlas-fintech-alb-5xx-errors" in EU (Ireland) Boîte de réception ×

AWS Notifications <no-reply@sns.amazonaws.com> 16:08 (il y a 29 minutes)
À moi

You are receiving this email because your Amazon CloudWatch Alarm "atlas-fintech-alb-5xx-errors" in the EU (Ireland) region has entered the ALARM state, because "Manual test of SNS chain" at "Thursday 20 August, 2026 15:08:49 UTC".

View this alarm in the AWS Management Console:
https://eu-west-1.console.aws.amazon.com/cloudwatch/deeplink.js?region=eu-west-1#alarmsV2:alarm/atlas-fintech-alb-5xx-errors

Alarm Details:
- Name: atlas-fintech-alb-5xx-errors
- Description: More than 5 backend 5xx errors in 1 minute
- State Change: OK -> ALARM
- Reason for State Change: Manual test of SNS chain
- Timestamp: Thursday 20 August, 2026 15:08:49 UTC
- AWS Account: 622532143366
- Alarm Arn: arn:aws:cloudwatch:eu-west-1:622532143366:alarm:atlas-fintech-alb-5xx-errors

Threshold:
- The alarm is in the ALARM state when the metric is GreaterThanThreshold 5.0 for at least 1 of the last 1 period(s) of 60 seconds.

Monitored Metric:
- MetricNamespace: AWS/ApplicationELB
- MetricName: HTTPCode_Target_5XX_Count
- Dimensions: [LoadBalancer = app/atlas-fintech-app/ef8e653ade59fcfa]
- Period: 60 seconds
- Statistic: Sum
- Unit: not specified
- TreatMissingData: notBreaching

FIGURE 6.21 – Courrier électronique de notification effectivement reçu à la suite du déclenchement forcé de l'alarme.

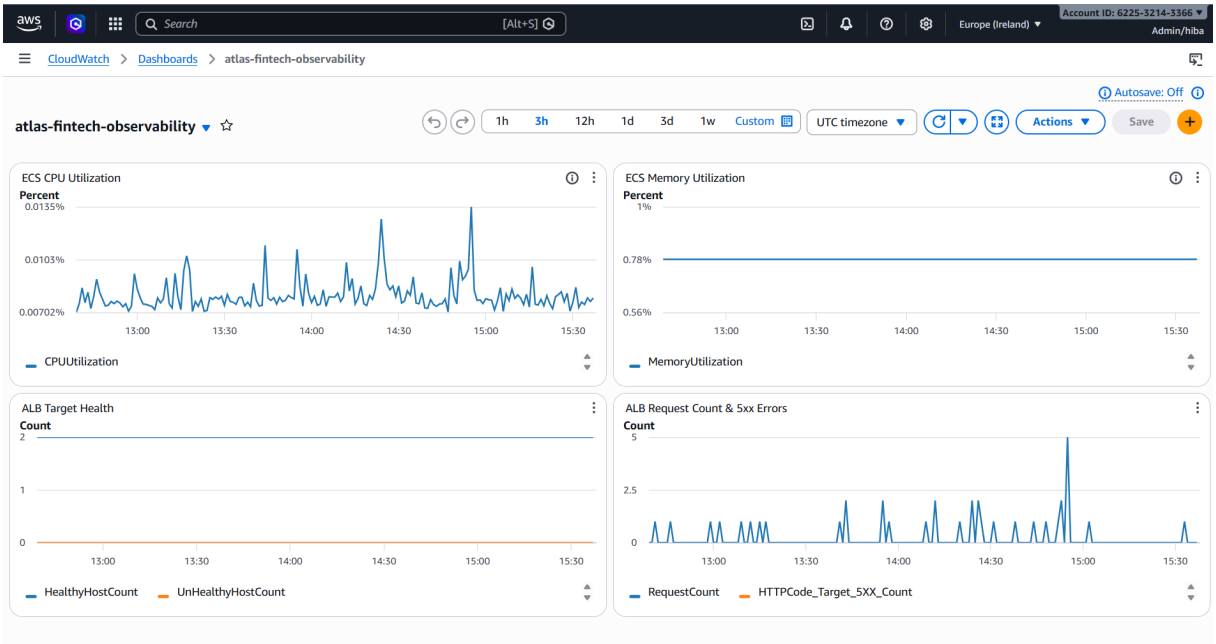


FIGURE 6.22 – Tableau de bord CloudWatch agrégeant les quatre indicateurs opérationnels de la plateforme applicative.

6.3.4 Phase 07 – Pipeline d'intégration et de déploiement continus

Objectif

Cette phase automatise le cycle de vie de l'application conteneurisée, depuis un *commit* sur le dépôt de code jusqu'à un déploiement effectif dans le cluster ECS, sans manipulation manuelle d'identifiants AWS de longue durée. Elle introduit également un point de contrôle de sécurité bloquant automatiquement tout déploiement dont l'image contiendrait des vulnérabilités critiques.

Point de contrôle bloquant sur les vulnérabilités critiques (*security gate*)

Le pipeline ne se limite pas à publier une nouvelle image et à redéployer le service. Une étape supplémentaire, intercalée entre la publication ECR et le déploiement ECS, interroge le service ECR pour connaître le résultat de l'analyse de vulnérabilités de l'image, et fait échouer le workflow si le nombre de vulnérabilités classées CRITICAL est strictement positif. Cette étape a été validée sur un cas concret : l'image nginx utilisée comme placeholder contient quatre vulnérabilités CRITICAL réelles issues de sa base Debian, et le pipeline bloque effectivement le déploiement lorsque cette image est publiée, matérialisant l'efficacité du contrôle.

Point de vigilance : évolution du format de la revendication `sub` de GitHub

Le déploiement initial de la fédération OIDC s'est traduit par un échec systématique de toutes les exécutions du workflow, avec le message d'erreur `Not authorized to perform sts:AssumeRoleWithWebIdentity`. Une inspection méthodique des six éléments impliqués (fournisseur OIDC dans IAM, empreinte du certificat, ARN référencé dans la *trust policy*, valeur de l'audience, motif de restriction sur la revendication `sub`, existence effective de SCPs susceptibles de bloquer l'opération) a conclu que chacun était individuellement correct.

La cause s'est révélée située en dehors du périmètre observable : GitHub a introduit au cours de l'année 2026 un nouveau format pour la revendication `sub` des jetons OIDC émis pour les dépôts créés après cette date. Le format ancien, de la forme `repo:owner/repo:ref:...`,

est désormais complété par des identifiants immutables intégrés dans la revendication, de la forme `repo:owner@id/repo@id:...`. Le motif `StringLike` de la *trust policy*, écrit pour l'ancien format (`repo:hiba05555/aws-secure-landing-zone:*`), ne correspondait plus. La correction a consisté à élargir le motif en insérant des caractères génériques autour du nom d'utilisateur et du nom du dépôt (`repo:hiba05555*aws-secure-landing-zone*`), rétablissant la correspondance.

Cette difficulté illustre un enseignement récurrent : une configuration IAM peut être individuellement correcte et pourtant devenir inopérante en raison d'un changement chez un fournisseur d'identité externe, changement qui n'est visible ni dans le code Terraform ni dans les journaux d'échec directement accessibles.

Vérification

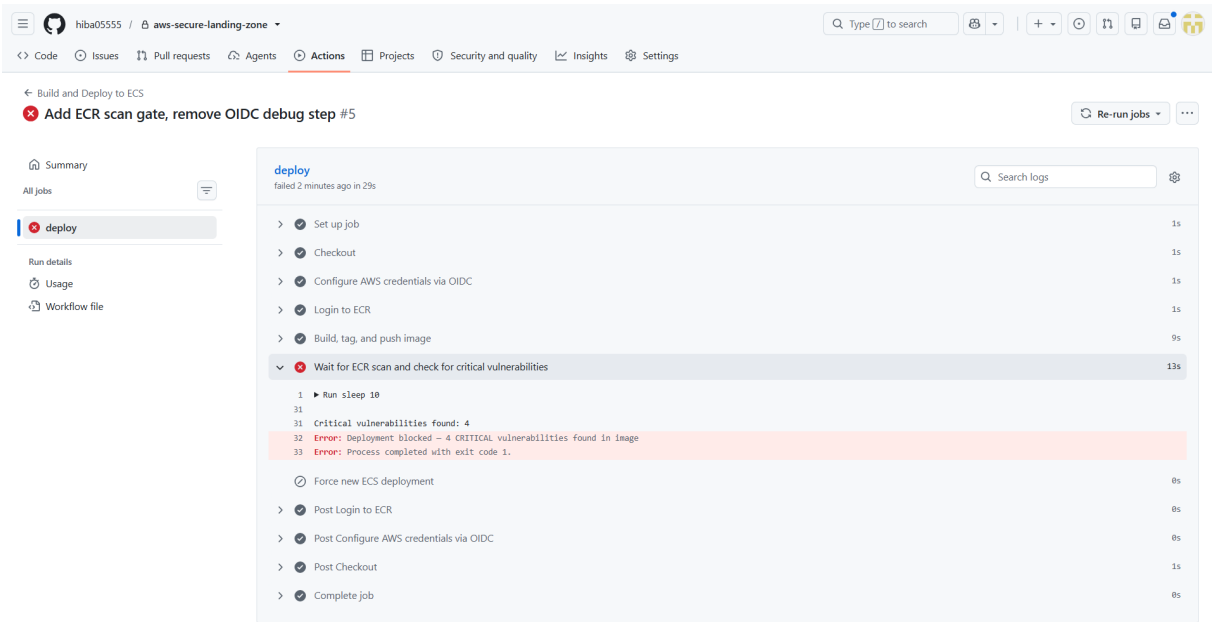


FIGURE 6.23 – Exécution du workflow GitHub Actions bloquée par le point de contrôle des vulnérabilités : quatre CVEs critiques détectées entraînent un code de sortie non nul et l'annulation du déploiement.

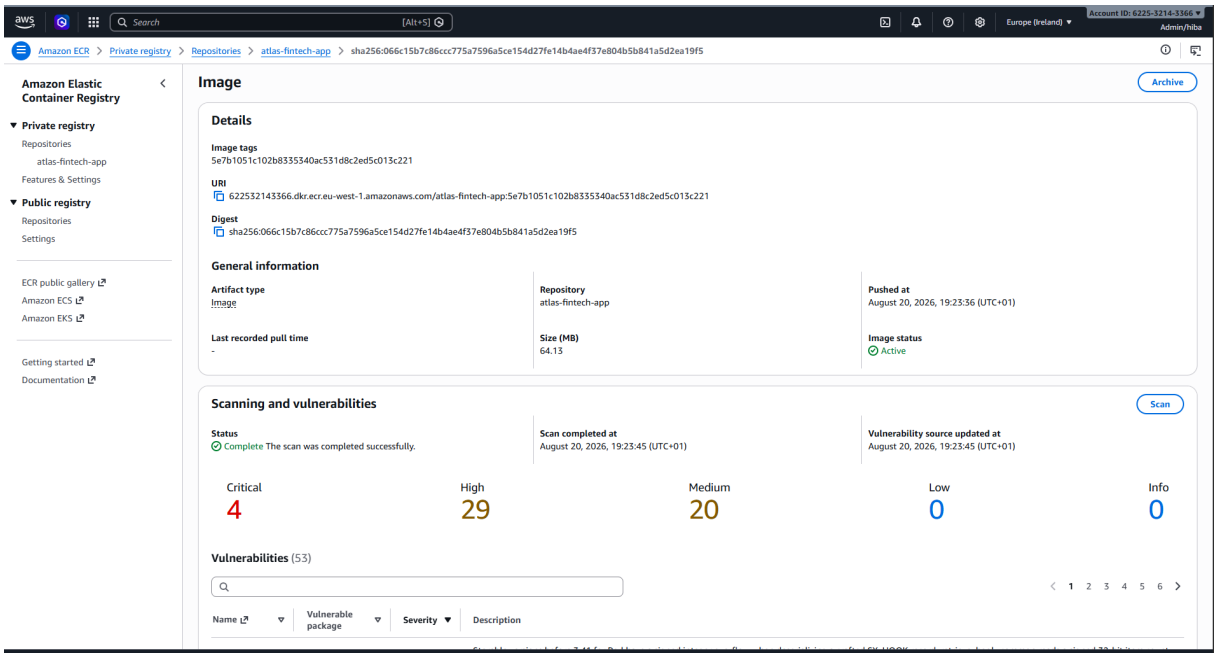


FIGURE 6.24 – Résultat de l'analyse de vulnérabilités du dépôt ECR détaillant les quatre vulnérabilités CRITICAL identifiées sur l'image nginx de base.

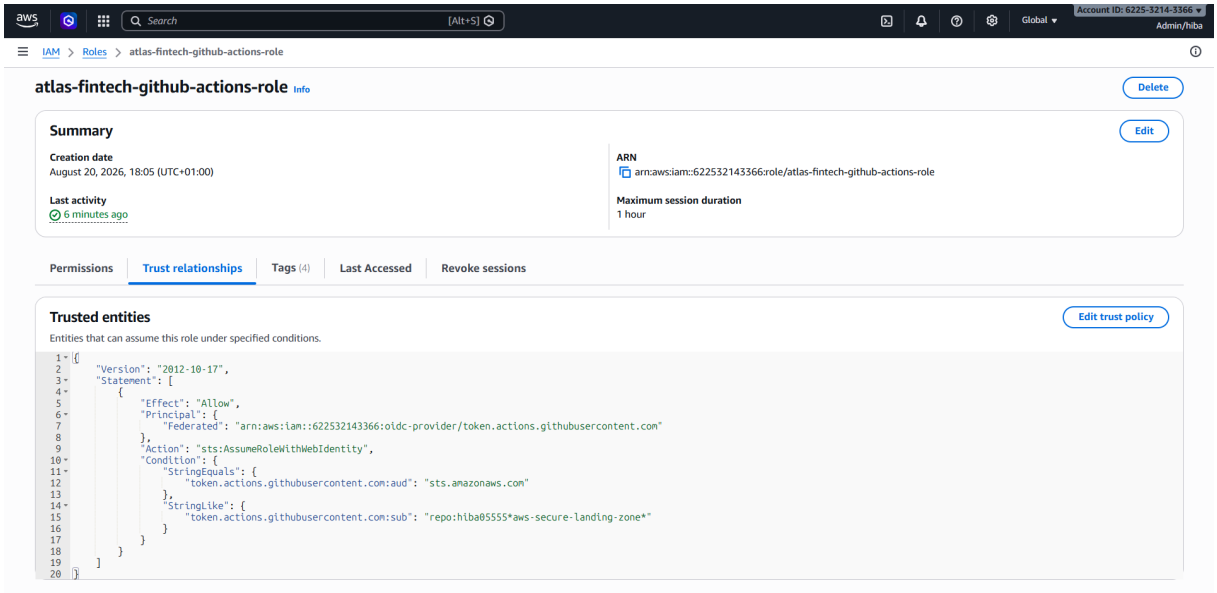


FIGURE 6.25 – Relations de confiance du rôle IAM après application du correctif sur le motif `StringLike` de la revendication `sub`.

Bilan de la section

Ces quatre phases produisent une plateforme applicative en pleine maturité : déployée dans une topologie multi-AZ, chiffrée de bout en bout au repos, dotée d'une capacité d'élasticité automatique, supervisée et alertée en temps réel, et mise à jour sans intervention manuelle grâce

à un pipeline lui-même contrôlé par un mécanisme de blocage sur vulnérabilités. La section suivante aborde les capacités qui viennent enrichir cette base au-delà de son cycle de vie strict : maîtrise financière, stockage objet, distribution de contenu, et découplage asynchrone.

6.4 Extensions et gouvernance des coûts

Cette dernière section regroupe des capacités hétérogènes par leur nature technique, mais qui partagent une même caractéristique : elles n'appartiennent pas au cycle de vie strict de la plateforme applicative (déploiement, sécurité, observabilité, automatisation), mais viennent en enrichir le contexte opérationnel. La maîtrise des coûts est traitée en premier, car elle constitue la contrepartie financière de l'ensemble des décisions techniques prises jusqu'ici. Trois capacités additionnelles suivent : le stockage objet, la distribution de contenu à échelle mondiale, et le découplage asynchrone des événements de sécurité. Ces quatre phases achèvent la couverture des besoins BF5, BNF3 et BNF5, dernières lignes non encore illustrées de la matrice de traçabilité (tableau 5.3).

6.4.1 Phase 08 – Gouvernance des coûts

Objectif

Cette phase provisionne les mécanismes de suivi et d'alerte budgétaire à l'échelle organisationnelle. Elle rompt avec le pattern établi dans les phases précédentes : elle est déployée directement dans le compte de gestion, sans assomption de rôle cross-compte.

Activation manuelle des étiquettes d'allocation de coûts

Les étiquettes d'allocation de coûts (*Cost Allocation Tags*) constituent le mécanisme permettant à AWS Cost Explorer de ventiler les dépenses selon des dimensions métier (projet, environnement, composant). Leur activation ne peut cependant pas être automatisée : l'API sous-jacente n'est pas exposée à Terraform, et l'opération doit être effectuée via la console. Cette limitation reproduit une caractéristique déjà rencontrée dans les phases précédentes (bootstrap manuel en

Phase 00, configuration IAM Identity Center en Phase 02), qui illustre les frontières concrètes du modèle infrastructure-as-code.

Point de vigilance : réconciliation d'une ressource préexistante

La commande `terraform apply` initiale de cette phase a produit une erreur `DuplicateRecordExcept` révélant qu'un budget portant le nom cible existait déjà dans le compte de gestion, résiduel d'expérimentations antérieures. La réponse adoptée a consisté à importer cette ressource dans l'état Terraform via la commande `terraform import`, puis à réappliquer la configuration : Terraform a alors constaté qu'il manquait la notification FORECASTED sur la ressource existante, et l'a ajoutée. Cette approche présente un enseignement méthodologique : la présence d'une ressource dans l'état AWS n'est pas systématiquement révélatrice d'une gestion Terraform, et le mécanisme d'importation constitue un outil de réconciliation à connaître.

Vérification

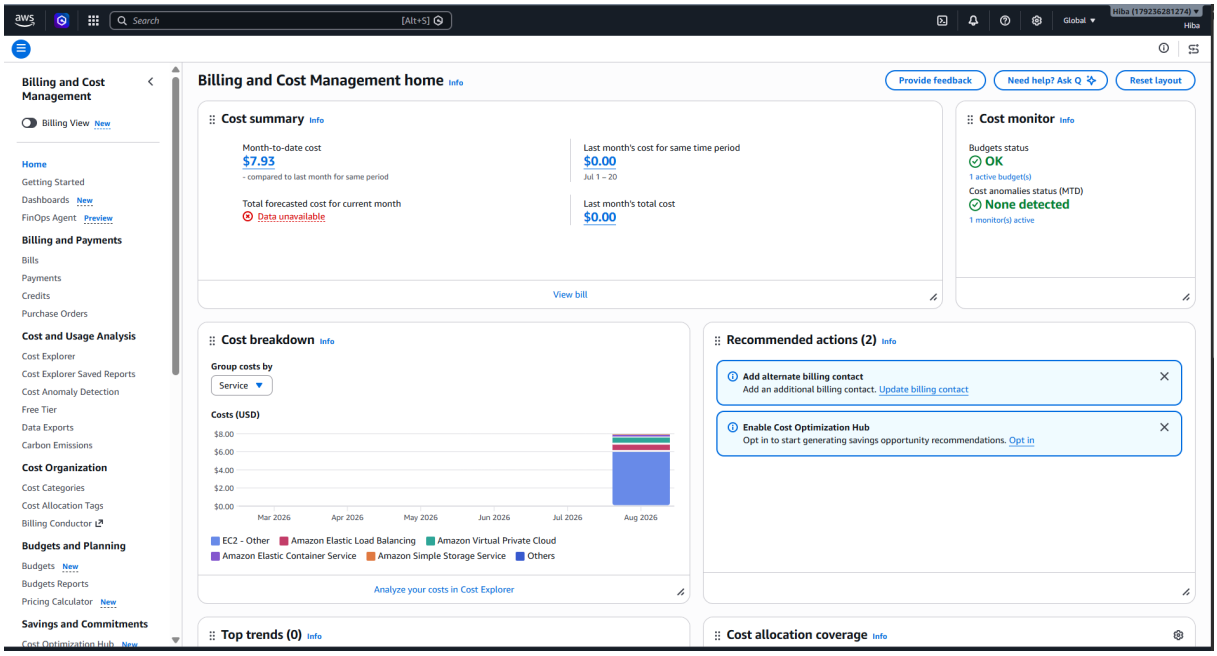


FIGURE 6.26 – Console AWS Budgets affichant le budget mensuel et ses deux seuils de notification.

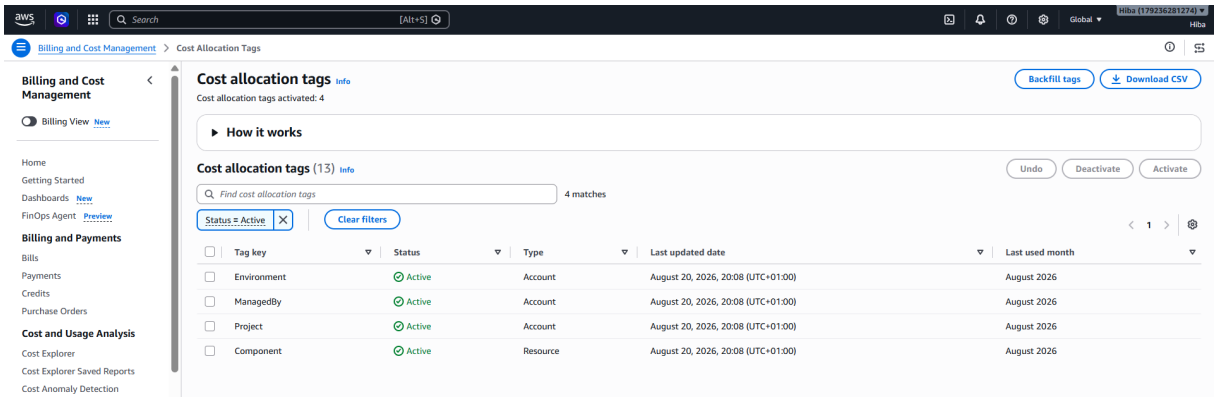


FIGURE 6.27 – Étiquettes d'allocation de coûts activées via la console, matérialisant l'action manuelle complémentaire au déploiement Terraform.

6.4.2 Phase 09 – Stockage objet

Objectif

Cette phase introduit un compartiment S3 destiné à héberger deux catégories de contenu aux caractéristiques d'accès radicalement différentes : des ressources statiques fréquemment sollicitées, et des archives à conservation longue mais très rarement lues.

Point de vigilance : durées minimales des classes Glacier

Le premier déploiement de la règle `ColdContentArchive` a échoué avec une erreur explicite énonçant que la durée entre la transition vers Glacier et celle vers Deep Archive était inférieure à la durée minimale de résidence dans Glacier Flexible Retrieval (90 jours). Les classes de stockage AWS imposent en effet des durées minimales de facturation, indépendamment de la présence effective de l'objet : un objet supprimé ou transitionné avant l'échéance est néanmoins facturé pour la durée complète. La correction a consisté à revoir l'ensemble de la chaîne de transitions pour vérifier que chaque intervalle respecte le minimum applicable de la classe source (30 jours pour Standard, 30 jours pour Standard-IA, 90 jours pour Glacier).

Vérification

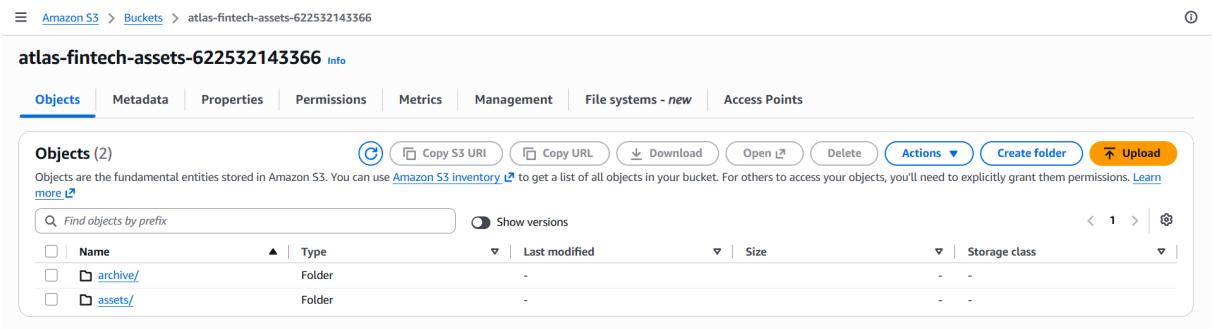


FIGURE 6.28 – Contenu du compartiment matérialisant la coexistence des deux préfixes conventionnels.

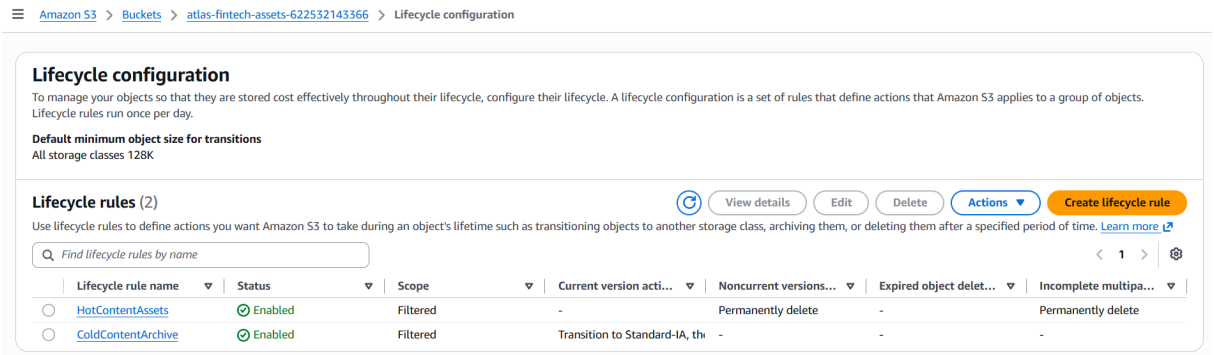


FIGURE 6.29 – Règles de cycle de vie configurées côte à côte, avec la chaîne complète de transitions pour le préfixe froid.

6.4.3 Phase 10 – Distribution de contenu à échelle mondiale

Objectif

Cette phase déploie une distribution CloudFront qui place l'ensemble de la plateforme derrière un point d'entrée unique répliqué sur les points de présence AWS. Elle modifie également les politiques d'accès de deux ressources antérieures (le compartiment S3 de la Phase 09 et la clé KMS de la Phase 05), matérialisant pour la première fois une phase transverse touchant simultanément à plusieurs dossiers Terraform déjà déployés.

Chaîne d'autorisations cross-services

L'introduction de CloudFront exige la modification coordonnée de deux politiques déjà déployées dans d'autres dossiers Terraform. La politique du compartiment S3 doit autoriser le principal service CloudFront à lire les objets, sous condition que la requête provienne de la distribution effectivement déployée. La *key policy* de la CMK doit également autoriser CloudFront à déchiffrer les données à la volée lors de la restitution des objets, sous la même condition de provenance.

```
{
  "Sid": "AllowCloudFrontServicePrincipalReadOnly",
  "Effect": "Allow",
  "Principal": {
    "Service": "cloudfront.amazonaws.com"
  },
  "Action": "kms:Decrypt",
  "Resource": "*",
  "Condition": {
    "StringEquals": {
      "AWS:SourceArn": "arn:aws:cloudfront::622532143366:distribution/
          ECQFVYBEI6XFU"
    }
  }
}
```

Listing 6.7 – Cinquième énoncé de la *key policy* KMS, autorisant CloudFront à déchiffrer les objets S3 servis via la distribution.

Ce cinquième énoncé constitue une propriété émergente de la combinaison OAC et SSE-KMS. L'OAC seul suffirait si le compartiment était chiffré en SSE-S3 ; le SSE-KMS seul serait suffisant si l'accès s'effectuait par IAM identité au sens classique. Leur combinaison exige que deux principals distincts (l'IAM Principal de CloudFront pour l'accès S3, et le Service Principal de CloudFront pour l'usage KMS) obtiennent chacun leur autorisation propre. Cette exigence n'est pas explicitement documentée par AWS comme un pattern unifié, et sa découverte au moment du déploiement a fait l'objet d'un diagnostic itératif.

Points de vigilance

Deux autres difficultés ont marqué cette phase. La première a trait à une régression involontaire : lors d'une manipulation intermédiaire, la règle d'entrée du groupe de sécurité de l'ALB sur le port 80 a été retirée, provoquant une erreur `504 Gateway Timeout` sur toute requête servie via CloudFront. Le diagnostic a été facilité par la possibilité de tester l'origine ALB directement, sans passer par CloudFront, ce qui a permis d'isoler la couche fautive. La leçon opérationnelle est explicite : lorsqu'un CDN se trouve en amont d'une origine, disposer d'un moyen de contourner le CDN pour tester l'origine directement est un instrument de diagnostic indispensable.

La seconde difficulté est plus anecdotique mais illustre un point structurant : une faute de frappe dans le nom d'une action KMS (`kms:DesscribeKey`, avec deux `s`, au lieu de `kms:DescribeKey`) a été acceptée sans erreur par l'API IAM et Terraform, l'ensemble étant considéré comme un déploiement réussi. Les politiques IAM et KMS ne font pas l'objet d'une validation syntaxique de leurs actions au moment du déploiement, et une action inexistante est simplement ignorée. Ce comportement met en évidence la nécessité d'une revue attentive des politiques manuellement rédigées.

Vérification

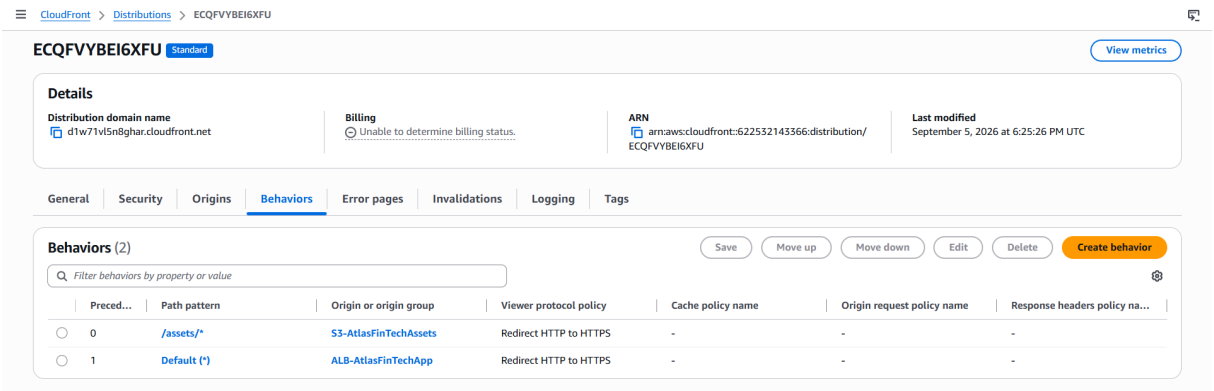


FIGURE 6.30 – Configuration des *cache behaviors* de la distribution, matérialisant le routage par chemin entre l'origine S3 (préfixe `/assets/*`) et l'origine ALB (comportement par défaut).

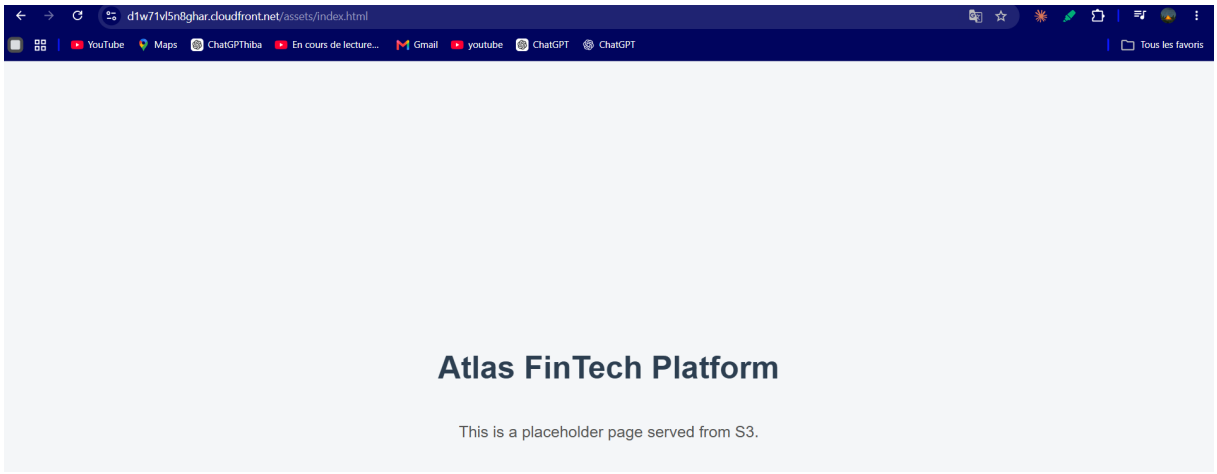


FIGURE 6.31 – Contenu servi depuis l'origine S3 via l'OAC, accessible à travers la distribution CloudFront.

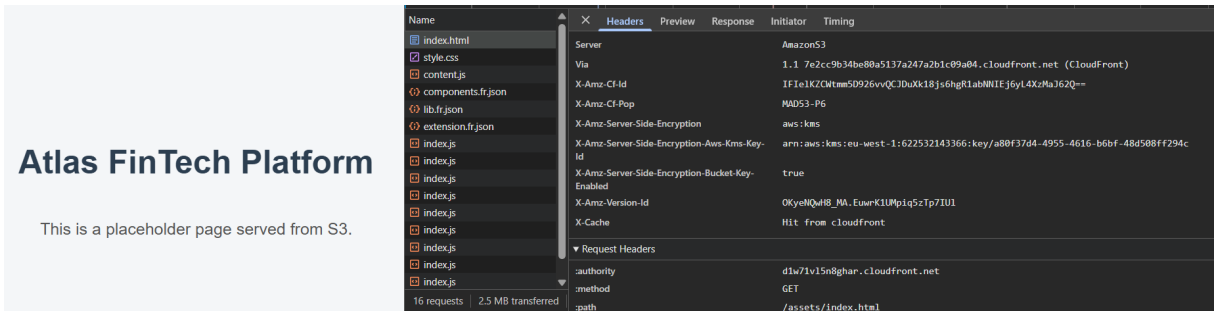


FIGURE 6.32 – En-têtes de réponse HTTP indiquant un `x-cache: Hit from cloudfront` depuis le point de présence de Madrid, confirmant le fonctionnement de la mise en cache et la couverture géographique européenne.

```
    "Sid": "AllowCloudFrontServicePricnipal",
    "Effect": "Allow",
    "Principal": {
      "Service": "cloudfront.amazonaws.com"
    },
    "Action": [
      "kms:Decrypt",
      "kms:GenerateDataKey*",
      "kms:DescribeKey"
    ],
    "Resource": "*",
    "Condition": {
      "StringEquals": {
        "AWS:SourceArn": "arn:aws:cloudfront::622532143366:distribution/ECQFVYBEI6XFU"
      }
    }
  }
```

FIGURE 6.33 – Cinquième énoncé ajouté à la *key policy* KMS pour autoriser CloudFront à déchiffrer les objets S3.

6.4.4 Phase 11 – Découplage asynchrone des événements de sécurité

Objectif

Cette dernière phase introduit un pipeline de traitement asynchrone déclenché par les événements produits par le service de détection GuardDuty déployé en Phase 02. Les constatations émises par GuardDuty transitent par EventBridge, sont mises en file d'attente dans Amazon SQS, puis consommées par une fonction Lambda de triage. Un mécanisme de rebasculement (*redrive policy*) associé à une file d'attente des messages non délivrés (*Dead Letter Queue*, ou DLQ) protège contre les échecs répétés de traitement.

Vérification par provocation d'un scénario d'erreur

Cette phase illustre une dernière fois le principe posé en introduction du chapitre. Un premier test simple, envoyant un message conforme au format attendu, confirme la chaîne nominale : le message est reçu par la file, consommé par la fonction Lambda, et son traitement se solde par un succès visible dans les journaux CloudWatch. Un second test, plus significatif, injecte un message volontairement mal formé (par exemple, un JSON invalide) et suit son parcours : la fonction Lambda tente trois fois de le traiter, échoue trois fois avec une exception visible dans les journaux, et le message est effectivement transféré vers la DLQ, où il devient récupérable pour investigation manuelle.

Cette vérification par provocation d'erreur constitue la démonstration la plus tangible que la politique de rebasculement fonctionne effectivement, et non simplement qu'elle a été correctement provisionnée.

Vérification

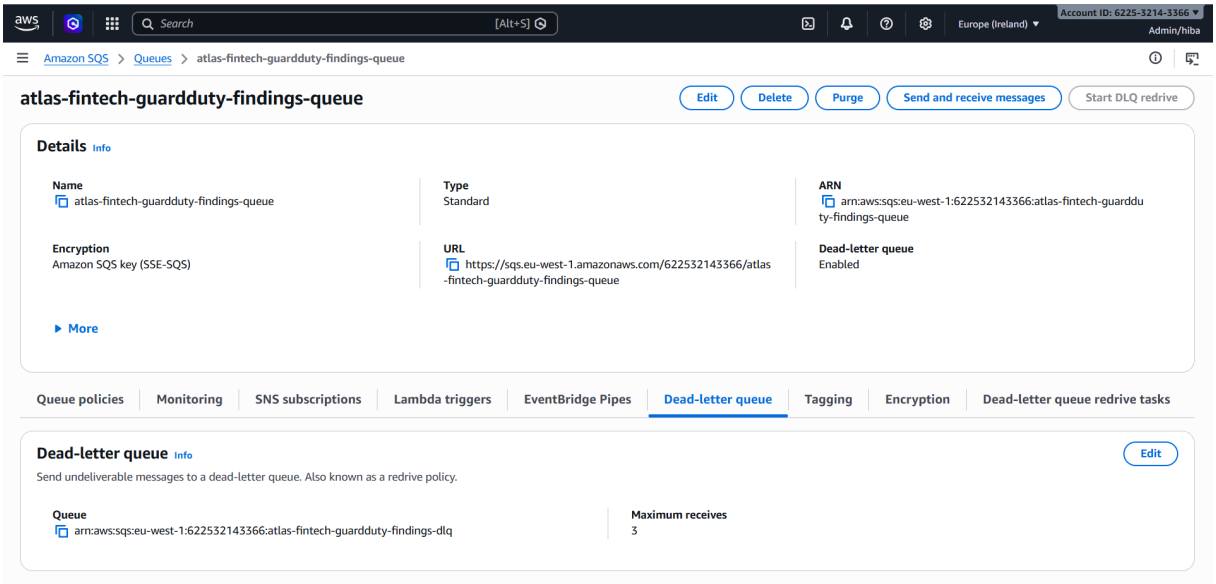


FIGURE 6.34 – Configuration *Dead-letter* de la file d'attente principale, matérialisant le lien avec la DLQ et la valeur du `maxReceiveCount`.

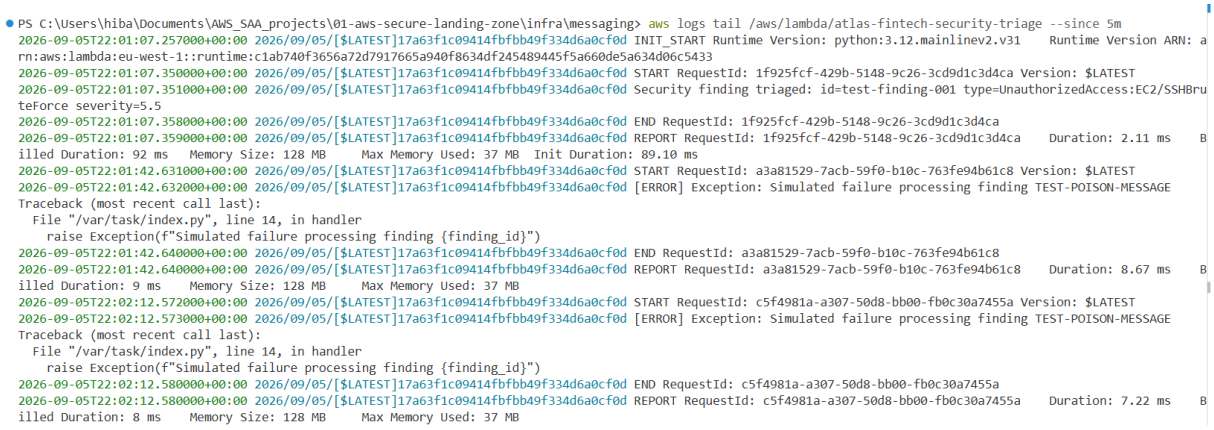


FIGURE 6.35 – Journaux CloudWatch de la fonction Lambda affichant successivement un traitement réussi et les tentatives infructueuses ayant conduit au rebasculement.

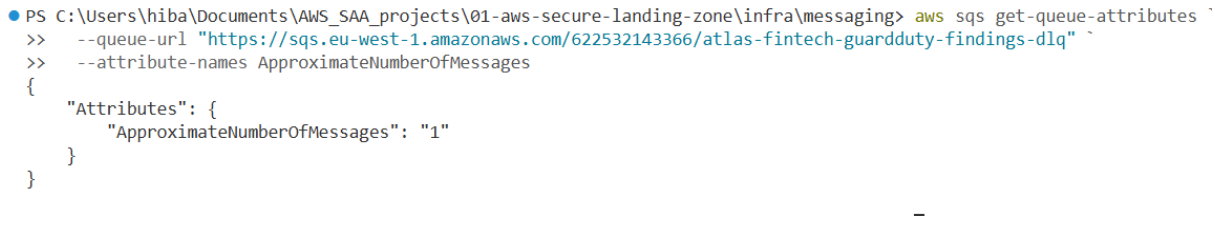


FIGURE 6.36 – Message volontairement mal formé retrouvé dans la DLQ après trois échecs de traitement, confirmant le fonctionnement effectif de la politique de rebasculement.

Bilan de la section

Ces quatre phases finales enrichissent la plateforme applicative de capacités structurantes qui n'appartenaient pas au périmètre initial : maîtrise financière assurée par un budget alerteur, stockage objet dimensionné selon la nature des contenus, distribution mondiale du contenu par un réseau de diffusion, et découplage asynchrone du traitement des événements de sécurité. Chacune de ces phases a été l'occasion de mobiliser à la fois les mécanismes déjà éprouvés (dossiers Terraform séparés, authentification cross-compte, condition `aws:SourceArn`) et de rencontrer de nouvelles subtilités propres au service considéré. Le chapitre suivant reviendra sur l'architecture d'ensemble ainsi produite et sur les choix de conception qui la sous-tendent.

Conclusion du chapitre

Ce chapitre a présenté la mise en œuvre concrète de la landing zone Atlas FinTech au fil de onze phases progressives, depuis les prérequis manuels initiaux jusqu'aux capacités additionnelles construites autour de la plateforme applicative. L'infrastructure couvre désormais l'ensemble des dimensions attendues d'une landing zone AWS moderne : gouvernance multi-comptes durcie par des SCPs, réseau à trois couches multi-AZ, plateforme applicative chiffrée de bout en bout, observabilité assortie d'alertes vérifiées, pipeline CI/CD authentifié par OIDC et bloquant sur vulnérabilités critiques, discipline budgétaire, ainsi que stockage, distribution de contenu et découplage asynchrone des événements de sécurité.

Plusieurs enseignements transverses se dégagent de cette mise en œuvre : la séparation stricte des dossiers Terraform et l'authentification cross-compte se sont révélées indispensables à mesure que l'architecture gagnait en composants interconnectés, et le principe de vérification systématique – y compris par provocation volontaire d'erreurs – s'est imposé comme le seul moyen fiable de distinguer une capacité opérationnelle d'une capacité simplement provisionnée. Les difficultés rencontrées, qu'il s'agisse de comportements non documentés, de dépendances implicites entre services ou d'évolutions imprévues chez un fournisseur tiers, ont chacune été résolues par élimination méthodique des hypothèses plutôt que par contournement.

Conclusion générale

Ce rapport a présenté la conception et la mise en œuvre d'une landing zone AWS sécurisée et automatisée, construite autour du cas d'usage fictif Atlas FinTech. La démarche a privilégié la construction depuis les composants primitifs d'AWS plutôt que le recours à un service géré de type Control Tower, afin de garantir une maîtrise complète des décisions d'architecture et de couvrir le périmètre de la certification AWS Solutions Architect Associate (SAA-C03). Le cheminement a suivi une progression descendante : cadrage et risques (chapitre 1), besoins et acteurs (chapitre 2), fondements théoriques (chapitre **??**), technologies retenues (chapitre 4), conception en cinq couches (chapitre 5), puis mise en œuvre vérifiée au fil de onze phases (chapitre 6).

La matrice de traçabilité (section 5.7) confirme que les besoins classés *Must have* et *Should have* (tableau 2.3) ont été implémentés et vérifiés : gouvernance multi-comptes par SCPs, fédération d'identité avec MFA, exposition sécurisée par ALB et CloudFront, chiffrement systématique par KMS, haute disponibilité multi-AZ, pipeline CI/CD authentifié par OIDC et bloquant sur vulnérabilités critiques, supervision continue des *findings* GuardDuty, et gouvernance financière par alertes budgétaires. Les sept risques identifiés (section 2.6) ont chacun trouvé une mitigation démontrée, parfois par provocation volontaire d'un échec – blocage réel du pipeline sur des vulnérabilités CRITICAL, rebasculement effectif vers la *Dead Letter Queue*. La réalisation présente néanmoins des écarts assumés : le quota implicite de cinq comptes par organisation (Phase 02) a limité le projet à quatre comptes membres, réseau et charge de travail cohabitant sans compte de journalisation dédié (tableau 6.1) ; les besoins *Could have* (domaine personnalisé, WAF, tableaux de bord enrichis) et *Won't have this time* (reprise après sinistre multi-régions, astreinte, charge de production réelle) n'ont pas été implémentés, conformément au périmètre académique fixé au chapitre 2.

Bibliographie

[1] AMAZON WEB SERVICES. *Amazon CloudFront Developer Guide*. URL : https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/Introduction.html (visité le 11/09/2026).

[2] AMAZON WEB SERVICES. *Amazon CloudWatch User Guide*. URL : https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/WhatIsCloudWatch.html (visité le 11/09/2026).

[3] AMAZON WEB SERVICES. *Amazon DynamoDB Developer Guide*. URL : https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Introduction.html (visité le 11/09/2026).

[4] AMAZON WEB SERVICES. *Amazon Elastic Container Registry User Guide*. URL : https://docs.aws.amazon.com/AmazonECR/latest/userguide/what-is-ecr.html (visité le 11/09/2026).

[5] AMAZON WEB SERVICES. *Amazon Elastic Container Service Developer Guide*. URL : https://docs.aws.amazon.com/AmazonECS/latest/developerguide/Welcome.html (visité le 11/09/2026).

[6] AMAZON WEB SERVICES. *Amazon EventBridge User Guide*. URL : https://docs.aws.amazon.com/eventbridge/latest/userguide/eb-what-is.html (visité le 11/09/2026).

[7] AMAZON WEB SERVICES. *Amazon GuardDuty User Guide*. URL : https://docs.aws.amazon.com/guardduty/latest/ug/what-is-guardduty.html (visité le 11/09/2026).

[8] AMAZON WEB SERVICES. *Amazon Route 53 Developer Guide*. URL : https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/Welcome.html (visité le 11/09/2026).

[9] AMAZON WEB SERVICES. *Amazon Simple Notification Service Developer Guide*. URL : https://docs.aws.amazon.com/sns/latest/dg/welcome.html (visité le 11/09/2026).

[10] AMAZON WEB SERVICES. *Amazon Simple Queue Service Developer Guide*. URL : https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/welcome.html (visité le 11/09/2026).

[11] AMAZON WEB SERVICES. *Amazon Simple Storage Service User Guide*. URL : https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html (visité le 11/09/2026).

[12] AMAZON WEB SERVICES. *Amazon VPC User Guide*. URL : https://docs.aws.amazon.com/vpc/latest/userguide/what-is-amazon-vpc.html (visité le 11/09/2026).

[13] AMAZON WEB SERVICES. *Application Load Balancers – User Guide*. URL : https://docs.aws.amazon.com/elasticloadbalancing/latest/application/introduction.html (visité le 11/09/2026).

[14] AMAZON WEB SERVICES. *AWS Budgets User Guide*. URL : https://docs.aws.amazon.com/cost-management/latest/userguide/budgets-managing-costs.html (visité le 11/09/2026).

[15] AMAZON WEB SERVICES. *AWS Certificate Manager User Guide*. URL : https://docs.aws.amazon.com/acm/latest/userguide/acm-overview.html (visité le 11/09/2026).

[16] AMAZON WEB SERVICES. *AWS Config Developer Guide*. URL : https://docs.aws.amazon.com/config/latest/developerguide/WhatIsConfig.html (visité le 11/09/2026).

[17] AMAZON WEB SERVICES. *AWS Fargate*. URL : https://docs.aws.amazon.com/AmazonECS/latest/developerguide/AWS_Fargate.html (visité le 11/09/2026).

[18] AMAZON WEB SERVICES. *AWS IAM Identity Center User Guide*. URL : https://docs.aws.amazon.com/singlesignon/latest/userguide/what-is.html (visité le 11/09/2026).

[19] AMAZON WEB SERVICES. *AWS Lambda Developer Guide*. URL : https://docs.aws.amazon.com/lambda/latest/dg/welcome.html (visité le 11/09/2026).

[20] AMAZON WEB SERVICES. *AWS Organizations User Guide*. URL : https://docs.aws.amazon.com/organizations/latest/userguide/orgs_introduction.html (visité le 11/09/2026).

[21] AMAZON WEB SERVICES. *AWS Security Hub User Guide*. URL : https://docs.aws.amazon.com/securityhub/latest/userguide/what-is-securityhub.html (visité le 11/09/2026).

[22] AMAZON WEB SERVICES. *AWS Well-Architected Framework*. URL : https://docs.aws.amazon.com/wellarchitected/latest/framework/welcome.html (visité le 11/09/2026).

[23] AMAZON WEB SERVICES. *Control traffic to subnets using network ACLs*. URL : https://docs.aws.amazon.com/vpc/latest/userguide/vpc-network-acls.html (visité le 11/09/2026).

[24] AMAZON WEB SERVICES. *Control traffic to your AWS resources using security groups*. URL : https://docs.aws.amazon.com/vpc/latest/userguide/vpc-security-groups.html (visité le 11/09/2026).

[25] AMAZON WEB SERVICES. *Creating a trail for an organization*. URL : https://docs.aws.amazon.com/awscloudtrail/latest/userguide/creating-trail-organization.html (visité le 11/09/2026).

[26] AMAZON WEB SERVICES. *Disaster Recovery of Workloads on AWS : Recovery in the Cloud*. URL : https://docs.aws.amazon.com/whitepapers/latest/disaster-recovery-workloads-on-aws/disaster-recovery-workloads-on-aws.html (visité le 11/09/2026).

[27] AMAZON WEB SERVICES. *Landing zone design considerations – AWS Prescriptive Guidance*. URL : https://docs.aws.amazon.com/prescriptive-guidance/latest/migration-aws-environment/landing-zone-considerations.html (visité le 11/09/2026).

[28] AMAZON WEB SERVICES. *NAT gateways*. URL : https://docs.aws.amazon.com/vpc/latest/userguide/vpc-nat-gateway.html (visité le 11/09/2026).

[29] AMAZON WEB SERVICES. *Organizing your AWS environment using multiple accounts*. URL : https://docs.aws.amazon.com/whitepapers/latest/organizing-your-aws-environment/organizing-your-aws-environment.html (visité le 11/09/2026).

[30] AMAZON WEB SERVICES. *Reliability Pillar – AWS Well-Architected Framework*. URL : https://docs.aws.amazon.com/wellarchitected/latest/reliability-pillar/welcome.html (visité le 11/09/2026).

[31] AMAZON WEB SERVICES. *Security best practices in IAM – Grant least privilege*. URL : https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html (visité le 11/09/2026).

[32] AMAZON WEB SERVICES. *Security Pillar – AWS Well-Architected Framework*. URL : https://docs.aws.amazon.com/wellarchitected/latest/security-pillar/welcome.html (visité le 11/09/2026).

[33] AMAZON WEB SERVICES. *Service Control Policies (SCPs)*. URL : https://docs.aws.amazon.com/organizations/latest/userguide/orgs_manage_policies_scps.html (visité le 11/09/2026).

[34] AMAZON WEB SERVICES. *Using cost allocation tags*. URL : https://docs.aws.amazon.com/cost-management/latest/userguide/cost-alloc-tags.html (visité le 11/09/2026).

[35] HASHICORP. *AWS Provider – Terraform Registry*. URL : https://registry.terraform.io/providers/hashicorp/aws/latest/docs (visité le 11/09/2026).

[36] HASHICORP. *Backend Type : s3*. URL : https://developer.hashicorp.com/terraform/language/backend/s3 (visité le 11/09/2026).

[37] HASHICORP. *The Remote Backend and Remote State Data Source*. URL : https://developer.hashicorp.com/terraform/language/state/remote-state-data (visité le 11/09/2026).